

OPERATIONS · DEVELOPMENT · RELEASE

Furive OS 통합 사용 설명서

설치, 일상 운영, 독립 패키지 연결, 차량 플랫폼, 검증과 OTA를
한 권의 실제 작업 흐름으로 정리한 공식 편집본

EDITION
PUBLISHER
FORMAT

2026-09-11
FutureDrive
A4 · Korean · Offline PDF

ABOUT THIS MANUAL

이 책은 명령 목록이 아니라 작업을 끝내는 순서입니다

Furive OS의 공개 개념을 더 만들지 않고, 현재 문서가 소유하는 내용을 독자의 실제 작업 순서로 다시 편집했습니다. 1-8장은 그대로 따라 하는 가이드이고, 9-17장은 문제가 생기거나 내부 계약을 바꿀 때만 여는 참고편입니다.

문서 기준일	2026-09-11
발행	FutureDrive
구성	5개 실제 UI 해설 · 8개 작업 가이드 · 9개 상세 참고 · 빠른 명령표 · 장애 판단표 · 색인
읽는 법	처음에는 역할별 기본 경로만 따라가고, 문제의 소유자를 찾을 때 필요한 reference 장을 엽니다.

01 · DEFAULT FIRST

기본 경로를 먼저

자동으로 되는 일을 수동 단계로 늘리지 않고, 예외와 내부 명령은 뒤로 보냅니다.

02 · ONE OWNER

상태 소유자를 한 곳에

Dashboard는 보여주고 전달하며, package·OTA·Autoware 상태를 새로 소유하지 않습니다.

03 · PUBLIC LANGUAGE

main·edge·normal·test

사람이 쓰는 공개 용어를 먼저 쓰고 내부 호환 이름은 필요한 reference에서만 표시합니다.

04 · EVIDENCE BY STAGE

단계에 맞는 검증

일상 개발, 한 차량 test, 고객용 normal을 구분하고 실행하지 않은 검사는 PASS로 쓰지 않습니다.

CHOOSE YOUR ROUTE

내 역할에 필요한 장만 읽으세요

처음부터 끝까지 정독할 필요가 없습니다. 아래 경로의 앞쪽 장을 먼저 수행하고, 막힌 경계의 reference만 이어서 찾는 것이 가장 빠릅니다.

차량 설치자 · 운전자

받은 설치 묶음을 적용하고 운영

설치 뒤에는 역할을 다시 입력하지 않습니다. 정상일 때는 자동 실행에 맡깁니다.

01 → 02(p.22) → 03(p.23)

독립 패키지 개발자

프로그램을 Furive에 연결

기능은 원래 저장소에서 완성하고 실행·health·로그 계약만 연결합니다.

01 → 04 → 05(p.37) → 07

플랫폼 · 릴리즈 개발자

차량 한 대에서 시험하고 승격

플랫폼 구성과 Autoware revision을 고정한 뒤 test·복구·normal 순서를 지킵니다.

06 → 07 → 08(p.47) → 16

장애 조사 · 공통 기반 유지보수

소유자와 host 영향을 추적

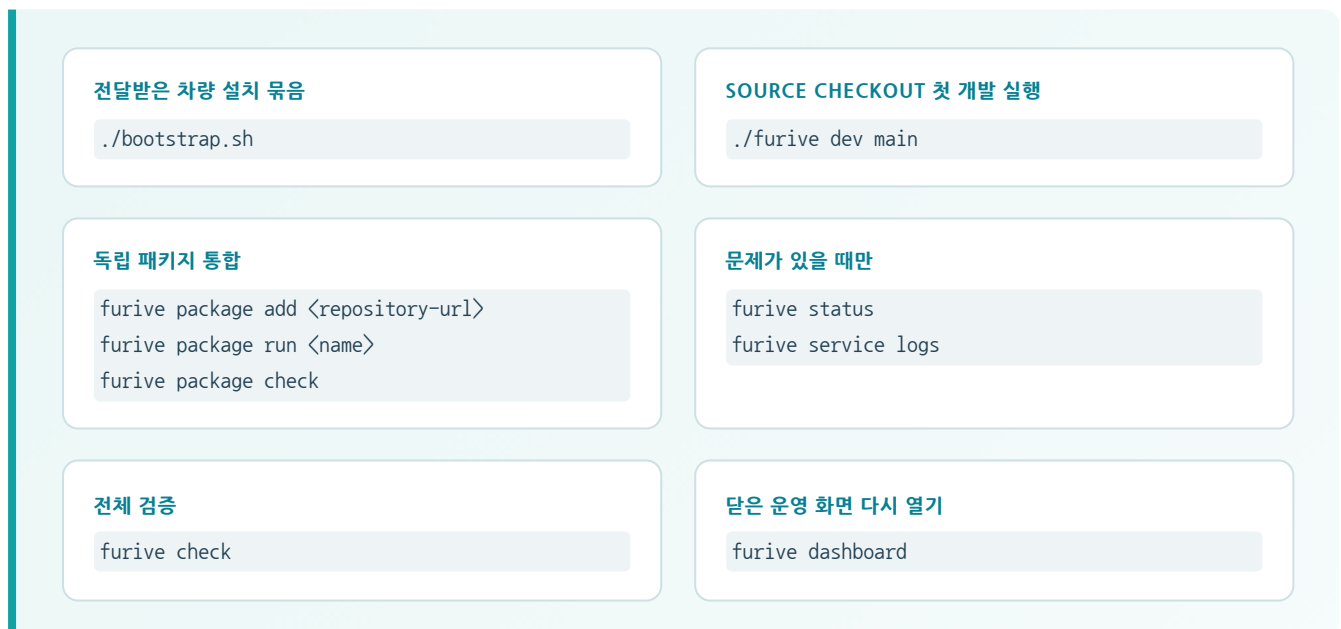
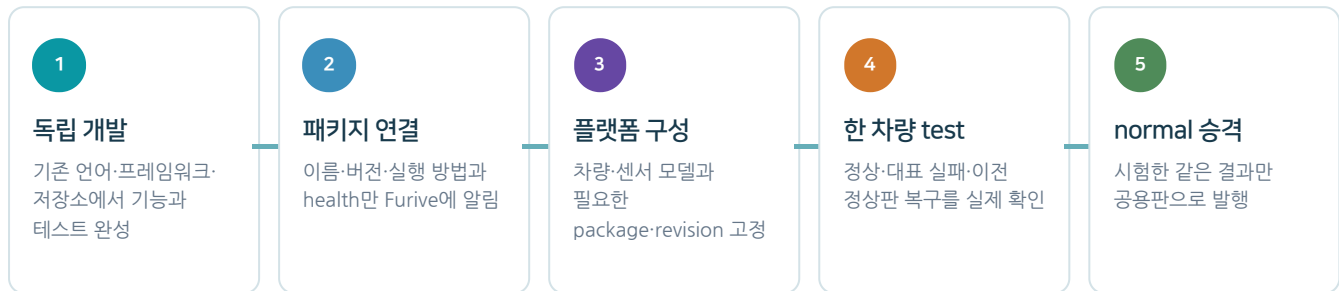
먼저 상태와 로그로 범위를 좁히고 해당 계약 한 장만 엽니다.

03 → 09 → 10 · 11 → 관련 reference

FIVE-MINUTE START

가장 짧은 작업 경로

차량 설치자는 받은 묶음에서 한 명령만 실행합니다. source checkout의 첫 개발 실행만 역할을 적고, 설치된 PC는 저장된 역할을 자동으로 사용합니다.



이 여덟 단어만 먼저 기억하세요

PC 한 대로 시작할 수 있습니다. 분산 엣지 컴퓨팅은 자율주행과 부가 기능의 부하를 나누는 권장 구성입니다. 기능 배정은 플랫폼을 따릅니다.

1PC

자율주행·운영 화면과 배정된 HMI·로깅 등 부가 패키지를 한 PC에서 함께 실행

한 설치 · 명시적인 패키지 배정

분산 구성

A-ECU는 자율주행·기본 운영, HMI-ECU는 별도 PC에서 로깅·부가 기능을 실행

부하와 장애 영향 분리

패키지

독립 프로그램을 발견·실행·health·로그·OTA 계약에 연결한 운용 단위

package/core · package/plugin

플랫폼

차량·센서 모델, package 조합과 고정 Autoware revision을 공유하는 구성

config only: capability_profile

배포 버전

차량이 실제 실행할 코드·패키지·이미지·Autoware revision을 고정한 signed release

release

설치 묶음

기존 배포 버전에 차량별 접근 정보·지도·전달 파일을 더한 설치 ZIP

bundle

normal

차량 시험을 통과해 일반 차량이 받는 공용 업데이트 종류

internal channel: main

test

지정한 차량 한 대에서만 임시 시험하는 업데이트 종류

internal channel: preview/<vehicle_id>

실제 작업

1-8장은 작업을 수행하고 완료 조건을 확인하는 순서입니다.

UI	운영 화면 해설	10 - 14
	실제 main 화면 5종의 요소·동작·주의점을 번호로 설명	
01	 Furive OS 한눈에 보기	16
	독립 프로그램에서 차량 실행과 OTA까지 이어지는 전체 흐름	
02	 차량 PC 설치	22
	차량별 설치 묶음을 한 명령으로 적용하는 순서	
03	 Furive OS 기본 사용법	23
	설치 뒤 화면·설정·상태·로그·서비스를 다루는 기본 절차	
04	 Furive OS 개발 시작	34
	수정할 저장소를 고르고 가장 가까운 테스트부터 시작하는 방법	
05	 프로그램을 Furive에 연결하기	37
	독립 프로그램을 발견·실행·health·로그 계약에 연결하는 방법	
06	차량 플랫폼 만들기	40
	차량·센서 차이와 Autoware revision을 플랫폼 경계에 두는 방법	
07	변경 검증	44
	변경이 도달한 단계만 정확히 검증하고 기록하는 기준	
08	 차량 업데이트	47
	한 차량 test·복구 뒤 같은 변경을 normal로 승격하는 절차	

상세 계약

9-17장은 기본 경로에서 막혔거나 내부 소유권을 바꿀 때만 엽니다.

09	설치가 PC에 미치는 영향	52
	설치와 선택 패키지가 PC 전체에 줄 수 있는 영향을 조건별로 확인	
10	Furive OS 기술 구조	55
	상태·정책의 단일 소유자와 실행·통신·업데이트 내부 흐름	
11	실행 구성요소	69
	Dashboard·Package Manager·Launcher·OTA와 통신 구성요소의 경계	
12	프로그램 연결 설정 (package.yaml)	77
	package.yaml의 기본값·권한·lifecycle·health·테스트 상세 계약	
13	Autoware 플랫폼 탐색	87
	Autoware 차량·센서 모델 쌍의 저장·지정·자동 탐색 순서	
14	이전 정상 버전 복구 기준	90
	후보 준비·폴더 교환·확정 실패·직전 정상판 복구의 상태 계약	
15	업데이트 호환판과 구버전 설정 변환	95
	오래된 차량이 검증된 호환판과 설정 변환을 거쳐 목표에 도달하는 규칙	
16	CI/CD 기준	98
	배포 영향 판단부터 image·SBOM·provenance·서명·발행까지의 자동화	
17	보안·개인정보 기준	122
	접근 정보·데이터·차량 제어·공급망의 단계별 최소 기준	

COMMAND QUICK REFERENCE

공개 명령 빠른 표

사람과 LLM이 직접 호출하는 source 저장소의 공개 진입점은 `./furive` 하나입니다. 설치 묶음만 그 안의 `./bootstrap.sh` 로 시작합니다.

상황	명령	기대 결과
새 차량 PC 설치	<code>./bootstrap.sh</code>	묶음이 가진 역할·차량·플랫폼·버전으로 설치하고 서비스를 시작
source 첫 개발 실행	<code>./furive dev main</code>	설치 역할이 없는 checkout에서 main 개발 세션 시작
설치된 PC 개발 실행	<code>furive dev</code>	설치 상태가 가진 역할을 자동 사용
운영 화면 다시 열기	<code>fui</code>	닫힌 main 화면을 다시 열기
전체 상태 확인	<code>furive status</code>	역할·버전·핵심 프로그램 상태 요약
서비스 로그 확인	<code>furive service logs</code>	시작·중지·오류 경계를 시간 순서로 확인
패키지 연결	<code>furive package add <url></code>	독립 저장소를 Furive package 경계에 연결
패키지 단독 실행	<code>furive package run <name></code>	의존 패키지와 함께 실행하고 Ctrl+C로 함께 종료
패키지 계약 검사	<code>furive package check</code>	manifest·entrypoint·package 자체 테스트 확인
전체 저장소 검증	<code>furive check</code>	코드·문서·package·배포 기본 계약 검사
정비 중 서비스 재시작	<code>furive service restart</code>	설치 역할의 전체 서비스를 안전하게 다시 시작
차량 배경 확인	<code>./furive vehicle show <id></code>	플랫폼·package·업데이트 종류·버전 확인

TROUBLESHOOTING FIRST RESPONSE

문제를 넓히지 말고 이 순서로 좁히세요

정상 동작을 복구한다는 이유로 알려지지 않은 port owner나 process를 종료하지 않습니다. 먼저 관찰하고, 해당 소유자의 명령으로만 조치합니다.

1 · 전체 상태 furive status 역할·버전·main/edge 연결·필수 package 범위를 확인	2 · 서비스 로그 furive service logs 시작 실패인지 실행 뒤 기능 실패인지 분리	3 · 해당 소유자 package 로그·Launcher·OTA 화면 중 실제 상태를 가진 한 곳을 확인	4 · 복구 경로 정비 상태에서 restart, 업데이트 직후 문제면 이전 정상판 복구
---	---	--	--

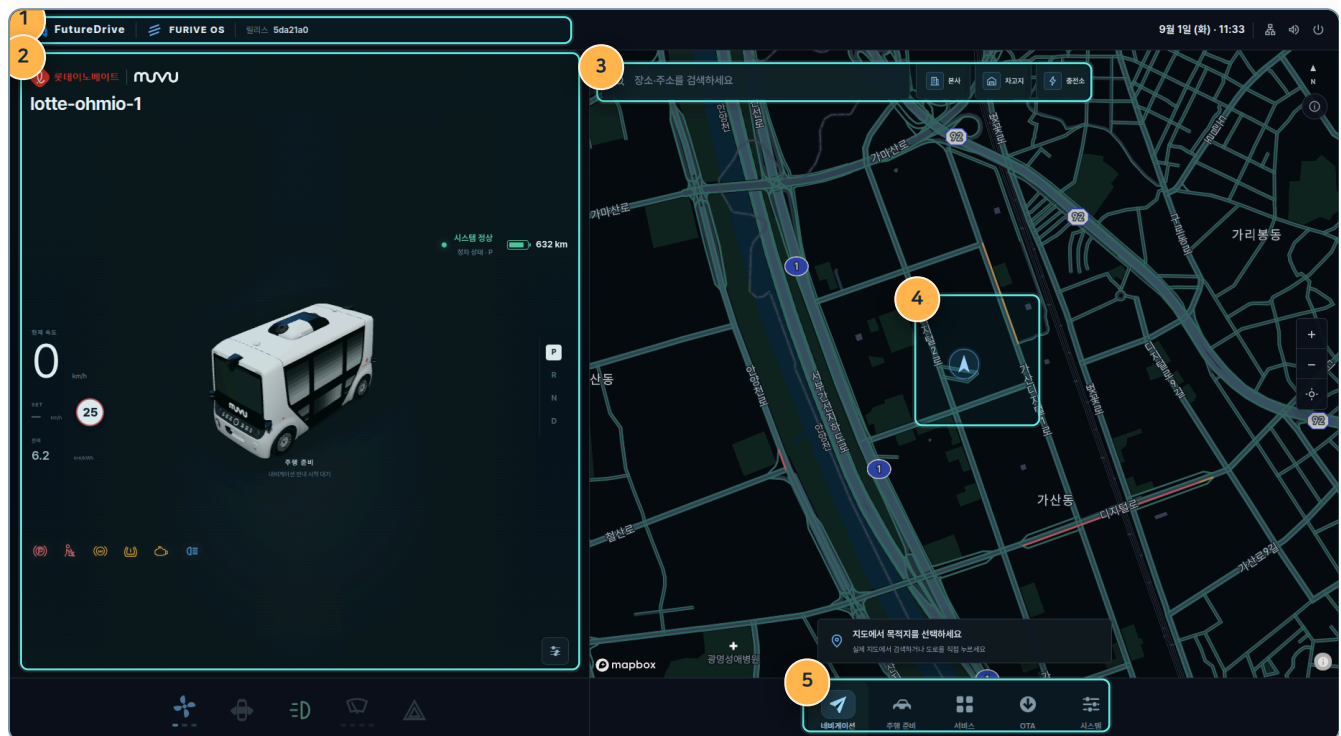
보이는 현상	먼저 확인	하지 않을 것
main 화면이 닫힘	fui ; 서비스가 정상인지 상태 확인	Dashboard를 별도 상태 소유자로 복제
패키지 하나가 unhealthy	정규화된 package 로그와 manifest health 조건	패키지 자체 재시작 loop 추가
Autoware가 시작되지 않음	Launcher의 모델 쌍·지도·고정 workspace 상태	Furive 플랫폼명과 Autoware 모델명 강제 일치
업데이트 뒤 필수 기능 실패	목표/현재 버전, 후보 상태, 이전 정상판 복구 가능 여부	실행 폴더에서 직접 git pull
네트워크·CAN·장치 이상	9장 host 영향의 조건·실제 명령·확인 방법	owner를 확인하지 않은 process·port 종료
개발 변경이 차량에 안 보임	release 영향인지 bundle 영향인지, 차량 assignment	deploy 저장소 직접 수정

SCREEN 01 / USER INTERFACE

내비게이션 · 공통 화면 구조

실제 main 개발 화면

지도와 차량 상태를 함께 보여 목적지를 고르는 화면입니다. 이 페이지의 1·2·5번은 다른 화면에서도 계속 보이는 공통 영역입니다.



1

상단 상태 바

FutureDrive·Furive OS 식별, 현재 release, 시간과 연결·오디오·전원 바로가기를 보여줍니다.

2

차량 HMI

차량 이름, 시스템 상태, 속도·주행 가능 거리·기어와 경고등을 표시합니다. 화면 표시만으로 출발 준비가 보증되지는 않습니다.

3

목적지와 빠른 검색

주소·장소를 검색하거나 본사·차고지·충전소 같은 자주 쓰는 목적지를 선택합니다.

4

지도와 현재 위치

현재 위치, 축척, 방위와 선택한 경로를 표시합니다. 지도 package와 위치 데이터 상태에 따라 내용이 달라집니다.

5

하단 화면 전환

내비게이션·주행 준비·서비스·OTA·시스템 사이를 이동합니다. 탭 전환 자체는 runtime 상태를 바꾸지 않습니다.

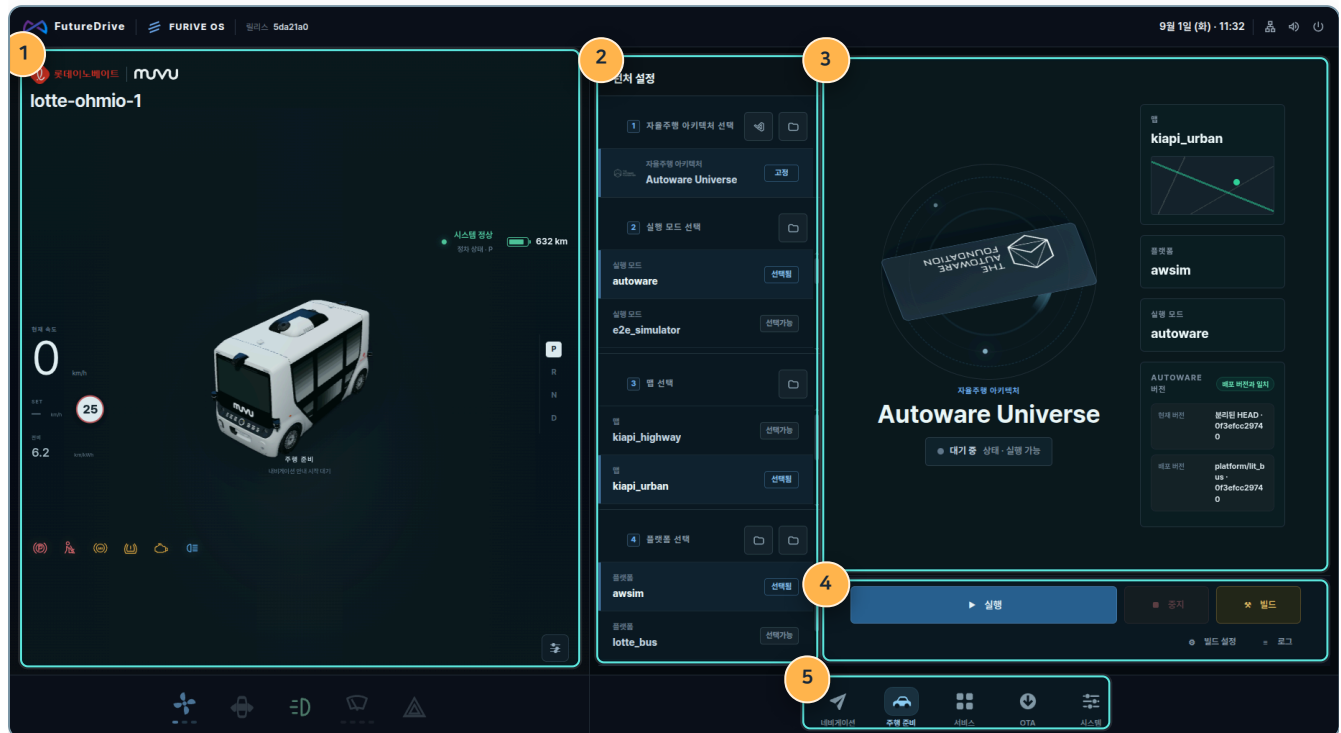
읽을 때 주의 실제 차량 운행 판단은 센서·차량 제어 상태와 현장 안전 절차를 함께 확인해야 합니다. 지도 화면만 보고 차량을 출발시키지 않습니다.

SCREEN 02 / USER INTERFACE

주행 준비

실제 main 개발 화면

차량 플랫폼에 맞는 Autware 구성과 실행 모드·지도·플랫폼을 확인한 뒤 Launcher lifecycle을 시작하는 화면입니다.



1 차량 상태 영역

Launcher 선택과 별개로 차량 측 관측값을 보여줍니다. '시스템 정상'과 Autware '실행 가능'은 서로 다른 확인입니다.

2 런처 설정 단계

자율주행 아키텍처, 실행 모드, 지도와 플랫폼을 순서대로 선택합니다. 고정된 값은 차량 assignment와 플랫폼 계약이 소유합니다.

3 선택 결과 요약

현재 선택한 지도·플랫폼·실행 모드와 Autware 현재/배포 revision을 한곳에서 대조합니다.

4 실행·중지·빌드·로그

실행과 중지는 Furive Launcher에 전달됩니다. 빌드는 workspace를 바꾸므로 정비 상태에서만 사용하고, 문제는 먼저 로그로 줍니다.

5 현재 화면

차량 아이콘이 강조되면 주행 준비 화면입니다. 다른 탭으로 이동해도 실행 중인 lifecycle은 자동으로 멈추지 않습니다.

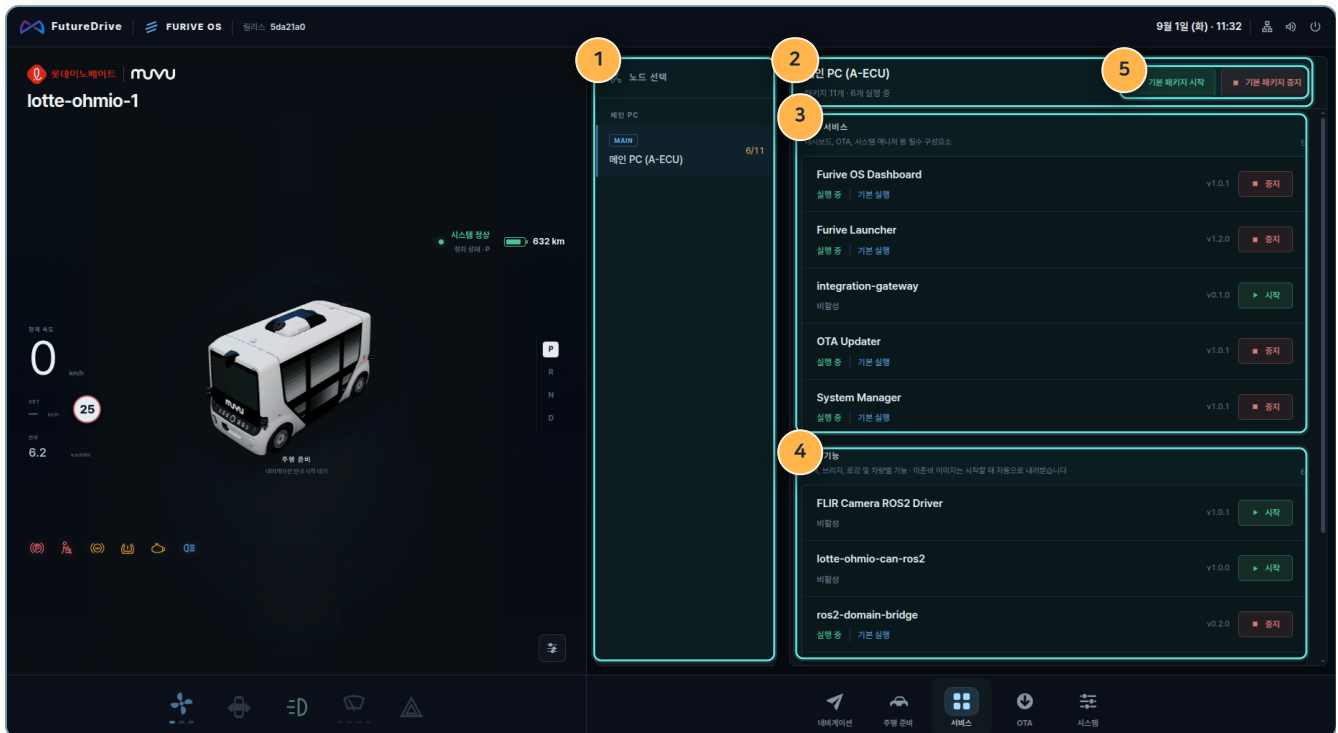
읽을 때 주의 실행 버튼을 누르기 전에 지도·플랫폼·revision과 차량의 정비/운행 상태를 확인합니다. 이 설명서 촬영 중에는 실행·중지·빌드를 누르지 않았습니다.

SCREEN 03 / USER INTERFACE

서비스

실제 main 개발 화면

main과 연결된 edge를 고르고, 그 노드에서 발견된 core·차량 기능 package의 실행 상태와 버전을 확인하는 화면입니다.



1 노드 선택

현재 main과 연결된 edge를 전환합니다. 각 노드 옆 숫자는 실행 중 package 수와 발견된 전체 수의 요약입니다.

2 선택 노드 요약

선택한 PC 역할과 package 실행 개수를 보여줍니다. 화면의 MAIN 표시는 내부 master 호환 이름이 아니라 사용자 역할 main을 뜻합니다.

3 코어 서비스

Dashboard, Launcher, OTA Updater, System Manager처럼 공통 운영에 필요한 package의 상태·버전·기본 실행 여부를 봅니다.

4 차량 기능 package

센서·브리지·로깅 등 플랫폼이 선택한 기능입니다. 비활성은 곧 오류가 아니며 default_start와 운용 조건을 함께 봅니다.

5 기본 package 일괄 동작

manifest가 기본 실행으로 정한 package만 시작·중지합니다. 개별 오류 원인을 찾기 전에 무조건 일괄 재시작하지 않습니다.

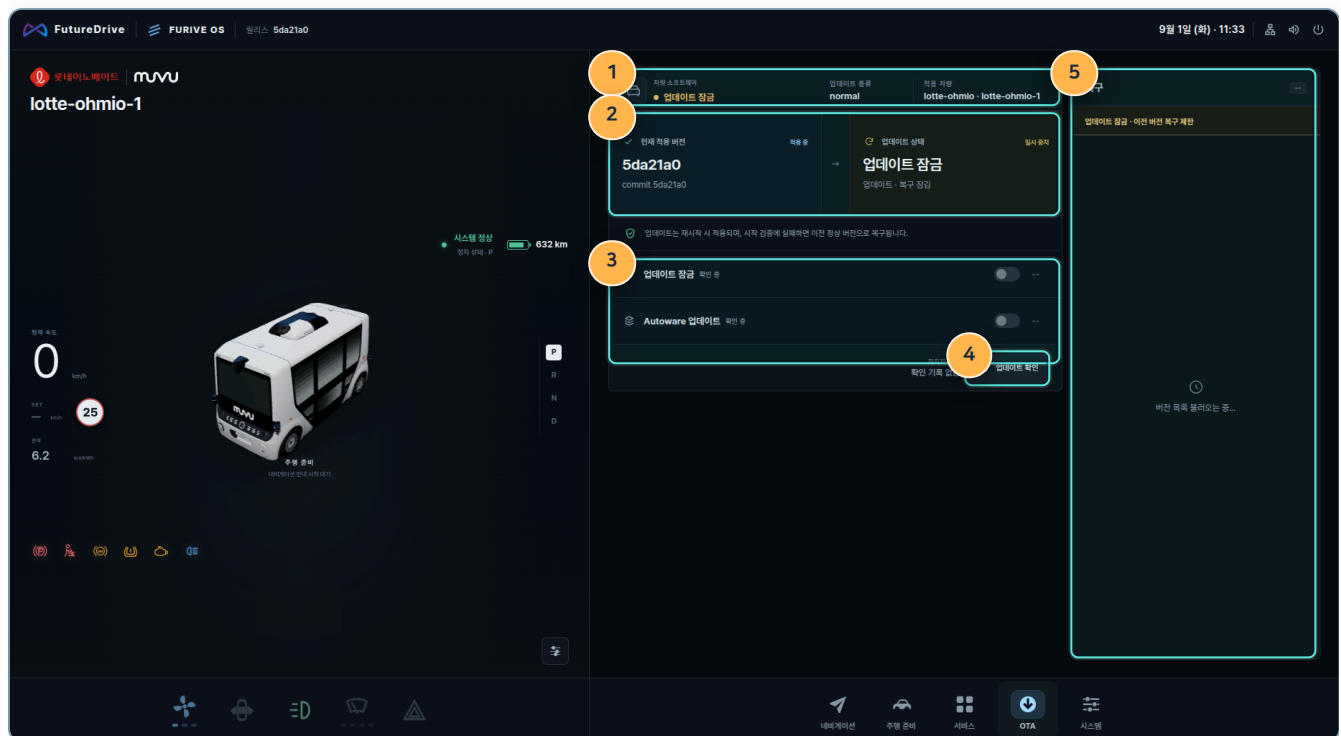
읽을 때 주의 Dashboard는 package 상태를 표시하고 명령을 전달합니다. 발견·실행·중지·health·로그의 실제 소유자는 root Package Manager입니다.

SCREEN 04 / USER INTERFACE

OTA

실제 main 개발 화면

적용 차량과 업데이트 종류, 현재/목표 버전, 선택 구성요소와 이전 정상판 복구 상태를 확인하는 화면입니다.



1

업데이트 대상

normal 또는 차량 한 대의 test 종류와 적용 차량을 먼저 확인합니다. 표시가 의도한 assignment와 다르면 진행하지 않습니다.

2

현재 버전과 상태

현재 적용 revision, 목표 버전과 준비·적용·잠금 상태를 보여줍니다. 재시작 전까지 현재 버전은 계속 실행됩니다.

3

업데이트 구성요소

Furive OS와 Autware 중 이번 배포가 포함된 대상을 확인합니다. 임의의 source pull 대신 signed release를 적용합니다.

4

업데이트 확인

차량 assignment와 호환되는 최신 후보를 다시 조회합니다. 조회 결과를 확인한 후에만 적용 단계로 이동합니다.

5

복구

직전 정상 버전과 복구 가능 상태를 보여줍니다. 적용 후 필수 기능이나 시작 검증이 실패하면 OTA 소유자의 복구 경로를 사용합니다.

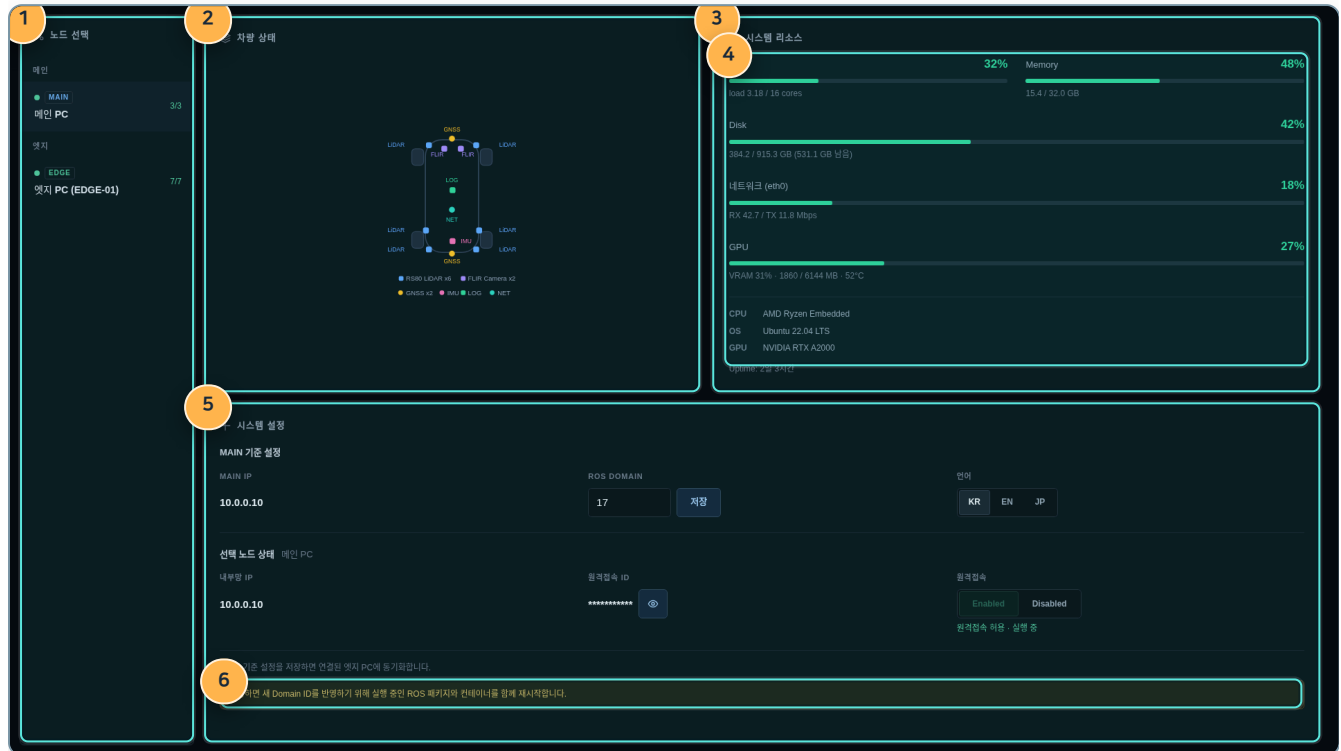
읽을 때 주의 이 스크린샷은 ./furive dev main 세션이라 OTA 보호가 활성화되어 '업데이트 잠금'으로 보이는 정상 예시입니다. 개발 중 보호를 억지로 해제해 업데이트하지 않습니다.

SCREEN 05 / USER INTERFACE

시스템 · System Manager

실제 main 개발 화면

MAIN과 edge의 연결, package 기반 차량 장치 요약, host 자원과 MAIN 기준 설정을 한 화면에서 확인하는 System Manager 화면입니다.

**1 노드 선택**

현재 MAIN 또는 edge의 연결 상태와 실행 중 전체 package 수를 확인합니다.

2 차량 장치 요약

관련 package 상태를 센서·LOG·NET 배치로 요약합니다. 녹색만으로 센서 데이터 품질까지 정상이라고 판단하지 않습니다.

3 시스템 리소스

CPU·memory·disk·network·GPU를 현재 선택한 node 기준으로 표시합니다.

4 자원 상세값

load, 사용량, RX/TX, VRAM·온도와 host 정보를 함께 읽어 지속 경고의 범위를 좁힙니다.

5 시스템 설정

MAIN 기준 IP·ROS Domain과 선택 node의 IP·원격접속 상태를 구분해서 확인합니다.

6 변경 영향

ROS Domain 저장에 관련 ROS package와 container를 재시작하므로 정비 상태에서만 실행합니다.

읽을 때 주의 System Manager는 host 자원과 설정을 표시·입력하는 adapter입니다. package 실행은 Package Manager, Autoware lifecycle은 Launcher, 업데이트는 OTA Updater에서 확인합니다.

PART I

먼저 이해하기

설치나 개발 전에 외울 구조가 아니라, 이후 절차가 왜 그렇게 짧은지 이해하는 한 장짜리 지도입니다.

01  Furive OS 한눈에 보기 · p.16

01. Furive OS 한눈에 보기

이 장의 목적 · 모든 독자

패키지·플랫폼·배포 버전·설치 묶음의 관계를 한 흐름으로 이해합니다.

완료 판단

1PC·분산 구성과 release/bundle의 차이를 짧게 설명할 수 있습니다.

각 팀은 프로그램을 개발하고, Furive OS는 지원 차량에서 그 프로그램의 설치·실행·업데이트를 맡습니다. 이 문서는 그 관계가 궁금할 때 읽는 자료입니다. 바로 코드를 고치려면 [개발 시작](#), 차량에 설치하려면 [차량 PC 설치](#)를 봅니다.

처음 알아둘 세 가지

이름	뜻	예
패키지	독립 프로그램에 Furive가 읽을 이름·버전·실행 방법을 붙인 운용 단위	센서 드라이버, 녹화 프로그램, 차량 화면
플랫폼	차량·센서 모델과 사용할 패키지의 조합	같은 구성을 쓰는 차량들이 공유하는 설정
설치할 PC	플랫폼이 정한 장비와 기능 배정	한 PC에 함께 설치하거나 A-ECU·HMI-ECU로 분산

각 팀은 기존 언어·프레임워크·저장소를 그대로 사용합니다. 일반 패키지는 `package.yaml`의 이름과 버전, 실행할 `run.sh`로 연결합니다. 기본값은 부가 기능 역할에서 실행하고 프로세스 상태를 확인하는 것입니다. 추가 요구가 생길 때만 연결 설정을 늘립니다. 실제 절차는 [프로그램 연결](#)에 있습니다.

Autoware 소스와 모델은 별도 작업 공간에서 개발합니다. Furive OS는 플랫폼별로 사용할 소스 버전과 실행 구성을 연결합니다. 새 차량·센서는 [차량 플랫폼 만들기](#)를 따릅니다. 같은 구성을 지원할 때 installer나 updater를 복사하지 않습니다.

PC 한 대로 시작하고 필요하면 분산하기

Furive OS는 PC 한 대에서도 사용할 수 있습니다. 한 PC에 Autoware·운영 화면과 필요한 HMI·로깅 등 부가 패키지를 함께 배정할 수 있습니다. A-ECU·HMI-ECU 분산 엣지 컴퓨팅은 연산·저장 부하와 장애 영향을 나누기 위한 권장 구성입니다.

PC 한 대부터, 필요하면 분산

플랫폼이 차량 · 센서 · 패키지와 실행 PC를 배정

1PC 한 PC에 함께 설치

Autoware · 운영 화면
자율주행 · 기본 운영

HMI · DLS 등 부가 패키지
함께 배정한 기능

한 설치 · 공통 실행 관리 · 로컬 ROS/DDS와 메시지

분산 엣지 컴퓨팅

자율주행과 부가 기능의 부하 분리 권장

A-ECU

Autoware · 기본 운영
자율주행 PC

HMI-ECU

데이터 로깅 · 부가 기능
부하를 분담하는 별도 PC



PC 간 연결: 등록 · 상태 · 운용 요청 / ROS 데이터 전달

두 구성 모두 공통 설치 · 실행 · 상태 · 업데이트와 복구

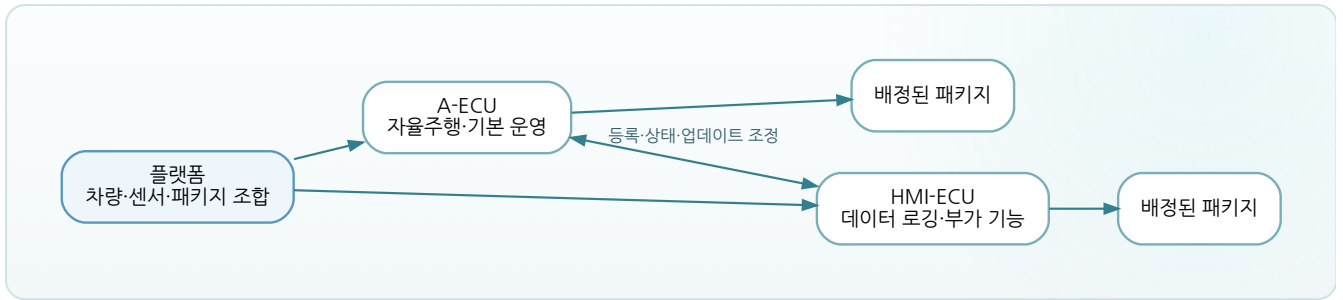
구성	패키지 실행 위치	준비할 것
1PC	한 PC에서 자율주행·기본 운영과 배정된 부가 패키지를 함께 실행	사용할 기능을 포함한 1PC 플랫폼 배정, CPU·GPU·메모리·저장공간과 장치
분산 구성	A-ECU는 자율주행·기본 운영, HMI-ECU는 로깅·부가 기능	각 PC의 배정과 인증된 PC 연결, 필요한 ROS 전달 경로

1PC는 한 설치 안에서 필요한 패키지를 함께 실행하는 구성입니다. 같은 PC에 A-ECU와 HMI-ECU 설치를 차례로 덧씌우는 절차가 아닙니다. 현재 차량용 기본 배정은 두 ECU로 나뉘어 있으므로 A-ECU만 설치하면 그쪽에 배정된 기능만 실행됩니다. HMI·DLS까지 함께 쓸 구성은 플랫폼 담당자가 준비합니다. 분산 구성의 HMI-ECU가 꺼졌을 때 그 패키지가 A-ECU로 자동 이동하지도 않습니다.

설치자는 [차량 PC 설치](#), 구성 담당자는 [1PC 배정과 현재 구현 범위](#)를 따릅니다.

누가 무엇을 하는가

아래는 PC를 분리한 경우의 실행 관계입니다. 1PC에서는 해당 패키지를 한 PC의 실행 관리자가 맡습니다.



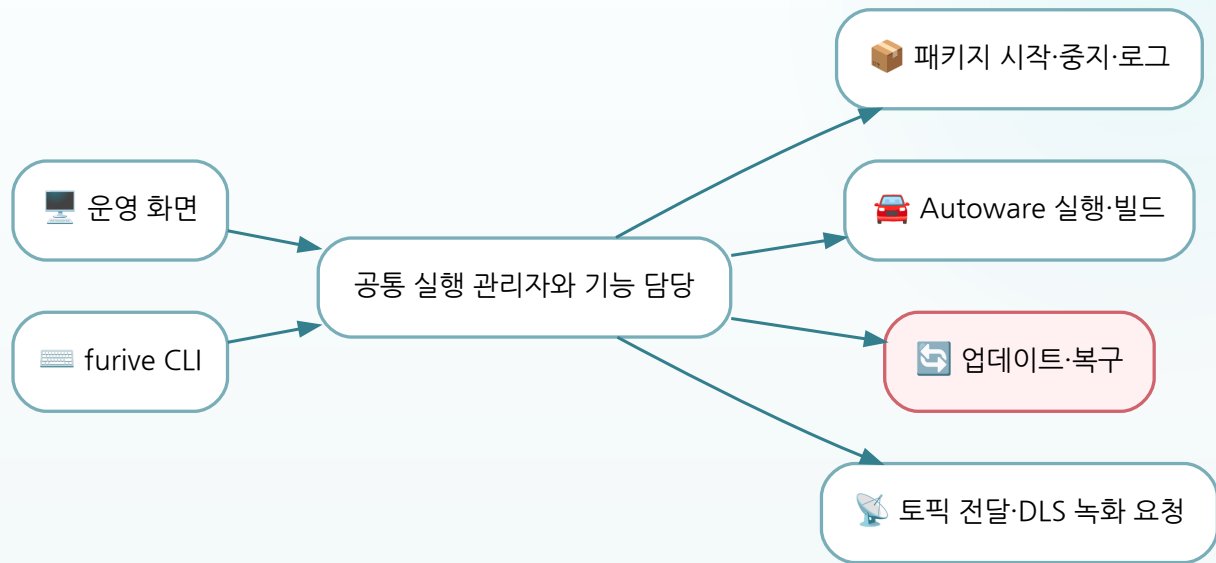
각 PC의 패키지 실행 관리자가 프로그램을 순서대로 시작하고 중지·상태 확인·재시작을 맡습니다. 위 장비 이름은 분산 구성의 예입니다. A-ECU의 자율주행 연산을 보호하도록 부가 기능의 부하를 별도 PC에 배정하며, 그 PC가 없다고 A-ECU로 자동 이동시키지 않습니다. 개별 프로그램의 기능은 그 패키지가 담당합니다. 운영 화면은 상태를 보여주고 사용자 입력을 담당 프로그램에 전달합니다.

바꾸려는 동작	담당
패키지 시작·중지·상태 확인	패키지 실행 관리자
녹화·센서 처리·차량 제어 같은 기능	해당 기능 패키지
Autoware 실행·종료	Autoware 실행 프로그램(Launcher)
업데이트·이전 정상판 복구	공통 OTA 흐름
상태 표시·사용자 입력	운영 화면

같은 상태와 실행 규칙은 한 곳에서만 관리합니다. 화면이나 각 패키지에 별도의 업데이트·재시작 규칙을 만들지 않습니다. 정확한 담당 파일은 [개발 계약](#)에 있습니다.

화면과 CLI로 같은 기능 사용하기

운영 화면과 터미널의 `furive` 명령은 같은 실행 관리자와 기능 설정을 사용합니다. 설치된 장비 배정은 자동으로 읽습니다. CLI를 쓰기 위해 운영 화면을 열어 둘 필요는 없습니다.



[!TIP] furive help 로 현재 PC에서 가능한 작업을 확인하세요. **CLI 명령어 표**에 기능별 명령, 실행 범위와 결과 확인 방법이 있습니다.

프로그램끼리 통신하기

같은 PC의 프로그램은 **Integration Gateway**라는 공용 메시지 통로로 요청과 이벤트를 주고받을 수 있습니다. 이 통로는 메시지만 전달하고 기능 상태를 직접 관리하지 않습니다. 현재 두 PC의 Integration Gateway를 서로 연결하지는 않습니다.

같은 PC의 ROS2 패키지는 로컬 ROS/DDS 환경을 사용합니다. 현재 HMI도 native ROS 2 adapter로 직접 연결합니다. HMI라는 이름 때문에 별도 HMI-ECU가 반드시 필요한 것은 아닙니다.

서로 다른 PC 사이의 ROS2 데이터는 **Zenoh Bridge**가 전달합니다. PC 사이의 등록·상태·업데이트 조정과 정의된 운용 요청은 별도 제어 연결을 사용합니다. 이 경로를 수정할 때만 **실행 구성요소**를 봅니다.

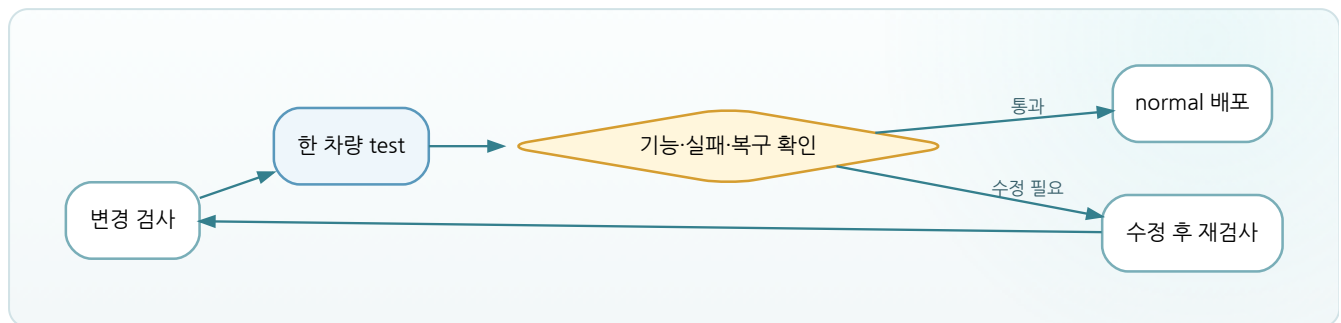
녹화는 DLS가 선택 토픽·세션·파일 검사·NAS 전송을 소유합니다. **토픽 전달 허용 → 수신 확인 → 녹화 → 종료 → 파일 검사 결과 확인**은 서로 다른 단계이며, 녹화 선택이 PC 간 전달 범위를 넓히지 않습니다. **DLS 사용 가이드**에서 검사 결과까지 확인합니다.

PC 시각 보정은 호스트의 chrony가 말합니다. PC가 여러이면 인증된 제어 연결로 시간 표본을 교환하고, 외부 표준시·차량 내부 기준·보정 중·미확인을 구분합니다. **시간 동기화**를 봅니다.

차량에 전달할 때 알아둘 것

이름	맡는 일
배포 버전(release)	차량에서 실행·설치할 내용을 고정하고 서명한 버전
설치 묶음(bundle)	기존 배포 버전 선택에 차량별 접근 정보·지도 같은 전달 파일을 더한 ZIP
test / normal	차량 한 대에서 확인하는 시험판 / 여러 차량에 배포하는 공용판

새 PC는 설치 묶음의 `./bootstrap.sh`로 설치합니다. 서비스는 부팅 때 자동으로 시작하고 A-ECU 운영 화면은 로그인 뒤 자동으로 열립니다. 내부 개발 PC의 화면은 필요할 때 `furive dashboard`로 엽니다. 설치된 차량은 지정된 배포 버전을 원격 업데이트(OTA)로 받습니다.



차량 배포 저장소의 서명된 버전을 별도 폴더에 준비하고, 재시작 때 전환합니다. 실행 중인 폴더에서 `git pull`로 덮어쓰지 않습니다. 전환 뒤 필수 패키지가 정상인지 확인하고, 실패하면 보존한 직전 코드·설정·Docker 이미지로 돌아갑니다.

실제 발행과 차량 적용은 **차량 업데이트**, 내부 복구 흐름은 **이전 정상 버전 기준**을 봅니다.

보안과 검증

구분	기본으로 포함된 것
프로그램 검사	패키지 설정·실행 파일과 각 프로그램 저장소의 자체 테스트
전체 검사	공통 코드 테스트, 설정·문서 링크·배포 규칙, 저장소에 들어간 비밀값 검사
기본 보안	PC 사이 장비 신원과 암호화 연결 확인, 필요한 프로그램에만 접근 정보 전달, 녹화·외부 전송 기본 꺼짐
차량 업데이트	서명된 배포 대상과 내용 확인, 한 차량 <code>test</code> 에서 실제 기능·대표 실패·이전 정상판 복구 확인

자동 검사만으로 센서·CAN·GPU·Docker·실제 네트워크 동작까지 확인되지는 않습니다. 관련 변경은 실차 시험으로 확인합니다. 검증 범위는 **변경 검증**, 운영자가 해야 할 일과 현재 한계는 **보안·개인정보 기준**에 있습니다.

PART II

차량 설치와 일상 운영

설치 묶음을 받은 순간부터 평소 자동 실행, 설정, 상태 확인과 안전한 정비까지 이어지는 운용 경로입니다.

02  차량 PC 설치 · p.22

03  Furive OS 기본 사용법 · p.23

02. 차량 PC 설치

이 장의 목적 · 차량 설치자

신규 PC만 장비를 한 번 선택하고 기존 PC의 배정을 유지하며 설치합니다.

완료 판단

완료 메시지와 서비스·배정된 화면의 자동 시작을 확인합니다.

이 문서는 `furive-os-install-*.zip` 을 전달받은 설치자가 차량 PC 한 대를 준비하는 순서입니다. 소스 코드 저장소를 개발하는 절차는 [Furive OS 개발 시작](#)에 따로 있습니다.

대상 차량 설치 담당자

시작 위치 설치 ZIP을 푼 폴더

완료 기준 설치 완료·서비스 자동 시작

[!TIP] 압축 해제 → `./bootstrap.sh` → **설치 완료 확인**. 신규 PC만 장비를 한 번 선택하며, 기존 설치는 저장된 배정을 유지합니다.

설치 전에 받은 것

PC 한 대로도 설치해 사용할 수 있습니다. 한 PC에서 자율주행·운영과 HMI·로깅 등을 함께 쓸 때는 그 기능이 포함된 1PC용 배정을 받습니다. A-ECU·HMI-ECU 분산 구성은 부하를 나누는 권장 방식입니다.

받은 구성	설치 순서와 완료 확인
1PC	한 PC에서 A-ECU를 선택해 한 번 설치하고, 함께 배정된 패키지가 실행되는지 확인
A-ECU·HMI-ECU 분산	각 PC에서 같은 ZIP을 사용해 해당 장비를 선택하고, 설치 뒤 PC 연결과 기능을 확인

현재 차량용 기본 배정에서 A-ECU만 설치하면 A-ECU의 기능부터 사용할 수 있습니다. HMI·DLS를 함께 실행하려면 해당 패키지를 포함한 구성이 필요합니다. 같은 PC에 두 장비의 설치를 덧씌우지 않습니다. 구성 차이는 [1PC와 분산 구성](#)을 봅니다.

차량용 ZIP 하나를 설치할 PC에서 사용합니다. 신규 설치에서는 장비 라벨을 확인하고 PC를 선택합니다. A-ECU(자율주행·운영 대시보드)가 기본 선택이며, HMI-ECU(데이터 로깅·부가 기능)는 해당 PC에서 선택을 바꿉니다. 기존 설치는 저장된 PC 배정을 사용합니다. 차량·플랫폼·버전은 다시 묻지 않습니다.

단일 묶음은 새 PC 설치 계약을 포함한 배포 버전부터 생성할 수 있습니다. 이전 버전에서 받은 장비별 ZIP은 지정된 PC 전용이며, 선택 화면 없이 기존 방식으로 설치됩니다.

같은 이름의 Docker 실행 이미지 파일을 함께 받았다면 ZIP과 같은 폴더에 둡니다. 설치 파일에는 차량 접근 정보가 들어 있으므로 Git, 공개 메신저나 공용 드라이브에 올리지 않습니다.

▶ Details

03. Furive OS 기본 사용법

이 장의 목적 · 운영자 · 정비 담당자

정상일 때는 자동 동작에 맡기고, 문제일 때만 좁은 순서로 확인합니다.

완료 판단

상태 → 로그 → 필요한 정비 명령 순서를 적용할 수 있습니다.

이 문서는 설치 ZIP의 `./bootstrap.sh`를 마친 뒤 차량 PC를 사용하는 데 필요한 내용만 설명합니다. 정상일 때 Furive OS는 자동으로 동작하므로 터미널 명령은 문제가 있거나 정비할 때만 사용합니다.

평소 동작

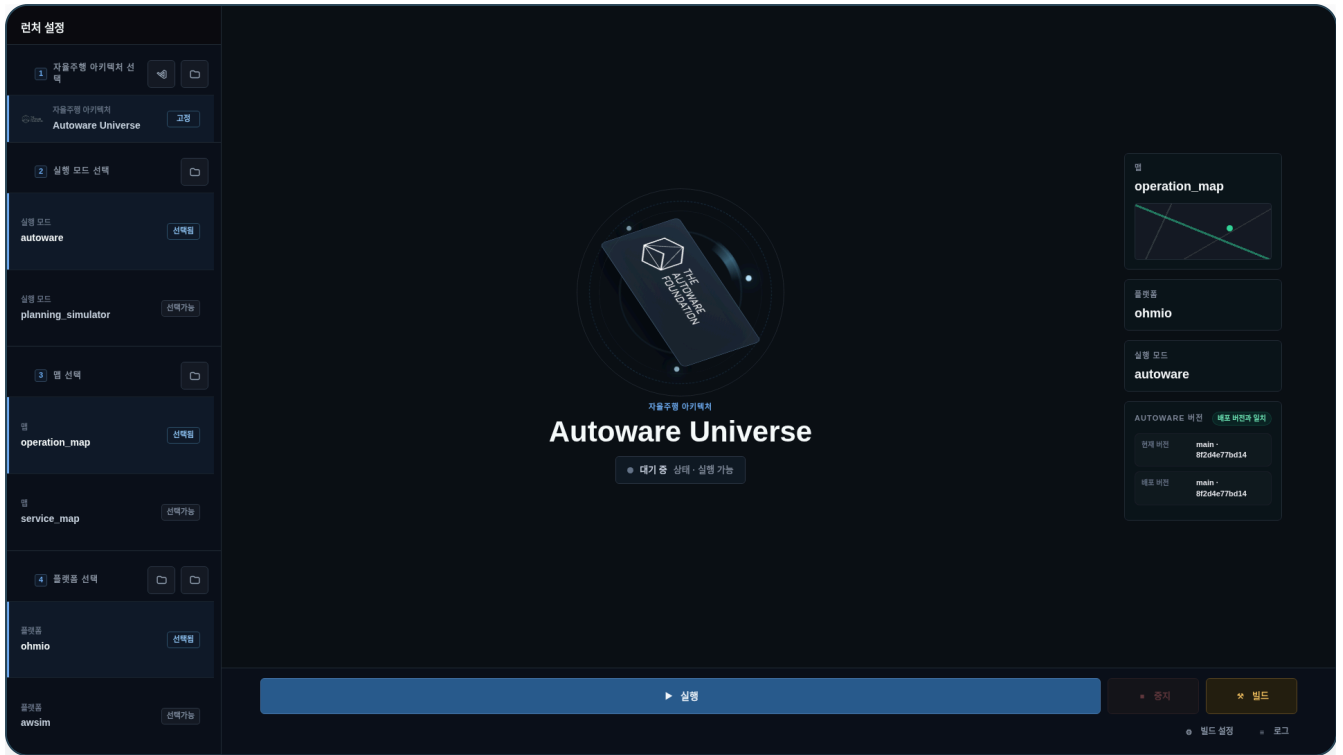
- 설치된 PC의 서비스는 부팅 때 자동으로 시작됩니다. 1PC는 함께 배정된 패키지를 실행하고, 분산 구성은 A-ECU와 HMI-ECU가 각자의 패키지를 실행합니다.
- A-ECU의 기본 화면은 보통 데스크톱 로그인 뒤 자동으로 열립니다. 내부 개발 PC는 필요할 때 `furive dashboard`로 엽니다.
- HMI-ECU에 배정된 패키지도 자동으로 동작하며, 화면 실행은 플랫폼 설정을 따릅니다.
- OTA Update는 지정된 버전을 확인하고, 검증되지 않은 업데이트는 적용하지 않습니다.

A-ECU의 기본 화면은 차량 플랫폼에 따라 달라질 수 있습니다. Lotte Ohmio에서는 Autoware 실행과 맵을 다루는 Launcher가 먼저 열립니다. 이 화면을 닫았다면 다음 명령으로 다시 엽니다.

 TERMINAL

```
furive dashboard
```

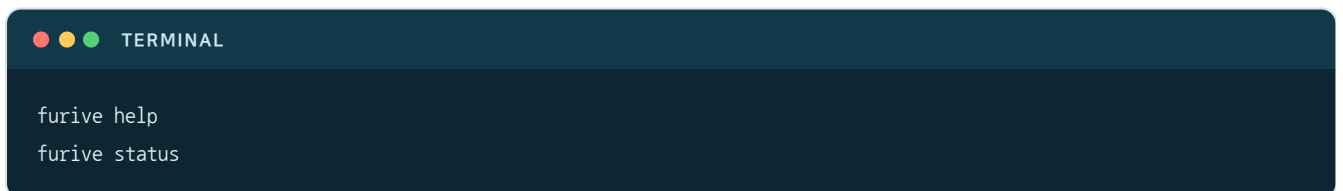
기존 `fui`도 `furive dashboard`의 호환 별칭입니다. 서비스가 정상 실행 중일 때 이 명령으로 화면을 다시 엽니다. 서비스를 다시 시작하려면 아래의 `furive service restart`를 사용합니다.



왼쪽에서 실행 모드·지도·플랫폼을, 중앙에서 현재 실행 상태를 확인합니다. 실행·중지 순서와 각 버튼은 [Launcher 사용 가이드](#)에 화면별로 설명되어 있습니다. 이 문서의 캡처는 설명용 예시이며 실제 값과 화면 구성은 설치 버전·차량 플랫폼에 따라 달라집니다.

터미널에서 운용하기

화면을 닫아도 Furive 서비스가 실행 중이면 CLI를 사용할 수 있습니다. 설치한 장비 배경은 자동으로 사용합니다. 먼저 현재 PC에서 가능한 기능과 전체 운용 상태를 확인합니다.



[!TIP] CLI 명령어 표에서 패키지·OTA·Autware·시스템·토픽 작업을 기능별로 찾으세요. furive help 는 설치 상태에 맞는 안내이고, furive --help 는 이 버전의 전체 문법입니다. 시작이 필요한 패키지나 서비스도 도움말에 표시됩니다.

furive help
사용 가능한 기능

furive status
전체 문제 확인

package status · logs
문제 패키지 조사

필요한 작업 실행

상태와 결과 재확인

DLS에서 A-ECU 토픽이 보이지 않을 때

DLS는 **Furive OS 없이도 독립 실행**할 수 있습니다. 연결된 ROS 환경의 토픽을 선택해 녹화하며, Furive OS로 다른 ECU의 토픽을 전달받는 경우에만 아래 중앙 통신 설정을 사용합니다.

1. DLS의 “토픽이 목록에 없나요?”에서 **Furive OS 통신 토픽 관리**를 엽니다.
2. 발행 중인 토픽을 찾아 **통신에 추가**합니다. 실행·녹화 중이면 중앙 설정에 저장하고 적용 대기로 표시합니다.
3. 정비 상태에서 Autoware와 녹화를 중지한 뒤 **설정 다시 적용**을 누릅니다. 이미 중지되어 있으면 추가 시 적용합니다.
4. 수신 PC에서 **조회** 기능을 확인하고 기록 화면으로 돌아갑니다. DLS 목록이 갱신되면 기록할 토픽으로 선택합니다.

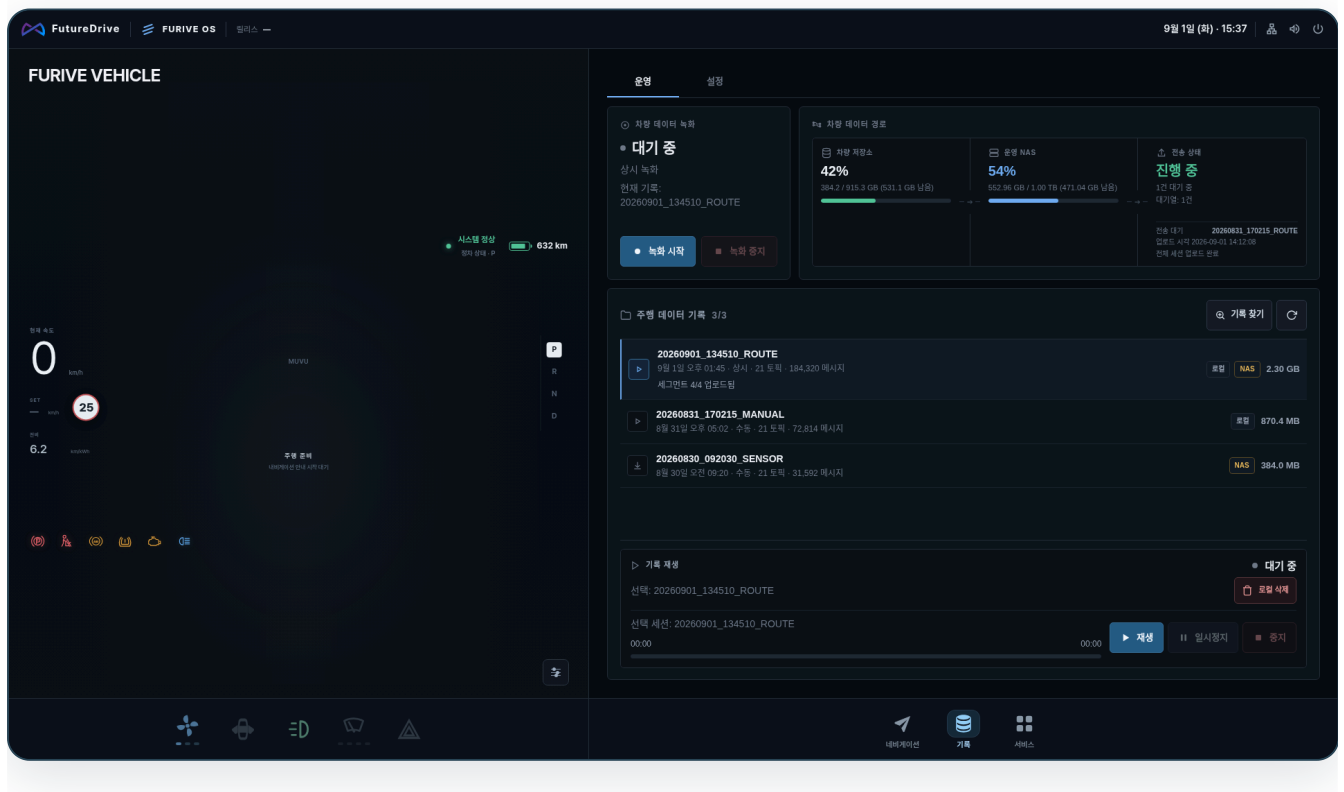
터미널에서는 `furive topic list`로 중앙 설정 토픽만 봅니다. 새 토픽을 찾을 때는 `--all`을 붙여 차량에서 발행 중인 미등록·미수신 토픽까지 확인합니다. 목록 조회는 통신 허용 범위를 바꾸지 않습니다.

```

furive topic list --all
furive topic add /sensor/example
# 적용 대기로 안내된 경우, 정비 상태에서 실행
furive autoware stop
furive topic stop
furive topic apply
# 발행 프로그램을 다시 실행한 뒤 수신 확인
furive topic list
furive topic record /sensor/example
furive topic stop

```

`record`는 기존 DLS 기록 선택에 토픽을 추가한 뒤 선택된 토픽들을 함께 녹화합니다. 중앙 통신 허용 범위는 바꾸지 않습니다. 불필요한 전달 경로는 `furive topic remove /sensor/example`로 제외합니다. 새 메시지 타입 지원이 양쪽 PC에 없으면 해당 패키지 업데이트가 필요합니다. 토픽 조회·녹화 시작 상태와 실제 파일 저장 결과는 다르므로 녹화 결과는 DLS 세션에서 확인합니다.



오른쪽 **주행 데이터 기록**에서 세션과 **로컬·NAS** 저장 위치를 확인합니다. 세션 선택·재생과 녹화 설정은 **DLS 사용 가이드**의 해당 화면으로 이어집니다.

⚙️ 설정 바꾸기

설치된 차량에서는 파일을 직접 고치지 않습니다. 지원하는 화면이나 CLI에서 설정을 저장하고, 차량 구성이나 배포 버전은 검증된 배포 버전(release) 또는 설치 묶음으로 바꿉니다.

PC 시작 동작

설치가 끝난 PC에서는 역할을 다시 쓰지 않습니다. 다음 네 명령으로 부팅 시 서비스 시작과 로그인 시 화면 열기를 각각 켜거나 끕니다.

```

# PC 부팅 시 Furive 서비스
furive setup --autostart on
furive setup --autostart off

# main PC 로그인 시 운영 화면
furive setup --dashboard-autostart on
furive setup --dashboard-autostart off
```

서비스 자동 시작 변경은 다음 부팅부터 적용되며 현재 실행 중인 서비스는 유지됩니다. 지금 바로 멈추거나 시작하려면 `furive service stop` 또는 `furive service start`를 사용합니다. 화면 자동 열기를 꺼도 이미 열린 창은 닫히지 않고, 켜도 다음 로그인 전에는 자동으로 뜨지 않습니다. 지금 열 때는 `fui`를 사용합니다. edge에는 로그인 화면 설정이 없습니다.

내부 개발 PC도 같은 명령으로 화면 자동 열기를 다시 켤 수 있습니다.

화면에서 바꾸는 운용 설정

먼저 `fui`를 열고 아래 화면을 사용합니다. 저장 버튼이 있는 값은 설치 경로나 배포 파일을 직접 고치지 않아도 유지되도록 담당 프로그램이 저장합니다.

바꾸려는 것	화면	적용 시점과 주의
main·edge의 ROS Domain ID	시스템 매니저 → 시스템 설정	저장 즉시 연결된 edge에 전달하고 관련 ROS 패키지와 컨테이너를 재시작하므로 차량을 정비 상태로 만든 뒤 변경
원격접속 허용·차단, 접속 ID 확인	시스템 매니저 → 선택 노드 상태	허용·차단은 즉시 적용되며 차단하면 진행 중인 원격 세션도 종료
Autoware 실행 모드, 지도, 차량·센서 모델	런처	Autoware를 멈춘 뒤 선택하고 다시 시작; 선택값은 다음 실행에도 사용
Furive 업데이트 잠금, 지금 확인·적용·이전 정상판 복구	OTA Update	잠금 중에는 새 업데이트와 수동 복구도 중지; 적용 전 복구 가능 상태를 화면이 확인
Autoware 버전 업데이트 허용	OTA Update	다음 Furive 원격 업데이트부터 적용; 개발 작업은 먼저 커밋하거나 별도 백업

바꾸려는 것	화면	적용 시점과 주의
수동·자동·상시 녹화, 파일 분할 시간과 최근 데이터 보관량	Data Logging System → 설정	저장 즉시 녹화 규칙에 반영될 수 있으므로 진행 중인 녹화 상태 확인
녹화 시작 조건·ROS topic·추가 정보	Data Logging System → 설정	새 구독과 다음 기록에 반영; topic 이름을 실제 ROS 연결 상태와 대조
NAS 주소·로그인·자동 전송·전송 후 삭제	Data Logging System → NAS 설정	연결 시험 뒤 저장; 자동 전송과 전송 후 삭제는 각각 별도로 켜며 실제 credential을 문서나 Git에 기록하지 않음

화면에 값이 보이지 않거나 저장에 실패하면 먼저 `furive status`, 그다음 `furive service logs` 와 해당 패키지 화면의 로그를 확인합니다. 파일을 먼저 고치면 화면 소유 상태와 서로 달라질 수 있습니다.

차량 구성과 배포 설정

바꾸려는 것	바꾸는 곳	반영 확인
Autoware 실행 방식·지도·차량/센서 모델	main의 Autoware 실행 화면 (Launcher)	Autoware를 다시 시작한 뒤 화면 상태
녹화·NAS 등 한 프로그램의 기능	해당 프로그램 화면	해당 화면과 프로그램 로그
차량의 플랫폼: <code>test</code> / <code>normal</code> ·버전	개발 저장소의 <code>furive vehicle</code>	배포 버전 적용 뒤 <code>furive status</code>
main/edge에서 실행할 프로그램 조합	<code>config/capability-profiles.yaml</code>	<code>./furive dev main</code> , 이후 한 차량 <code>test</code>
Autoware 소스 버전	<code>config/external-workspaces.yaml</code>	<code>./furive check --platform <이름></code> , 이후 한 차량 <code>test</code>
접근 정보·지도 같은 전달 파일	새 설치 묶음	<code>./bootstrap.sh</code> 뒤 해당 기능 화면

Autoware 실행 설정은 차량을 정비 상태로 만든 뒤 Autoware를 중지하고 바꿉니다. 화면에서 실행 방식, 지도와 차량/센서 모델을 저장한 다음 다시 시작하면 됩니다. 여기서 고르는 모델은 이번 Autoware 실행에 쓰는 모델이며, 차량에 설치할 전체 프로그램 조합인 Furive 플랫폼과는 다릅니다.

차량의 플랫폼이나 배포 버전은 Furive OS source에서 현재값을 먼저 보고 필요한 값만 바꿉니다.

```

 TERMINAL

./furive vehicle show <vehicle-id>
./furive vehicle set <vehicle-id> --platform <platform>
./furive vehicle set <vehicle-id> --update test
./furive vehicle show <vehicle-id>

```

이 명령은 배포할 설정만 바꾸며 차량에는 즉시 적용되지 않습니다. 실제 적용은 [차량 업데이트](#)의 `test` → 실차 확인 → `normal` 순서를 따릅니다.

주요 패키지의 개발 설정

아래 값은 일반 운용자가 설치 차량에서 직접 고치는 값이 아닙니다. 플랫폼·센서 통합 개발자가 개발 저장소에서 바꾸고 작은 실행 시험과 차량 `test` 업데이트로 전달합니다. `config.yaml` 은 현재 통합값이고, 같은 폴더에 `config.default.yaml` 이 있으면 지원하는 키와 기본 예시를 확인하는 기준입니다. 전체를 무조건 복사하지 말고 현재 장치에 필요한 값만 바꿉니다.

패키지	주로 바꾸는 값	기준 위치	적용·확인
<code>hmi</code>	창 자동 배치	<code>config.default.yaml</code>	<code>furive package run hmi</code> 후 실제 display 확인
<code>lotte-ohmio-mgmt</code>	창 자동 배치, 원격 모니터 초기 audio, 외부 명령 차단, CAN read-only	<code>config.yaml</code>	차량 명령 송신 허용 여부를 분리해 확인
<code>data-logging-system</code>	화면에서 제공하지 않는 진단·복구 기본값과 기본 topic 목록	<code>config.default.yaml</code>	먼저 화면 저장 경로 사용; 기본값 변경은 녹화·복구 시험 뒤 차량 <code>test</code>
<code>autonomy-studio-data-management</code>	거리 조건, 목적, trigger-operation topic	<code>config.yaml</code>	DLS와 함께 pending record·재전송 확인
<code>flir-camera</code>	카메라 serial-multicast 매핑: 지원하는 packet·pixel·노출·frame 키	<code>config.default.yaml</code> , 카메라별 예시	링크와 실제 image topic을 함께 확인
<code>rs80-lidar</code>	LiDAR 수, UDP port, 거리·각도, frame과 topic	<code>config.default.yaml</code>	port 충돌과 point cloud frame 확인
<code>ros2-sensor-dataset-recorder</code>	저장 경로, LiDAR-camera topic, 동기화-timeout	<code>params.yaml</code>	실제 두 센서 timestamp와 저장 파일 확인
<code>lotte-ohmio-can-ros2</code>	CAN interface·bitrate, 자동 구성, USB reset, TX 허용	<code>config.yaml</code>	정비 상태에서 RX부터 확인하고 TX는 별도 승인·실차 시험
ECU 간 ROS 전달	중앙 topic, 방향, type, QoS	<code>capability-profiles.yaml</code> 의 <code>connections.ros2</code> ; 운용 중에는 <code>furive topic</code>	양쪽 전달 상태·제어·녹화 확인; bridge 지원 항목은 기본값 참고
<code>zenoh-bridge</code>	일반 사용자가 직접 입력하지 않음	main/edge 설치 주소와 인증서가 자동 구성	주소를 package config에 중복 저장하지 말고 양쪽 연결 상태 확인

설정 변경은 다음 순서로 확인합니다.

```

furive package run <name>
furive package check
./furive check

```

센서, CAN, ROS 전달 경로 또는 패키지 조합이 바뀌면 한 차량 `test` 업데이트에서 실행과 복구까지 확인합니다. 차량이 받는 설정이나 실행 결과물이 달라졌다면 새 배포 버전이 필요합니다. 접근 정보나 지도 전달물만 바뀐 경우에만 기존 배포 버전을 유지하고 설치 묶음을 다시 만듭니다.

패키지별 전체 필드가 필요할 때는 각 저장소 README와 [프로그램 연결 설정](#)을 봅니다. 일반 운용 절차에 Docker 명령이나 package별 `run.sh`를 추가하지 않습니다.

Furive PC의 소스 코드 개발 설정

개발 PC 하나의 차이는 추적되지 않는 `config/master.yaml` 또는 `config/agent.yaml`에 **바꿀 값만** 적습니다. 예를 들어 main 개발 PC의 ROS domain, Dashboard 포트와 OTA 확인 주기만 바꾼다면 다음이면 충분합니다.

YAML · CONFIG

```
ros_domain_id: 18
dashboard_port: 18083
self_update:
  check_interval: 600
```

한 프로그램만 쓰는 설정은 그 저장소의 `config/config.yaml`에 바꿀 값만 적습니다. 팀 전체의 제품 기본값을 바꿀 때만 추적되는 `*.default.yaml`을 수정합니다. 목록 값은 일부 추가가 아니라 전체 교체되므로 필요한 항목을 모두 적어야 합니다.

TERMINAL

```
# Ctrl+C로 기존 개발 실행을 끝낸 뒤
./furive package run <name> # 프로그램 하나 확인
./furive dev main          # 전체 흐름 확인
```

설치 차량의 배포 폴더를 직접 고친 값은 다음 원격 업데이트나 재설치에서 사라집니다. 차량에 유지할 변경은 화면에서 저장하거나 검증된 배포 버전·설치 묶음으로 전달합니다.

🔍 문제가 있을 때 확인 순서

먼저 서비스, 설치 버전, OTA Update와 Autoware 실행 상태를 요약해서 봅니다.

TERMINAL

```
furive status
```

`status`는 관리 중인 패키지·PC 연결·호스트 복구 요구를 집계합니다. 오래되거나 받지 못한 상태는 정상으로 표시하지 않고 **확인 불가**로 구분합니다. 특정 프로그램을 조사할 때는 다음 명령을 사용합니다.

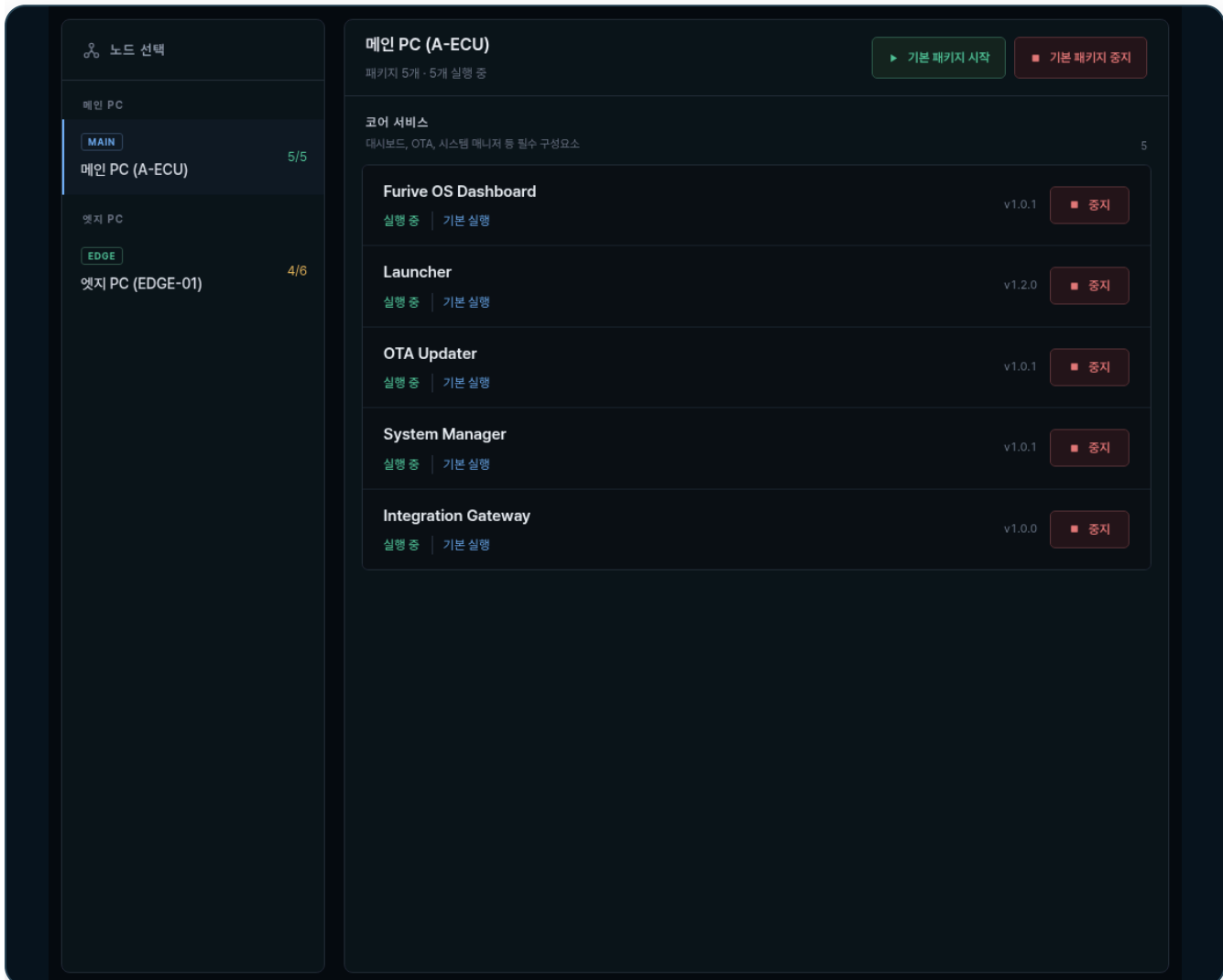
TERMINAL

```
furive package status <name>
furive package logs <name> --lines 200
```

패키지 내부의 상세 진단은 해당 기능 화면과 로그에서 확인합니다. A-ECU의 전체 Dashboard에서도 같은 운용 문제 목록을 볼 수 있습니다.

```
furive dashboard --web
```

Dashboard의 Package Manager에서 main/edge 패키지 상태와 패키지별 로그를 확인합니다. 정상 운용 중인 기본 화면이 Launcher인 차량에서도 이 명령은 전체 진단 화면을 브라우저로 엽니다.



왼쪽에서 조사할 PC를 고른 뒤 오른쪽 패키지 상태를 확인합니다. 상태의 의미와 로그 여는 위치는 [Package Manager 사용 가이드](#)에 설명되어 있습니다.

로그 보기

Furive 서비스의 최근 100줄을 본 뒤 새 로그를 계속 따라갑니다.

```
furive service logs
```

Ctrl+C 는 로그 보기를 끝낼 뿐 서비스는 중지하지 않습니다. 최근 로그만 출력하고 바로 끝내려면 다음처럼 실행합니다.

```

    ● ● ●  TERMINAL

furive service logs --no-follow --lines 200

```

개별 패키지는 `furive package logs <name> --lines 200` 으로 최근 로그를 읽거나 전체 운영 화면에서 확인합니다. **패키지 로그 명령은 출력 후 끝나고**, 서비스 로그 명령은 기본적으로 새 로그를 계속 따라갑니다. 개발 또는 차량 배포 폴더에서 파일을 조사할 때는 다음 경로를 봅니다.

FLOW / OUTPUT	
logs/<package>.log	실행 로그
logs/<package>.build.log	준비 · 빌드 로그

파일 이름에서는 패키지 이름의 하이픈이 밑줄로 바뀝니다. 예를 들어 `ota-updater` 의 실행 로그는 `logs/ota_updater.log` 입니다.

정비 중 서비스 제어

차량을 안전한 정비 상태로 만든 뒤에만 전체 Furive 서비스를 중지하거나 재시작합니다.

```

    ● ● ●  TERMINAL

furive service stop
furive service start
furive service restart

```

`stop` 은 Furive 서비스와 그 패키지를 멈추며 PC 전원을 끄지는 않습니다. `restart` 뒤에는 `furive status` 와 전체 Dashboard에서 필요한 동작을 다시 확인합니다.

소스 코드 개발 실행은 systemd 서비스와 흐름이 다릅니다. `./furive dev main` 을 `Ctrl+C` 로 끝낸 뒤 코드를 수정하고 같은 명령을 다시 실행합니다.

주요 구성요소와 설정 위치

차량 플랫폼에 따라 실행되는 패키지는 달라집니다. 아래 이름을 모두 외울 필요는 없고, 문제가 생긴 기능이 어느 항목인지 찾을 때만 사용합니다.

이름	하는 일	바꾸거나 확인하는 곳
운영 화면(Dashboard)	여러 기능의 상태와 로그를 한 화면에 모음	평소 설정 없음. 전체 화면은 <code>furive dashboard --web</code> ; 개발 포트는 <code>config/master.yaml</code> 의 <code>dashboard_port</code>
프로그램 실행 관리자 (Package Manager)	프로그램을 순서대로 시작·중지하고 상태·재시작·로그 관리	프로그램 조합은 <code>config/capability-profiles.yaml</code> ; 상태는 Dashboard의 Package Manager
Autoware 실행 프로그램(Furive Launcher)	지도와 차량/센서 모델을 준비하고 Autoware 시작·중지	운용 선택은 실행 화면; 소스 버전은 <code>config/external-workspaces.yaml</code>

이름	하는 일	바꾸거나 확인하는 곳
ota-updater	새 버전을 검증·전환하고 실패하면 이전 정상 버전으로 복구	목표 버전은 <code>furive vehicle</code> ; 개발 확인 주기는 <code>config/master.yaml</code> 또는 <code>config/agent.yaml</code> 의 <code>self_update.check_interval</code> ; 상태는 OTA 화면과 <code>furive status</code>
system-manager	main PC의 CPU·메모리· 네트워크·GPU 수집	사용자 설정 없음; Dashboard의 시스템 상태와 <code>logs/system_manager.log</code>
zenoh-bridge	main↔edge ROS 데이터를 암호화해 전달	주소와 인증서는 설치가 정함; 통합 개발 설정은 해당 package의 <code>config/</code> ; 확인은 양쪽 상태와 <code>logs/zenoh_bridge.log</code>
ros2-domain-bridge	Autoware와 전송 영역 사이에서 허용한 ROS 경로만 전달	중앙 플랫폼 설정에서 파생; 운용 변경은 <code>furive topic</code> ; 확인은 양쪽 전달 상태
integration-gateway	같은 PC의 프로그램 사이 요청 ·응답·이벤트 전달	사용자 설정 없음; 사용하는 프로그램이 의존성으로 연결

Furive에서 실행하는 각 프로그램을 **패키지**라고 부릅니다. `zenoh-bridge`의 보안 연결과 `ros2-domain-bridge`의 전달 경로는 중앙 통신 설정에서 관리합니다. 경로를 바꾸면 수신 상태를 확인하고, 플랫폼 기본값을 바꾼 통합 개발자는 양방향 제어까지 시험해야 합니다.

각 구성요소의 정확한 소유권, 통신 흐름, 설정과 로그 위치는 [실행 구성요소](#)에서 필요한 항목만 확인합니다.

PART III

개발과 플랫폼 연결

source checkout의 첫 실행부터 독립 프로그램과 새 차량·센서 구성을 Furive 계약에 연결하는 개발 경로입니다.

04  Furive OS 개발 시작 · p.34

05  프로그램을 Furive에 연결하기 · p.37

06 차량 플랫폼 만들기 · p.40

04. Furive OS 개발 시작

이 장의 목적 · Furive OS 개발자

공동 운영 기반과 독립 패키지·Autoware 저장소의 경계를 구분합니다.

완료 판단

수정 위치를 정하고 첫 개발 실행과 작은 검증을 완료합니다.

이 문서는 처음 이 저장소를 수정하는 개발자를 위한 순서입니다. 목표는 바꿀 코드 한 곳을 찾고, 작은 변경을 테스트한 뒤 다시 실행하는 것입니다. 평소 소스 개발에는 영구 설치가 필요하지 않습니다. 차량 설치 ZIP을 받았다면 [차량 PC 설치](#)로 시작합니다.

대상 처음 수정하는 개발자

시작 위치 source 저장소 루트

완료 기준 작은 변경·검사·재실행

[!TIP] 첫 실행 → 코드 한 곳 수정 → 작은 검사 → 전체 검사. 설치한 차량의 운용 명령은 [CLI 명령어 표](#)에서 찾습니다.

1. 준비하고 저장소 받기

개발 PC 한 대로 시작할 수 있습니다. A-ECU·HMI-ECU는 분산 구성의 장비 배정이며 개발에 PC 두 대를 요구하지 않습니다. 필요한 패키지를 한 PC에 함께 배정하는 방법과 현재 지원 범위는 [1PC 구성 기준](#)을 봅니다.

Linux 개발 PC에 Git, Python 3.10 이상과 코드 검색용 ripgrep([rg](#))이 필요합니다. 아래 SSH 주소와 연결된 패키지 저장소를 읽을 GitHub 권한도 필요합니다. 전체 화면·패키지 실행에 필요한 도구는 각 패키지의 README를 따릅니다. Docker·GPU·차량 장치가 필요한 패키지는 해당 환경에서 실행합니다.

 TERMINAL

```
git clone --recurse-submodules git@github.com:Furive-ADS/furive-os.git
cd furive-os
```

--recurse-submodules 는 package/ 아래 연결된 별도 저장소까지 받습니다. 이미 받은 저장소에서 패키지 폴더가 비어 있을 때만 git submodule update --init --recursive 를 실행합니다.

Python 검사 환경이 없는 새 PC에서는 한 번 준비합니다. 아래 환경은 공통 Python 코드의 작은 테스트용이며, 각 패키지의 빌드·실행 의존성은 해당 패키지가 관리합니다.

 TERMINAL

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install PyYAML pytest
```

새 터미널에서는 저장소 최상위 폴더에서 `source .venv/bin/activate` 로 같은 환경을 사용합니다. `venv` 를 만들 수 없다면 OS의 Python `venv` 패키지를 준비합니다.

이후 `./furive` 명령은 이 저장소의 최상위 폴더에서 실행합니다. 설치한 PC의 `furive` 는 설치된 저장소를 가리키므로, 지금 수정 중인 `source` 를 실행할 때는 `./furive` 를 씁니다.

2. 한 번 실행해 기준 동작 확인하기

아직 Furive를 설치하지 않은 개발 PC의 첫 실행입니다.



터미널에 개발 실행 안내가 나오고 `main` 운영 화면과 선택된 패키지가 시작됩니다. 화면과 로그에서 바꾸려는 기능의 현재 동작을 확인하고 `Ctrl+C` 로 종료합니다.

`dev` 는 개발 중 원격 업데이트를 잠시 보호하고, 종료할 때 자신이 만든 임시 보호를 해제합니다. 기존 Furive 서비스가 실행 중이면 개발하는 동안 멈추고 종료 뒤 다시 시작합니다.

`edge` 동작을 개발하는 미설치 PC에서만 `./furive dev edge` 를 사용합니다. 이미 설치된 PC에서는 `./furive dev` 가 그 PC의 역할을 읽습니다. 화면 없는 PC 등 예외 옵션은 `./furive dev --help` 에서 확인합니다. 하드웨어가 필요한 기능은 코드의 작은 테스트부터 진행할 수 있습니다.

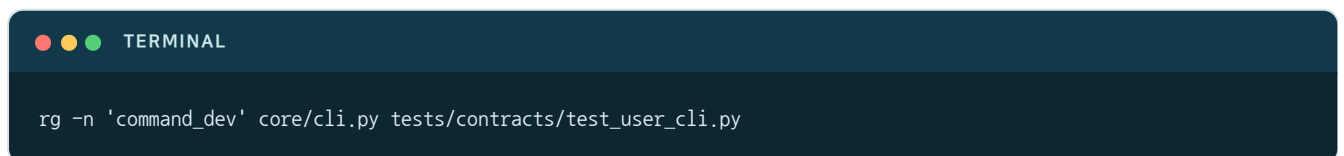
3. 바꿀 코드 한 곳 찾기

처음에는 세 가지만 구분합니다. **패키지**는 연결한 독립 프로그램, **플랫폼**은 차량·센서 모델과 패키지 조합, **main/edge**는 프로그램을 실행하는 PC의 역할입니다. 관계가 궁금할 때만 [한눈에 보기](#) 를 읽습니다.

먼저 [루트 폴더 지도](#) 에서 담당 폴더를 고릅니다. 공통 코드를 수정한다면 [공통 소스 안내](#) 에서 기능별 시작 파일과 실행 순서를 확인합니다. 아래는 첫 변경에 가까운 테스트를 찾는 예시입니다.

바꾸려는 것	처음 열 곳	가까운 테스트
명령·도움말	core/cli.py	test_user_cli.py
패키지 실행·종료	core/package_manager.py	test_package_manager_lifecycle.py
차량·플랫폼 배경	core/fleet_assignment.py	test_fleet_assignment.py
프로그램 자체 기능	해당 <code>package/</code> 저장소의 README	그 저장소의 테스트
Autoware·Unity 소스	해당 프로그램의 별도 개발 저장소	그 저장소의 테스트

다른 작업의 정확한 담당 파일은 [개발 계약의 수정 위치](#) 를 기준으로 합니다. 담당 폴더 안에서 함수와 가까운 테스트를 검색합니다. 예를 들어 CLI의 개발 실행을 찾으려면:



같은 상태나 실행 규칙을 새 helper·설정·화면 코드에 다시 만들지 않고 기존 담당 코드를 수정합니다. 상세 호출 관계가 필요할 때만 [기술 구조](#)를 엽니다.

4. 수정하고 검사하기

먼저 `git status --short` 로 기존 변경을 확인합니다. 문제를 재현하는 작은 테스트를 찾거나 추가한 뒤 담당 코드를 수정하고 그 테스트를 실행합니다. 예를 들어 CLI를 바꿨다면:

```
python3 -m pytest -q tests/contracts/test_user_cli.py
```

패키지 기능을 바꿨다면 해당 저장소에서 자체 테스트를 먼저 실행합니다. 통과한 뒤 Furive OS 최상위 폴더에서 전체 검사를 실행하고 2단계의 개발 명령으로 바꾼 동작을 다시 확인합니다.

```
./furive check
```

일상 개발의 완료 조건: 바꾼 동작과 대표 실패를 확인하고, 가까운 테스트와 전체 검사가 통과하며, 사용 방법이 달라졌다면 해당 가이드를 함께 고칩니다. 실제 장치 등 실행하지 못한 검증은 남겨 둡니다. 검사의 범위와 테스트 배치는 [테스트 안내](#)를 봅니다.

`package/` 를 수정했다면 **그 저장소에서** 테스트하고 `commit·push`합니다. Furive OS가 새 버전을 사용해야 할 때 최상위 저장소에 그 패키지의 `commit` 변경을 반영합니다.

막혔을 때

증상	다음 행동
저장소 받기가 권한 오류로 실패	실패한 저장소 주소의 GitHub·SSH 접근 권한 확인
패키지 폴더가 비어 있음	1단계의 submodule 초기화 실행
Python 모듈을 찾지 못함	가상 환경 활성화 확인; 공통 검사 환경은 1단계, 패키지 의존성은 해당 README 확인
패키지가 시작하지 않음	개발 터미널 안내와 패키지 로그 확인
전체 검사 실패	출력에서 처음 실패한 검사·패키지를 찾고 테스트 안내 에서 범위 확인

변경을 차량에 전달할 때만

문서·테스트만 바뀌었다면 코드 변경으로 끝냅니다. 차량의 실행·설치 내용이 바뀌었다면 [차량 업데이트](#)의 `test` → `실차 확인` · `복구` → `normal` 순서를 따릅니다. 접근 정보·지도 같은 설치 전달물만 바뀌었을 때의 판단도 그 문서에 있습니다.

새 프로그램을 연결하려면 [프로그램 연결](#), 새 차량·센서를 지원하려면 [차량 플랫폼 만들기](#)를 이어 읽습니다.

05. 프로그램을 Furive에 연결하기

이 장의 목적 · 패키지 개발자

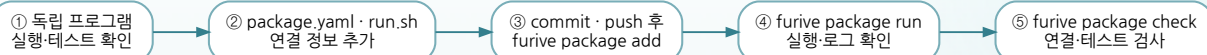
최소 package.yaml과 실행 진입점으로 프로그램을 연결합니다.

완료 판단

단독 실행·종료·로그·package check가 모두 통과합니다.

이미 독립 저장소에서 실행되는 프로그램을 Furive에 연결할 때 이 문서를 사용합니다. Furive에서는 이렇게 연결한 프로그램을 **패키지**라고 부릅니다. Autoware package나 ROS package를 새로 만드는 절차는 아닙니다.

처음 연결할 때의 흐름은 하나입니다.



대상 패키지 통합 개발자

시작 위치 독립 저장소 + Furive source

완료 기준 실행·연결 규칙·테스트 확인

[!TIP] 이름·버전과 실행 파일부터 연결하세요. Docker·장치·의존성처럼 기본값과 다른 요구만 설정에 추가합니다.

1. 프로그램과 테스트를 먼저 준비하기

Furive에 연결하기 전에 원래 저장소에서 프로그램의 빌드와 단독 실행을 확인합니다. Furive는 프로그램 코드, Dockerfile이나 테스트를 대신 만들지 않습니다.

저장소에는 실제 기능을 확인하는 테스트가 하나 이상 있어야 합니다. 정상 입력과 대표 실패 입력을 실행해 API 응답, ROS 메시지, 파일 또는 계산 결과처럼 밖에서 관찰되는 결과를 확인합니다. 프로그램이 살아 있거나 코드 문구가 존재한다는 검사만으로는 충분하지 않습니다. 자동으로 찾는 형식은 **패키지가 소유하는 테스트**에 있습니다.

2. 이름과 실행 방법 알려주기

저장소 루트의 package.yaml 에는 먼저 두 값만 적습니다.

YAML · CONFIG

```
name: hello-sensor
version: 0.1.0
```

다른 필드를 생략하면 edge에서 `run.sh`를 실행하는 일반 패키지가 되며, 프로그램이 실행 중인지 확인합니다.

`run.sh`는 필요한 환경을 준비한 뒤 마지막에 실제 프로그램을 `exec` 합니다.

```
#!/usr/bin/env bash
set -euo pipefail

PACKAGE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
exec "$PACKAGE_DIR/build/service" "$@"
```

실행 파일 경로와 빌드는 프로그램 저장소가 담당합니다. `run.sh` 안에 무한 재시작, `kill`, 자체 업데이트, 로그 순환이나 다른 패키지 제어를 넣지 않습니다. 시작·중지·상태 확인·재시작은 공통 프로그램 실행 관리자(Package Manager)가 한 번만 담당합니다.

Docker, main 실행, 장치, 의존성처럼 기본값과 다른 요구가 있을 때만 `package.yaml`에 항목을 추가합니다. 필드 이름과 예시는 [프로그램 연결 설정](#)을 봅니다.

3. 독립 저장소를 연결하기

`package.yaml`, `run.sh`와 테스트를 독립 저장소에 commit하고 원격 저장소에 push한 뒤 Furive OS 개발 저장소의 최상위 폴더에서 연결합니다.

```
./furive package add <repository-url>
```

이 명령은 기존 저장소를 `package/plugin` 아래 연결 저장소(Git submodule)로 추가합니다. 파일을 복사하거나 같은 프로그램의 두 번째 저장소를 만들지 않습니다. 아직 push하지 않은 로컬 파일은 `package add`가 가져올 수 없습니다.

4. 패키지 하나 실행하고 로그 보기

```
./furive package run hello-sensor
```

터미널에는 Furive의 시작·중지 안내만 표시됩니다. 프로그램의 일반 출력과 오류 출력(stdout/stderr)은 다음 파일에 기록됩니다.

```
FLOW / OUTPUT

logs/<정규화된-패키지명>.log
```

프로그램 실행 관리자는 패키지명의 하이픈을 밑줄로 바꾸므로 이 예시의 로그는 `logs/hello_sensor.log`입니다. 실행 중 로그를 볼 때는 다른 터미널에서 확인합니다.

● ● ● TERMINAL

```
tail -f logs/hello_sensor.log
```

실행한 터미널에서 `Ctrl+C`를 누르면 패키지와 이번 실행이 시작한 의존 패키지가 역순으로 종료됩니다.

5. 연결 정보와 테스트 확인하기

● ● ● TERMINAL

```
./furive package check
```

이 명령은 모든 연결된 프로그램의 `package.yaml`을 먼저 검사하고, 유효할 때 각 저장소의 테스트를 실행합니다. 중복 이름, 잘못된 필드, 없는 실행 파일, 테스트 없음과 테스트 실패는 모두 실패입니다.

Furive OS 변경과 함께 공유할 준비가 끝나면 전체 검사도 실행합니다.

● ● ● TERMINAL

```
./furive check
```

6. 차량 구성에 포함하기

패키지 단독 실행과 검사가 끝난 뒤에만 해당 플랫폼의 main 또는 edge 패키지 목록에 이름을 추가합니다. 플랫폼 구성은 [차량 플랫폼 만들기](#), 한 차량 적용은 [차량 업데이트](#)를 따릅니다.

완료 조건은 네 가지입니다.

- 독립 저장소에서 프로그램과 실제 기능 테스트가 통과한다.
- `package.yaml`과 `run.sh`의 최소 정보만으로 동작한다.
- `furive package run`으로 시작하고 `Ctrl+C`로 함께 종료된다.
- 정규화된 로그 파일을 확인했고 `furive package check`가 통과한다.

06. 차량 플랫폼 만들기

이 장의 목적 · 플랫폼 개발자

차량 차이를 공통 설치·운영 코드 복사 없이 플랫폼으로 표현합니다.

완료 판단

개발 PC 검증 뒤 한 차량 test와 복구 확인까지 계획합니다.

새 차량·센서 구성을 Furive에 연결할 때 이 문서를 사용합니다. 플랫폼은 여러 차량이 공유할 **차량·센서 구성, 정확히 고정된 Autware 소스 버전과 PC별 패키지 조합**입니다. Furive가 임의의 모든 차량을 자동 지원한다는 뜻은 아닙니다. 설치, 프로세스 감독, 로그, 대시보드, OTA는 공통 Furive 기능이므로 플랫폼마다 복사하지 않습니다.

플랫폼명과 Autware 모델명은 다릅니다

두 이름은 서로 다른 대상을 가리키며 같을 필요가 없습니다.

이름	예	소유하는 것
Furive 플랫폼	lotte-ohmio	패키지 구성, 화면·지도 기본값, Autware 버전 선택
Autware 모델 이름	lotte_bus	<이름>_vehicle 과 <이름>_sensor_kit 의 공통 앞부분

예를 들어 lotte-ohmio 플랫폼이 선택한 Autware 작업 공간 안에 lotte_bus_vehicle 과 lotte_bus_sensor_kit 이 있어도 정상입니다. vehicle 모델과 sensor-kit 모델은 서로 같은 이름 앞부분으로 한 쌍을 이루지만, 이를 Furive 플랫폼명과 맞추는 규칙은 없습니다.

실제 선택 흐름은 다음과 같습니다.

FLOW / OUTPUT

차량 ID

- Furive 플랫폼
- 한 PC에 함께 배경하거나 A-ECU · HMI-ECU로 분산
- 플랫폼에 고정된 Autware 소스 버전
- 그 작업 공간의 Autware 차량/센서 모델 선택
- Autware 실행 프로그램이 시작 · 중지

변경을 담당하는 곳 찾기

바꾸려는 것	담당하는 곳	기준 위치
차량별 플랫폼·업데이트·버전	차량 배경 정보	core/fleet_assignment.py , config/fleet-vehicles.yaml
플랫폼의 패키지·화면·지도 조합	플랫폼 구성	config/capability-profiles.yaml

바꾸려는 것	담당하는 곳	기준 위치
플랫폼이 받을 Autoware 소스 버전	소스 버전 고정 설정	<code>config/external-workspaces.yaml</code>
Autoware 차량 모델·보정	Autoware 저장소	<code><prefix>_vehicle</code> 계열
Autoware 센서 배치·보정	Autoware 저장소	<code><prefix>_sensor_kit</code> 계열
CAN·장치 연결 기능	해당 차량 패키지	독립 패키지 저장소, 공통 연결 코드는 <code>core/vehicle_control.py</code>
Autoware 탐색·시작·중지	Autoware 실행 프로그램	<code>package/core/furive-launcher</code> , <code>core/launcher_control.py</code>

HMI, 로깅, 장치 driver처럼 다른 차량도 독립적으로 사용할 수 있는 기능은 Autoware 모델에 넣지 않고 별도 패키지로 둡니다. 새 프로그램 연결은 [프로그램 연결](#)을 먼저 완료합니다.

1. 구현을 실제 소유 저장소에 두기

Autoware 실행·설정과 차량·센서 모델은 별도 Autoware 작업 공간에서 개발하고 테스트합니다. 독립 기능은 그 기능의 패키지 저장소에서 개발합니다. Furive OS에는 이 소스 코드를 복사하지 않고 선택 정보만 둡니다.

차량 구동·센서 배치는 Autoware 모델, CAN 장치·독립 서비스는 해당 패키지, 기능 사용 여부는 플랫폼 구성이 담당합니다. 설치·로그·상태 확인·업데이트는 Furive 공통 코드에서 수정합니다.

2. 플랫폼 구성을 연결하기

`config/capability-profiles.yaml`에 플랫폼 이름 하나를 만들고 필요한 main/edge 패키지와 기본 화면, 설치 묶음에 포함할 승인 지도만 연결합니다. 내부 YAML에는 `master / agent`가 남아 있고 source 개발에서는 `main / edge` 역할을 사용합니다. 일반 설치·운용 안내에서는 플랫폼의 장비 이름을 씁니다.

PC 한 대에서 함께 쓸 기능은 기존 `node_configs.master.packages`에 명시적으로 배정합니다. 분산 배정은 A-ECU의 자율주행과 HMI-ECU의 로깅·부가 기능으로 나눕니다. 1PC에서도 공통 runtime을 복사하거나 두 역할의 설치를 중복 실행하지 않습니다. 장치·ROS domain·의존성·화면·포트와 동시 부하, 현재 중앙 토픽 CLI의 제약은 [ECU 설치 설계를](#) 따릅니다.

그 플랫폼 이름을 `config/external-workspaces.yaml`의 `targets.capability_profiles`에도 사용해 Autoware 최상위 저장소와 관리 대상 하위 저장소를 정확한 commit으로 고정합니다. Git branch의 최신값을 차량에서 다시 해석하게 두지 않습니다.

두 파일의 역할은 겹치지 않습니다.

- `capability-profiles.yaml`: 차량이 어떤 패키지 구성을 실행하는가
- `external-workspaces.yaml`: 그 구성이 어떤 Autoware 소스 버전을 받는가

차량 한 대는 `furive vehicle set`으로 이 플랫폼을 참조합니다. 차량별 설정 파일을 복사하거나 영구 차량 전용 branch를 만들지 않습니다. 소스 버전 고정의 상세 기준은 [CI/CD 기준](#)을 봅니다.

3. 개발 PC에서 실제 흐름 확인하기

새 패키지가 있다면 먼저 하나만 실행하고 검사합니다.

```

.
├── ...
└── ...

```



```

./furive package run <패키지>
./furive package check

```

그다음 전체 구성을 실행합니다. 역할이 아직 없는 소스 코드 작업 폴더의 첫 실행만 `main` 을 적습니다.

```

./furive dev main

```

이미 설치 역할이 있는 PC에서는 `furive dev` 만 사용합니다. `main` 화면에서 패키지 구성과 Autoware 실행 화면을 열고 다음을 확인합니다.

- 고정한 Autoware 소스 버전이 준비됐는가
- 실행 화면이 완전한 차량/센서 모델 쌍을 찾는가
- 모델, 지도와 실행 방식을 선택해 Autoware를 시작·중지할 수 있는가
- 새 패키지의 실제 출력과 `logs/<이름>.log` 가 정상인가

작업 공간에 모델 쌍이 하나면 자동 선택하고, 여러 쌍이면 화면에서 하나를 선택합니다. 탐색 경로와 내부 선택 파일을 조사할 때만 [Autoware 플랫폼 탐색](#)을 봅니다. 이 참고 문서의 내부 `platform` 표현은 Furive 플랫폼이 아니라 Autoware 모델 이름 앞부분을 뜻합니다.

4. 검사한 뒤 차량 한 대에서 시험하기

```

./furive check

```

Autoware 소스 버전이나 빌드 입력을 바꿨고 해당 작업 공간을 빌드할 수 있는 PC라면 선택 진단을 추가합니다.

```

./furive check --platform <Furive-플랫폼>

```

이 검사는 고정한 소스 코드가 빌드되지만 확인하며 실제 CAN·센서·차량 동작을 증명하지 않습니다. 마지막으로 [차량 업데이트](#)의 `test` 흐름에서 한 차량의 기능과 이전 정상 버전 복구를 확인한 뒤에만 `normal` 로 승격합니다.

완료하려면 패키지 구성과 Autoware 소스 버전을 담당하는 곳이 분리되어 있고, 차량 차이가 Autoware 모델이나 독립 패키지 한 곳에 있어야 합니다. 개발 검사와 한 차량 `test` ·복구까지 확인합니다.

PART IV

검증·시험·배포

일상 개발, 한 차량 test, 고객용 normal을 구분하고 실제 변경만 차량 배포 버전으로 만드는 경로입니다.

07 변경 검증 · p.44

08 🚀 차량 업데이트 · p.47

07. 변경 검증

이 장의 목적 · 모든 개발자 · 검증 담당자

일상 개발·한 차량 시험·normal의 완료 조건을 구분합니다.

완료 판단

실행한 증거만 PASS로 기록하고, 적용되지 않는 항목은 N/A 사유를 남깁니다.

코드나 설정을 바꾼 뒤 어디까지 확인해야 완료인지 정할 때 사용합니다. 모든 변경에 실차 시험과 고객 배포 증적을 한꺼번에 요구하지 않고, 변경이 도달한 단계까지만 검증합니다.

세 단계만 고르기

단계	언제 필요한가	완료 조건
일상 개발	모든 변경	가까운 작은 테스트와 <code>furive check</code> 통과
한 차량 시험	설치·업데이트·장치·차량 데이터 흐름을 바꿈	한 차량 <code>test</code> 의 실제 기능과 이전 정상 버전 복구 확인
고객용 <code>normal</code>	고객 차량에 공용판을 배포함	시험 증거와 이번 변경에 해당하는 운영 항목 확인

뒤 단계는 앞 단계를 대체하지 않습니다. 한 차량 시험이 필요한 변경도 먼저 일상 개발 검사를 통과해야 합니다.

1. 일상 개발

바꾼 코드와 가장 가까운 작은 테스트부터 실행합니다.

TERMINAL

```
python3 -m pytest -q tests/contracts/test_user_cli.py
./furive package check
```

독립 패키지를 바꿨다면 그 저장소의 자체 테스트도 먼저 실행합니다. 실제 접근 정보, 개인정보나 고객 데이터를 테스트용 입력에 사용하지 않습니다.

마지막에 개발 저장소의 최상위 폴더에서 공통 검사를 실행합니다.

TERMINAL

```
./furive check
```

이 검사는 코드, 설정, 문서 링크, 프로그램 연결 정보와 차량 배포 기본 규칙을 확인합니다. 최상위 테스트와 패키지 테스트는 동시에 실행되며, 패키지 내부의 임시 파일과 cache는 서로 섞이지 않습니다. 센서, CAN, GPU, NAS가 없는 개발 PC에서 해당 장치가 동작했다는 뜻은 아닙니다.

구조가 복잡해지는 회귀도 같은 검사에서 확인합니다. `tests/contracts/test_core_dependencies.py` 는 공통 Python 코드가 `nodes`, `package`, `scripts` 구현을 직접 import하거나 순환 의존을 만드는 변경을 예외 없이 거부합니다. 설치 복구 전 OTA 상태 판정도 후보 적용·확정·복구를 담당하는 `core/release_activation.py` 가 소유합니다. 동적 로딩·shell 호출과 상태·정책의 중복 소유까지 증명하는 검사는 아니므로, 변경 검토에서는 기존 소유자와 사용자 개념이 늘었는지도 확인합니다. 새 예외를 추가해 검사를 통과시키기 전에 기존 소유자 안에서 해결합니다.

속도는 CI/CD의 변경 범위 검사와 이미지 재사용, 업데이트 복구는 이전 정상 버전 계약을 따릅니다. 검사 시간 단축과 실제 차량의 준비·전환·복구 시간은 구분해서 기록하며, 재사용은 해당 검증·산출물의 identity가 맞을 때만 허용합니다. 변경량과 무관한 속도나 무장애를 보장하는 것으로 해석하지 않습니다.

Autoware 소스 버전 또는 빌드 입력을 바꿨고 해당 작업 공간을 빌드할 수 있는 PC라면 선택 진단을 추가합니다.

```
./furive check --platform <플랫폼>
```

일상 개발은 여기서 끝낼 수 있습니다. 장치 시험이 아직 필요한 경우 결과를 실패로 숨기지 말고 “개발 PC에 장치 없음, 한 차량 시험 필요”라고 제한을 남깁니다.

2. 한 차량 시험

설치·OTA·복구, Autoware 실행, CAN·센서·GPU·화면·NAS, 녹화·외부 전송·차량 제어 또는 여러 패키지를 잇는 실제 데이터 흐름을 바꿨을 때 한 차량 `test` 를 추가합니다.

시험 순서는 고정합니다.

FLOW / OUTPUT

현재 normal과 정상 결과 기록

- test 적용
- 정상 입력 확인
- 대표 실패 입력 확인
- 이전 normal 복구
- 정상 결과 재확인

PASS는 프로그램이나 컨테이너가 떠 있지만 보지 않고 바꾼 기능의 실제 결과로 판단합니다. API 응답, ROS 메시지, 생성 파일, 화면 또는 장치 결과 중 관련된 것을 남깁니다. 적용 절차는 차량 업데이트를 따릅니다.

차량이나 장치가 없어 실행하지 못했다면 일상 개발은 완료할 수 있지만 한 차량 시험을 PASS로 기록할 수는 없습니다.

3. 고객용 normal

한 차량 시험을 통과한 같은 소스 코드와 결과물만 `normal` 로 승격합니다. 다음 중 이번 변경과 관련된 항목만 추가로 확인합니다.

- 목표 차량과 플랫폼, 서명된 배포 버전과 Docker 이미지·소스 버전 일치 여부
- normal 적용 뒤 프로그램 정상 상태와 실제 기능 결과
- 이전 정상 버전 복구 가능 여부
- 접근 정보·권한을 바꿨다면 전달, 저장, 회수·교체 담당자
- 위치·영상·고객 데이터 또는 외부 전송을 바꿨다면 목적, 수신자, 보유·삭제와 승인

- 차량 제어를 바꿨다면 안전한 시험 조건, 실패 상태와 수동 복구

문서·테스트처럼 차량 실행 결과물을 바꾸지 않은 변경에 `normal` 배포 버전을 만들지 않습니다. 개인정보, 권한이나 외부 전송을 바꾸지 않았다면 전체 보안·법적 검토를 반복하지 않습니다. 필요한 경우만 [보안·개인정보 기준](#)을 봅니다.

배포 버전과 설치 묶음 판단 및 승격 절차는 [차량 업데이트](#)와 [CI/CD 기준](#)을 따릅니다.

결과 기록

판정	뜻
PASS	현재 단계의 검사를 실행하고 기대한 실제 결과를 확인함
FAIL	필요한 검사가 실패했거나, 필수인데 실행하지 못함
N/A	이번 변경에는 적용되지 않으며 그 이유를 기록함

PR, 차량 시험 기록 또는 배포 기록 한 곳에 다음만 남깁니다.

FLOW / OUTPUT

변경:

단계: 일상 개발 | 한 차량 시험 | 고객용 `normal`

대상: Git commit / 패키지 / 플랫폼 / 차량 버전

실행한 명령:

확인한 실제 결과:

판정: PASS | FAIL | N/A

남은 제한:

미래 인증 계획이나 이번 변경과 무관한 운영 자동화를 완료 조건으로 만들지 않습니다. 실행하지 않은 검사는 PASS로 쓰지 않고, 해당하지 않는 항목은 이유와 함께 N/A로 남깁니다.

08. 🚀 차량 업데이트

이 장의 목적 · 릴리즈 담당자 · 차량 시험 담당자

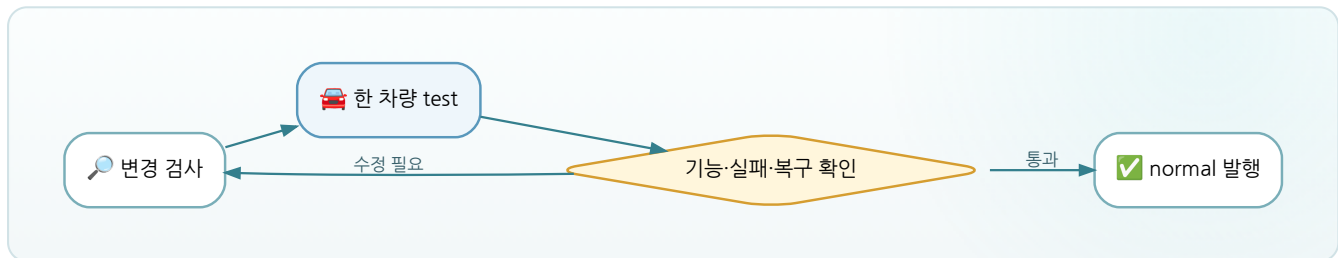
release와 bundle을 구분해 시험 차량과 공용판을 안전하게 갱신합니다.

완료 판단

test 기능·실패·복구를 확인한 같은 결과만 normal로 승격합니다.

이 문서는 차량 한 대에서 test를 확인하고 복구한 뒤, 같은 변경을 normal 공용판으로 승격하는 실제 흐름을 설명합니다. 명령은 Furive OS 개발 저장소에서 실행하고, 차량에서는 자동으로 열린 운영 화면을 사용합니다.

vehicle, release, bundle 명령은 개발 저장소에서만 실행하며 차량 배포 저장소를 직접 수정하지 않습니다.



대상 배포 담당 개발자

시작 위치 Furive OS source 저장소

완료 기준 한 차량 검증 후 normal 승격

[!TIP] 한 차량 test → 기능·복구 확인 → normal. 문서만 바뀌었다면 차량 release나 설치 bundle을 발행하지 않습니다.

먼저 배포 버전과 설치 묶음을 구분하기

두 산출물의 역할은 다릅니다.

바뀐 것	필요한 작업
차량이 실행할 코드·패키지·Docker 이미지·Autoware 버전	배포 버전(release)
같은 배포 버전에 더할 접근 정보·지도·전달 파일	설치 묶음(bundle)
두 종류가 모두 바뀜	release 성공 후 bundle
문서·테스트·CI 편의만 바뀌고 두 산출물이 같음	둘 다 만들지 않음

release는 서명 후 내용을 바꿀 수 없는 차량 배포 버전입니다. bundle은 이미 발행된 배포 버전에 접근 정보와 지도 같은 설치 파일을 더한 ZIP입니다. 설치 묶음이 패키지, 플랫폼, Autoware 버전이나 계속 유지할 실행 설정을 새로 결정하지는 않습니다.

판단이 애매하면 CI/CD 릴리즈 판단의 네 질문을 사용합니다. `furive release ... --run` 은 차량 실행 내용이 그대로면 발행을 거부하고, `furive bundle` 은 아직 배포 버전으로 만들지 않은 실행 변경이 있으면 기존 버전 재사용을 거부합니다.

1. 시험 차량 확인하기

차량은 처음 등록할 때만 플랫폼이 필요합니다.

```
● ● ● TERMINAL

./furive vehicle set lotte-ohmio-1 \
  --platform lotte-ohmio --update normal --version latest
./furive vehicle show lotte-ohmio-1
```

이미 등록된 차량은 `vehicle show` 에서 플랫폼, main/edge 패키지, main 화면과 지도를 확인합니다. 정확한 배포 버전이 머물러야 하는 예외에만 `--version vYYYY.MM.DD.N` 을 사용하고, 일반 경로는 `latest` 입니다.

`vehicle set` 은 개발 저장소의 차량 배정만 바꿉니다. 이미 설치된 차량의 업데이트 종류나 버전을 원격으로 즉시 바꾸지 않습니다. 설치 상태를 바꾸려면 새 배정으로 만든 설치 묶음을 해당 차량에 적용해야 합니다.

시험 전에는 현재 `normal` 버전과 바꾼 기능의 정상 결과를 기록합니다.

2. 한 차량에 test 발행하기

먼저 변경 범위의 테스트와 공통 검사를 통과시킵니다.

```
● ● ● TERMINAL

./furive check
```

차량에서 실행·설치할 내용이 바뀌었다면 시험 차량을 `test` 로 바꾸고 발행 계획을 봅니다.

```
● ● ● TERMINAL

./furive vehicle set lotte-ohmio-1 --update test
./furive release test lotte-ohmio-1 --date 2026-08-31 --number 1
```

계획에서 차량 ID, 플랫폼, 소스 코드와 변경 결과를 확인한 뒤 같은 명령에 `--run` 을 붙입니다.

```
● ● ● TERMINAL

./furive release test lotte-ohmio-1 --date 2026-08-31 --number 1 --run
```

서명된 `test` 배포 버전 발행이 성공한 뒤 시험 차량 설치 묶음을 만듭니다.

TERMINAL

```
./furive bundle lotte-ohmio-1
```

사용 설명서도 함께 전달할 때만 `--include-manual` 을 붙입니다. 내부 개발 PC용 `./furive bundle development` 에는 설명서가 기본으로 들어갑니다.

생성된 차량 묶음 하나를 정비 시간에 각 PC에서 `./bootstrap.sh` 로 적용합니다. 신규 PC만 장비를 선택하고 기존 설치는 저장된 배정을 유지합니다. 설치 흐름은 [차량 PC 설치](#)를 따릅니다.

접근 정보, 지도 또는 기존 배포 버전 선택처럼 설치 묶음만 바뀌었다면 새 배포 버전을 만들지 않습니다. 기존 서명 버전을 유지한 새 설치 묶음을 만들고 한 차량에서 전달 결과만 확인합니다.

3. 차량에서 기능과 복구 확인하기

main 운영 화면은 로그인 뒤 자동으로 열립니다. 업데이트를 준비·적용한 뒤 다음을 실제 결과로 확인합니다.

- 화면에 표시된 목표 버전과 실행 중인 버전이 일치한다.
- 바꾼 기능의 정상 입력이 기대한 ROS-API·파일·화면·장치 결과를 만든다.
- 대표 실패 입력이 정한 실패 상태를 만든다.
- main과 edge의 필수 프로그램이 정상 상태를 회복한다.
- 재시작 뒤에도 같은 결과가 유지된다.

프로세스나 container가 살아 있다는 사실만으로 기능 PASS를 기록하지 않습니다. 문제가 있을 때만 터미널에서 다음 순서로 확인합니다.

TERMINAL

```
furive status
furive service logs
```

운영 화면을 닫았을 때만 `furive dashboard` 로 다시 엽니다.

시험이 끝나면 운영 화면의 이전 정상 버전 복구를 적용합니다. 새 버전은 현재 버전과 떨어진 곳에 준비되고, 다음 시작에서 전환됩니다. 필수 프로그램이 안정된 정상 상태가 되지 않으면 Furive가 업데이트 직전 전체 배포 버전으로 자동 복구합니다.

복구 뒤에는 다음을 다시 확인합니다.

- 버전이 시험 전 `normal` 로 돌아왔다.
- 핵심 기능과 main-edge 연결이 정상이다.
- 실패한 test가 자동으로 다시 적용되지 않는다.

내부 준비 폴더와 변경 기록을 조사할 때만 [이전 정상 버전 복구 기준](#)과 [업데이트 호환 정책](#)을 봅니다.

4. 확인한 변경만 normal로 승격하기

차량 시험을 마친 코드를 공용 개발 저장소에 반영한 뒤 차량 배정을 `normal` 로 되돌리고 발행 계획을 확인합니다.

● ● ● TERMINAL

```
./furive vehicle set lotte-ohmio-1 --update normal --version latest
./furive release normal --date 2026-08-31 --number 2
```

계획에서 시험을 통과한 플랫폼만 새 승인판으로 이동하고, 시험하지 않은 플랫폼은 이전 승인판을 유지하는지 확인합니다. 맞으면 발행합니다.

● ● ● TERMINAL

```
./furive release normal --date 2026-08-31 --number 2 --run
```

이미 `normal latest` 로 설치된 일반 차량은 새 공용판을 원격으로 찾으므로 배포 버전마다 설치 묶음을 다시 설치하지 않습니다. 시험 차량에는 `test` 설치 묶음이 전달한 차량 배정과 시험용 접근 정보가 남을 수 있으므로, `normal` 운영으로 돌릴 때는 공용판 발행 뒤 새 설치 묶음을 만들어 적용합니다.

● ● ● TERMINAL

```
./furive bundle lotte-ohmio-1
```

한 번 발행한 버전 번호를 다른 소스 코드에 다시 사용하지 않습니다. `normal` 적용 뒤 운영 화면 또는 `furive status` 에서 목표 버전과 프로그램 상태를 확인합니다.

완료 조건

- 변경 범위 테스트와 `furive check` 가 통과했다.
- 한 차량의 `test` 에서 정상·대표 실패 결과를 확인했다.
- 시험 전 `normal` 로 복구해 기능을 다시 확인했다.
- 배포 버전과 설치 묶음을 실제 변경 내용으로 구분했다.
- 시험한 플랫폼만 `normal` 새 승인판으로 이동했다.
- test 차량의 설치 상태도 `normal bundle`로 되돌렸다.

단계별 확인 절차는 [변경 검증](#), 배포 버전 발행과 보존 세부사항은 [CI/CD 기준](#)을 봅니다.

PART V

시스템과 계약 참고

기본 작업 경로 밖에서 host 영향, 구현 소유권, package·launcher·OTA·CI/CD·보안 계약을 조사할 때 찾는 참고편입니다.

09	설치가 PC에 미치는 영향 · p.52	10	Furive OS 기술 구조 · p.55
11	실행 구성요소 · p.69	12	프로그램 연결 설정 (package.yaml) · p.77
13	Autoware 플랫폼 탐색 · p.87	14	이전 정상 버전 복구 기준 · p.90
15	업데이트 호환판과 구버전 설정 변환 · p.95	16	CI/CD 기준 · p.98
17	보안·개인정보 기준 · p.122		

09. 설치가 PC에 미치는 영향

이 장의 목적 · 설치 검토자 · 장애 조사자

기본 설치 영향과 선택 기능의 host-wide 영향을 분리해 판단합니다.

완료 판단

영향 수준·조건·확인 명령을 실제 대상과 대조합니다.

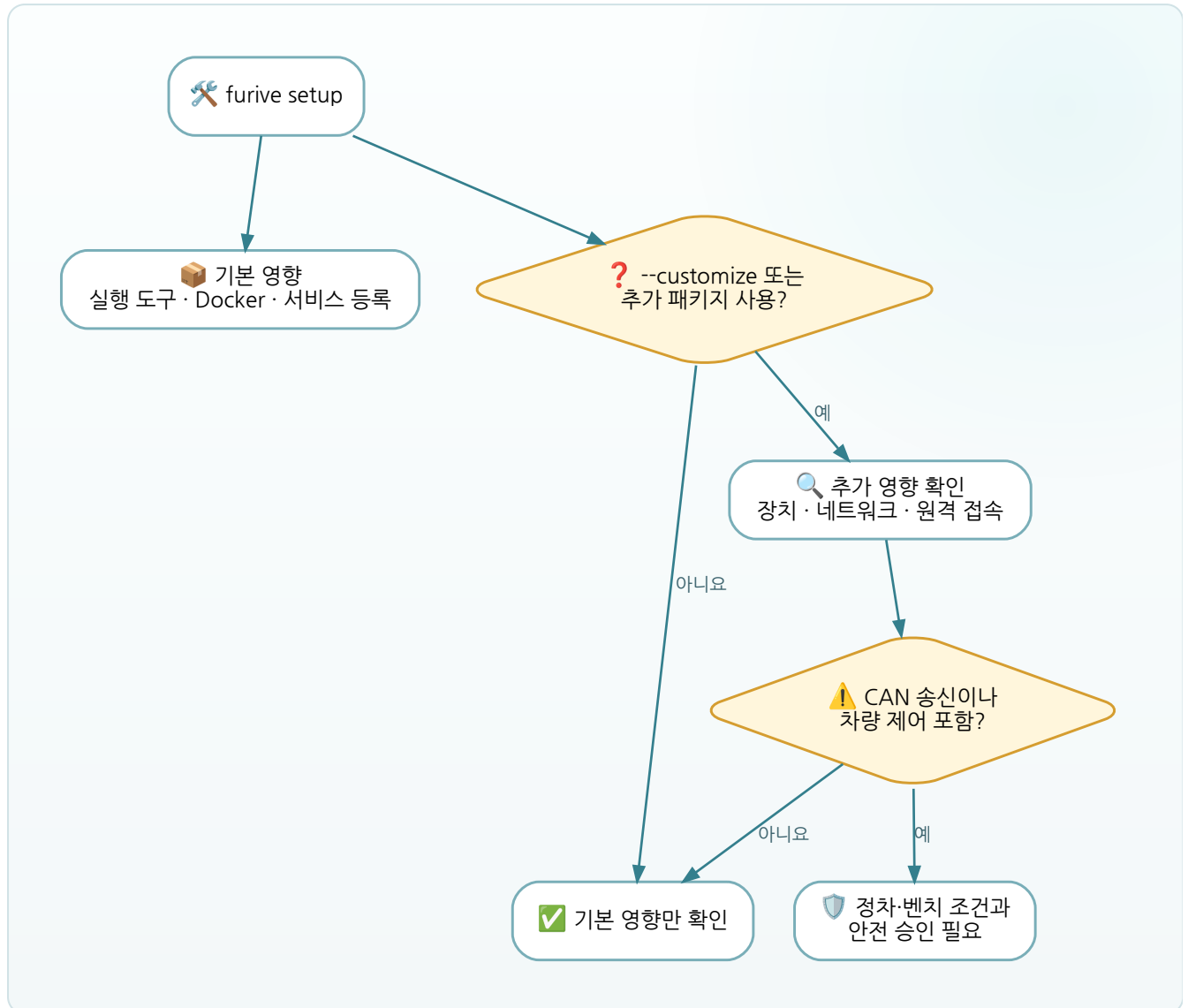
이 문서는 Furive OS를 설치하기 전에 PC 전체에 남는 변경과 차량·네트워크에 영향을 줄 수 있는 선택 기능을 확인할 때 사용합니다. 일반 설치 전에는 이 요약까지만 읽으면 됩니다.

필요한 만큼만 읽기

읽는 사람	여기까지만 보면 됨
처음 설치하는 사람	설치 전 요약과 빠른 판단
패키지를 추가하는 개발자	요약을 본 뒤 세부 목록에서 패키지 이름 검색
설치·권한·네트워크 문제를 조사하는 담당자	세부 목록에서 영향 태그와 실제 확인 명령 확인

기본 설치가 PC에 남기는 변경은 `install.sh`, 특정 패키지의 추가 영향은 그 패키지 저장소가 담당합니다. 실행 중 포트와 프로그램 시작·종료는 공통 실행 관리자가 맡습니다.

설치 전 요약



기본 설치의 다음 항목을 바꿀 수 있습니다.

바꿀 수 있는 것	이유	설치 후에도 남는가?
Python, Node.js, Docker	Furive와 패키지 실행	예
Docker 설정	Docker 이미지 다운로드를 위한 DNS 설정	예
chrony와 시간 동기화 설정	인터넷 유무와 PC 연결 상태에 따라 점진적으로 시각 보정. 정책·권한·검증 범위	예
main/edge 서비스와 docker / video / render 그룹	부팅 후 자동 실행과 선택된 GUI의 Docker/DRM 접근	예
main 운영 화면	기본 설치가 데스크톱 로그인 자동 시작을 등록한다. 화면을 닫았을 때만 furive dashboard 로 다시 연다.	예
/etc/furive-os 접근 정보	장비와 업데이트 확인	예

기본 설치만으로 모든 차량 패키지가 설치·실행되는 것은 아닙니다. 다음 영향은 해당 옵션이나 패키지를 선택하고 실제로 실행할 때만 생길 수 있습니다.

선택 기능	가능한 영향	확인할 때
Docker 패키지	포트·이름 충돌, X11 접근	패키지 추가 전
멀티캐스트	네트워크 경로와 NIC 설정 변경	<code>--customize</code> 사용 전
Autoware·CAN 패키지	CAN 연결 장치 설정, 직접 컨 경우 실제 송신	정차·벤치 시험 전
카메라 패키지	NIC down/up 또는 MTU 변경	실제 장치 연결 전
데이터 로깅	접근 정보, 방화벽, 저장·외부 전송	고객 데이터 사용 전
원격 접속	서비스, 로그인 화면과 sudo 권한 변경	기능 활성화 전
차량 <code>test</code> 용 Git 쓰기 키	실제 시험 차량의 main/edge에만 설치	고객 전달 전과 <code>normal</code> 전환 전

설치 전 빠른 판단

- 다른 프로그램이 같은 포트, Docker 이름 또는 ROS domain을 쓰는지 확인합니다.
- Docker 그룹에 들어간 사용자는 높은 권한을 가질 수 있으므로 설치 계정을 제한합니다.
- CAN 송신, NIC 변경과 원격 접속은 기본 기능으로 간주하지 말고 별도 시험·승인을 거칩니다.
- 실제 위치·영상·고객 데이터를 쓰면 [보안·개인정보 기준](#)을 추가로 확인합니다.

여기까지 확인했으면 일반 설치 전 검토는 끝입니다. 아래 내용은 충돌을 조사하거나 특정 패키지의 정확한 파일·명령·환경변수를 찾을 때만 펼칩니다.

► Details

10. Furive OS 기술 구조

이 장의 목적 · 공통 기반 유지보수자

무엇을 바꿀 때 어느 소유자에서 시작해야 하는지 찾습니다.

완료 판단

같은 상태를 둘 이상의 모듈에 새로 만들지 않고 변경 위치를 정합니다.

이 문서는 Furive OS 내부를 수정하거나 장애 원인을 추적하는 개발자를 위한 지도다. 처음 사용하는 사람은 [루트 README](#)에서 시작하고, 실제 작업은 [문서 안내](#)에서 해당 가이드 하나를 선택한다.

이 문서의 앞부분만 읽어도 다음 질문에 답할 수 있어야 한다.

- Furive OS와 각 기능 패키지가 각각 무엇을 맡는가?
- PC 한 대에 함께 실행하거나 A-ECU·HMI-ECU로 나눌 때 무엇이 달라지는가?
- 패키지는 누가 발견하고 실행하며 상태를 확인하는가?
- 패키지 메시지, ROS2 데이터, 업데이트는 각각 어느 경로로 흐르는가?
- 기능을 바꾸려면 어느 파일부터 확인해야 하는가?

코드와 설정에는 호환을 위해 `master`, `agent`, `capability_profile` 이 남아 있다. 일반 사용자는 플랫폼이 정한 장비 이름과 플랫폼이라는 말을 쓴다. `main / edge` 는 source 개발·자동화에서 쓰는 운영 역할이며 설치 뒤 명령에 반복 입력하지 않는다. 내부 이름은 해당 데이터 형식이나 파일을 설명할 때만 표시한다.

1. 한 장으로 보는 구조

아래 main/edge는 공통 코드의 운영 역할이다. 일반 설치와 장비 표시는 플랫폼이 정한 A-ECU/HMI-ECU 등의 이름을 사용한다. 단일 묶음의 최초 PC 선택, 기존 배정 유지와 자율주행 연산 보호 기준은 [ECU 설치 설계](#)에 정리한다. 기존 통신·인증 역할 이름은 유지한다.

처음 알아둘 세 가지

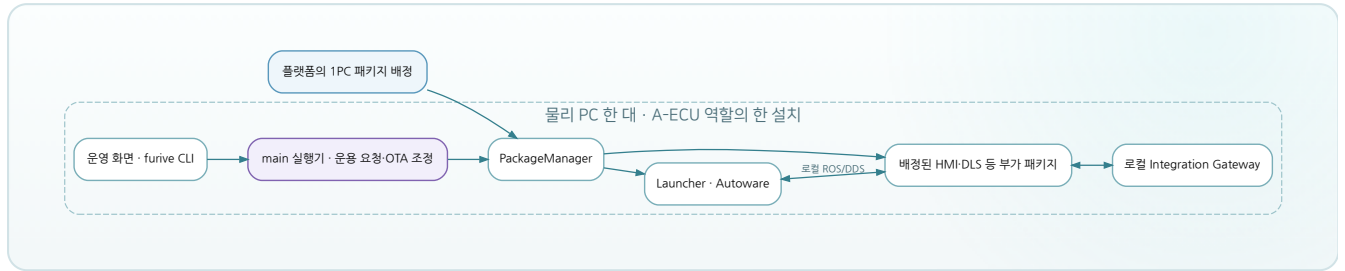
개념	뜻
패키지	독립 프로그램을 Furive가 찾고 실행하고 상태와 로그를 확인할 수 있게 연결한 단위다.
플랫폼	차량 모델, 센서 모델과 그 차량에서 사용할 패키지의 조합이다.
설치할 PC	한 PC에 필요한 패키지를 함께 배정하거나 A-ECU의 자율주행과 HMI-ECU의 부가 기능으로 분산한다.

[한눈에 보기](#)와 같은 개념이다. 아래 main/edge 도식은 이 배정을 실행하는 내부 역할을 보여준다. `normal / test` 는 별도 PC 역할이 아니라 공용판과 한 차량 시험판을 구분하는 업데이트 이름이며 [업데이트 정책](#)을 따른다.

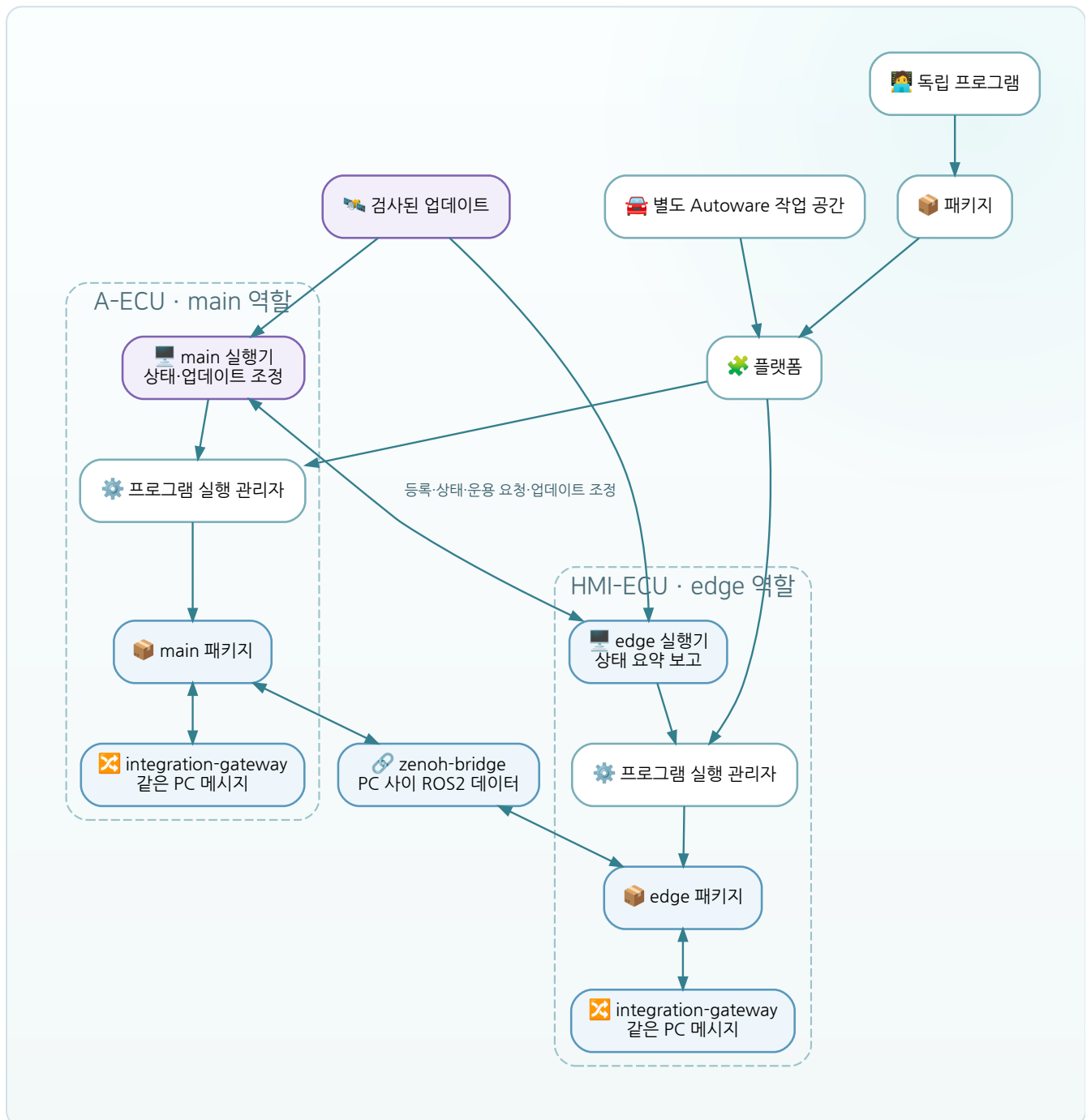
Autoware package나 ROS package는 여기서 말하는 패키지와 다르다. Autoware 소스는 별도 저장소와 작업 공간에서 개발한다. Furive OS는 검증된 소스 버전 선택과 Autoware 시작·종료만 맡는다.

구성요소 관계

1PC와 분산 구성 모두 같은 패키지·설치·실행·OTA 기반을 사용한다. 아래 첫 그림은 필요한 기능을 한 PC에 함께 배정하는 구성이다. 실제 지원 조합과 현재 CLI 범위는 ECU 설치 설계를 따른다.



분산 엣지 컴퓨팅에서는 자율주행·기본 운영과 로깅·부가 기능을 별도 PC에 배정한다.



각 PC에는 프로그램 실행 관리자(PackageManager)가 하나씩 있다. 이 관리자는 선택된 패키지와 먼저 필요한 패키지를 찾아 시작·중지·상태 확인·재시작을 맡는다. 각 기능의 상태와 규칙은 해당 패키지가 맡는다.

연결 정책은 플랫폼 한 곳에서 정의하고 실제 전달은 각 PC가 실행한다. 같은 목적의 통신은 기존 경로로 모으며, 현재는 ROS/DDS 연동과 로컬 전달 범위의 차이 때문에 다음 전송 adapter를 사용한다.

- main/edge 제어 연결은 PC 등록, 상태 요약, 업데이트 조정과 정의된 운용 요청에 쓴다.
- `integration-gateway` 는 같은 PC의 패키지 사이 요청·응답·이벤트만 전달한다.
- `zenoh-bridge` 는 main과 edge 사이 ROS2 데이터를 전달한다.

현재 `integration-gateway` 끼리 main과 edge를 연결하는 기능은 구현되어 있지 않다. 패키지 이름이나 양쪽 역할에서 실행 가능하다는 설명 파일만 보고 PC 사이 전달이 된다고 가정하면 안 된다.

각 상태를 한 곳에서만 담당하기

상태·규칙·실행 흐름	담당 프로그램	담당하지 않는 곳
패키지 시작·중지·상태 확인·재시작	각 PC의 PackageManager	패키지별 자체 재시작, Dashboard, CLI
main/edge 등록과 호스트 상태 요약	main/edge 실행기	기능 패키지, <code>integration-gateway</code>
패키지 사이 전달과 요청 ID 연결	<code>integration-gateway</code>	main/edge 실행기, Dashboard
구버전 HMI용 임시 상태 변환	<code>hmi_dls_legacy.py</code>	원본 상태, 녹화·전송 규칙
녹화·세션·NAS 전송 상태와 규칙	<code>data-logging-system</code>	메시지 통로, main/edge 실행기, 운영 화면
Autoware 시작·중지와 실행 선택	<code>furive-launcher</code>	Dashboard, PackageManager
ROS2 데이터 전달	<code>zenoh-bridge</code>	main/edge 제어 연결, gateway
업데이트 정책·목표·준비·적용·복구	하나의 OTA 흐름	개별 패키지, Dashboard
표시와 사용자 입력 전달	운영 화면·CLI	패키지·OTA·Autoware의 실제 기능 상태
중앙 ROS 연결 정책과 파생 설정	플랫폼 해석기·중앙 토픽 관리	개별 bridge의 중복 정책, DLS 기록 선택
설치한 PC 역할	<code>setup</code> 이 만든 설치 상태	일반 명령의 반복 인자

`integration-gateway` 는 수신자와 요청 번호를 보고 메시지를 전달한다. 녹화 상태의 원본과 전송 규칙은 `data-logging-system` 이 담당한다. 구버전 HMI의 임시 응답 변환 범위와 제거 조건은 [패키지 계약의 책임·권한 경계](#)에서 정의한다.

원격 업데이트도 하나의 흐름입니다. 허용 규칙, 목표 선택과 별도 폴더 준비, 폴더 교환·확정·복구, 운영 화면 연결로 코드만 나뉘며 서로 다른 업데이트 방식은 아닙니다.

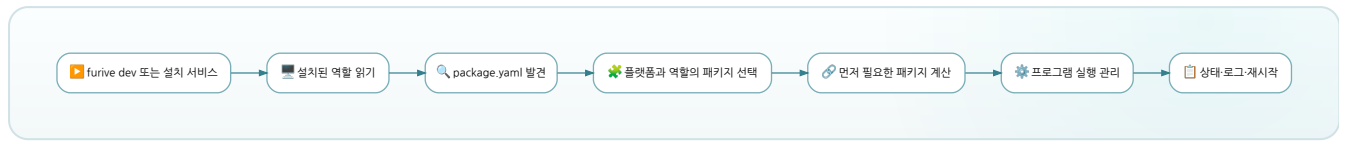
무엇을 바꿀 때 어디서 시작하는가

공통 소스 안내에서 기능별 시작 파일, 연결 파일, 설정과 읽는 순서를 찾는다. 폴더 전체 배치는 [루트 지도](#), 책임 경계는 [개발 계약의 수정 위치](#)를 기준으로 한다. 화면과 프로그램 자체 기능은 해당 `package/` 저장소가 담당한다.

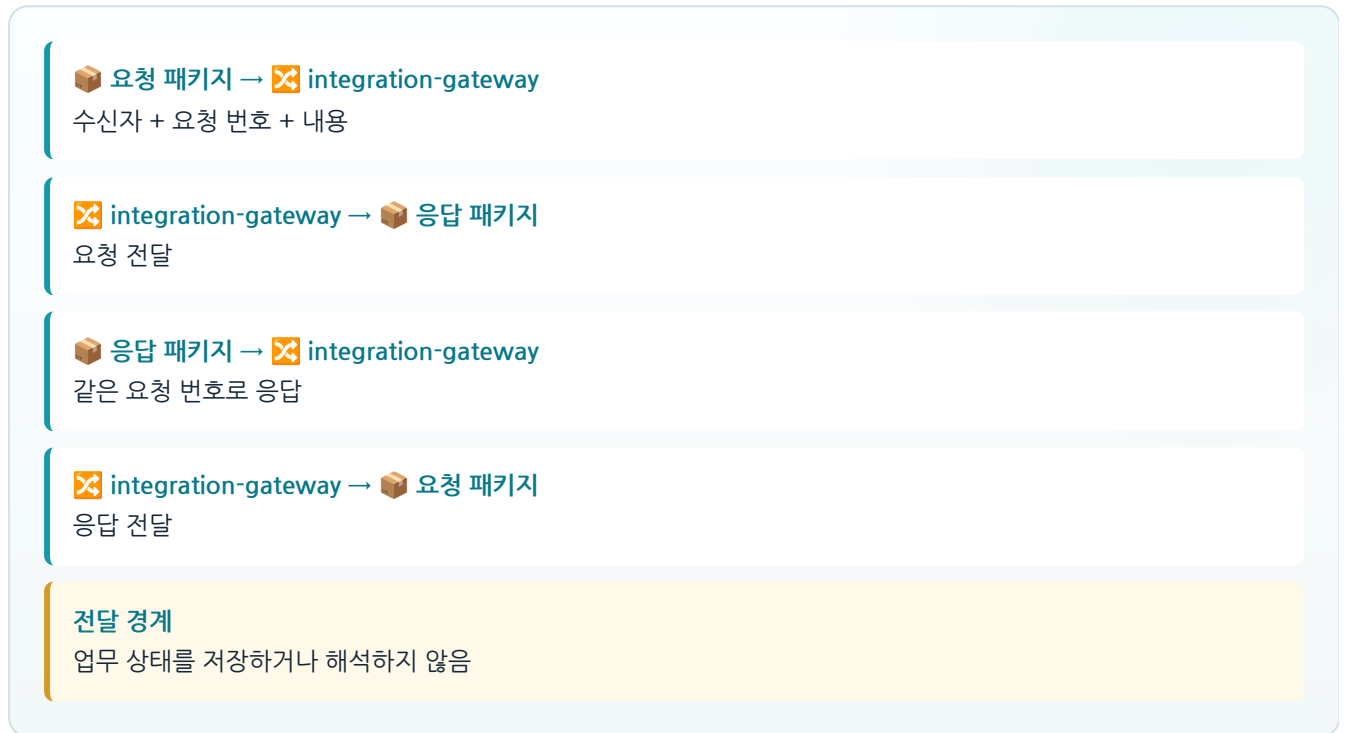
새 보조 코드, 서비스, 설정 항목을 만들기 전에 기존 담당 코드가 처리할 수 있는지 확인한다. 같은 값이나 시작·종료 상태를 두 곳에 저장하지 않는다.

대표 흐름 세 가지

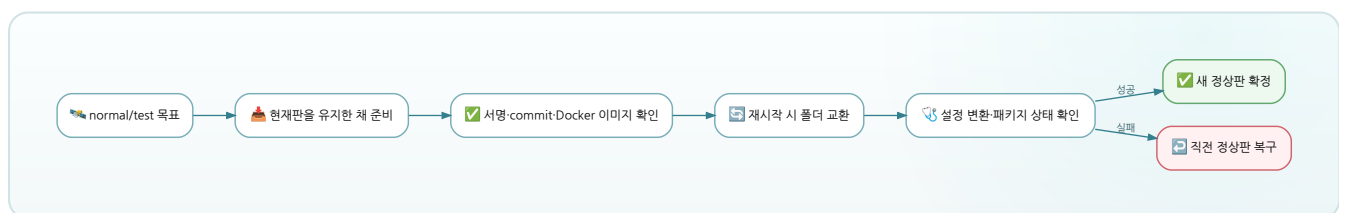
패키지 실행:



같은 PC의 요청:



업데이트:



2. 처음부터 실행까지

공개 작업 경로

차량 PC의 정상 진입점은 전달받은 설치 묶음이다.



설치가 끝나면 main/edge 서비스는 부팅 때 자동으로 시작되고 main 운영 화면은 로그인 뒤 자동으로 열린다. 평소시에는 별도 시작 명령을 호출하지 않는다.

소스 코드 작업 폴더를 직접 수정하는 개발자는 저장소 최상위 폴더에서 현재 코드를 실행한다.

```

    ./.furive dev main

    ./.furive check

```

edge 전용 코드를 다룰 때만 `main` 을 `edge` 로 바꾼다. 영구 설치를 마친 개발 PC에서는 저장된 역할을 자동으로 사용하므로 `furive dev` 만 실행할 수 있다. 독립 프로그램을 연결하는 단계에서는 다음 흐름을 사용한다.

```

    furive package add <repository-url>
    furive package run <name>
    furive package check

```

설치된 차량에서 터미널은 문제가 있을 때만 사용한다.

```

    furive status
    furive service logs

```

다른 main 운영 화면을 다시 열 때만 `furive dashboard` 를 사용한다.

시작 내부 흐름

공개 명령 아래에서는 다음 순서로 동작한다.

1. 설치 상태에서 main 또는 edge 역할을 읽는다.
2. pending update가 있으면 적용 가능 여부를 확인한다.
3. runtime asset과 역할별 설정을 준비한다.
4. 해당 node runtime을 시작한다.
5. PackageManager가 manifest를 발견하고 플랫폼이 선택한 package 목록을 읽는다.
6. dependency closure를 계산해 의존 패키지부터 시작한다.
7. PackageManager와 공통 watchdog가 health를 확인하고 필요한 재시작을 수행한다.
8. main/edge node는 호스트-패키지 상태 요약과 정의된 운용 요청-결과를 교환한다. 업무 원본 상태는 패키지에 남긴다.

내부 호환 이름 때문에 구현 파일은 `nodes/master.py` , `nodes/agent.py` , 설정은 `config/master.default.yaml` , `config/agent.default.yaml` 이다. 이 파일 이름을 새 사용자 명령이나 일반 문서의 역할명으로 확장하지 않는다. `run.sh` , `install.sh` , `scripts/*` 도 공개 진입점이 아니라 CLI, runtime, CI가 사용하는 내부 구현이다.

개발 실행과 설치 서비스

`furive dev` 는 개발 세션 동안 OTA hold와 backend lifecycle을 소유한다. 종료하면 이전 hold와 설치 서비스 상태를 복원한다. main에서는 backend와 Electron 표시 계층을 분리해 실행한다. 브라우저가 필요할 때만 `furive dev --browser` , 화면이 필요 없을 때는 `furive dev --no-ui` 를 사용한다.

설치 후 main backend는 `furive-os-master.service` , edge backend는 `furive-os-agent.service` 라는 내부 systemd unit이 소유한다. Electron autostart는 기존 backend에 연결하는 표시 계층일 뿐 backend lifecycle을 소유하지 않는다.

Docker package는 기본적으로 `restart=no` 로 실행한다. container와 process의 health·restart는 모두 PackageManager가 맡는다. script·colcon·binary package의 stdout/stderr는 `logs/<package>.log` 에 기록된다. package 자체에 restart loop나 다른 package 제어를 넣지 않는다.

3. 패키지 모델

독립 저장소의 프로그램은 이름·버전과 `run.sh` 만 알려주면 Furive가 시작·중지·상태·로그를 관리한다. 이 운용 단위를 **패키지**라고 한다.

YAML · CONFIG

```
name: example
version: 0.1.0
```



PackageManager는 등록된 저장소에서 패키지를 찾고 현재 PC 역할과 플랫폼이 선택한 것만 실행한다. Docker·장치·의존성·특수 상태 확인처럼 기본과 다른 요구만 `package.yaml` 에 추가한다. 프로그램의 언어·빌드·기능 상태는 해당 저장소가, 공통 실행 순서와 재시작은 PackageManager가 한 번만 소유한다.

실제 연결 순서는 [프로그램 연결 가이드](#), 모든 필드·발견·실행·권한 규칙은 [프로그램 연결 설정](#)이 소유한다. 배포 산출물 경계는 [CI/CD 기준](#)을 따른다.

4. 세 가지 통신 경로

main/edge node 제어 연결

main과 edge는 PC 등록, heartbeat, package-host 상태 요약과 업데이트 조정 정보를 교환한다. 인증된 같은 연결로 명시적으로 지원하는 운용 요청도 전달한다. 예를 들어 HMI-ECU에서 요청한 Autoware 작업은 A-ECU의 담당 기능으로 전달하고, 녹화 요청은 DLS가 배정된 PC로 전달한다. node는 요청과 결과를 연결하며 기능 package의 세션·업무 상태를 복제하지 않는다. 임의의 package 메시지를 전달하는 범용 통로는 아니며, [명령과 기능 상태의 소유권](#)을 따른다.

PC 시간 표본도 이 인증된 heartbeat에 포함된다. `core/time_sync.py` 가 표본의 지연·유효성을 검사해 호스트 chrony로 전달하며, 실제 시각 보정은 chrony가 소유한다. `furive system status` 와 `furive status` 는 외부 기준·차량 내부 기준·보정 중·미확인을 구분한다. 1PC는 그 호스트의 시계를 사용한다. 설치 적용 범위와 점진 보정·재연결 기준은 [PC 시간 동기화](#)를 따른다.

Autoware 실행 상태는 launcher가 소유한다. main은 heartbeat 응답에 launcher의 `runtime` 조회 결과만 담고, edge는 기존 공유 상태 파일에 수신 시각과 함께 전달한다. edge의 로컬 Dashboard는 이 결과가 heartbeat 유효 시간 안에 있을 때만 실행

상태 API로 제공한다. 원격 Dashboard HTTP는 loopback 전용이므로 이 전달이 필요하며, 새 포트·설정·사용자 명령은 없다. 조회 실패나 연결 단절로 오래된 실행 상태를 정상으로 표시하지 않는다.

표준 2PC에서 main의 TCP 15001은 차량망에 bind하고 TLS 1.3 상호 인증으로 차량·역할·node 신원을 확인한다. edge가 광고한 protocol 범위에서 직전 또는 현재 protocol만 협상한다. 오래된 config에 identity나 bind 항목이 없더라도 외부 비인증 연결로 낮추지 않는다. 인증 계약이 없는 pre-epoch 장비는 정차된 유지보수에서 역할별 credential bundle로 edge, main 순서로 등록한다.

각 역할 bundle에는 자기 private key와 상대 leaf pin만 둔다. /etc/furive-os/control.env의 token은 같은 host의 loopback Dashboard·OTA 제어에만 사용하며 PC 사이에 공유하지 않는다. PackageManager는 기본적으로 이 credential을 package 환경에서 제거한다. manifest에 control.credentials: true를 선언한 Dashboard, OTA owner, launcher 같은 trust duty만 받을 수 있다.

같은 PC의 integration-gateway

현재 gateway는 각 PC의 127.0.0.1:15003에서만 요청·응답·이벤트를 전달한다. 필요한 package의 dependency로 자동 시작되므로 사용자용 설치 선택, CLI, endpoint config가 없다. HMI, DLS와 Autonomy Studio 연계 package가 이 경로를 사용한다.

구버전 HMI/DLS 메시지는 경계의 legacy adapter에서 현재 형식으로 변환한다. 이 adapter가 허용하는 임시 상태와 제거 조건은 **패키지 계약**을 따른다.

현재 제한은 다음과 같다.

- main gateway와 edge gateway 사이 routing은 **미구현**이다.
- loopback은 차량 밖 접근을 막지만 같은 PC process identity를 인증하지는 않는다.
- gateway는 message durability, 업무 retry, schema registry나 domain state store가 아니다.
- legacy adapter 밖의 새 domain projection이나 command 해석은 허용하지 않는다.

PC 사이 package 메시지가 필요해져도 현재 node control 연결이나 Zenoh ROS2 경로에 임의로 섞어서는 안 된다. 그것은 현재 기능이 아니라 별도 보안·routing 계약이 필요한 후속 변경이다.

ROS2의 zenoh-bridge

ROS2 데이터는 zenoh-bridge가 전달한다. 표준 2PC에서 main은 QUIC/UDP 7447을 listen하고 edge가 연결한다. TLS 1.3 mTLS와 peer-CN 기반 default-deny ACL을 사용한다.

main의 Autoware domain과 Zenoh 쪽 shadow domain은 ros2-domain-bridge가 분리한다. 232는 Furive 내부 shadow domain 예약값이며 일반 ROS2 app의 ROS_DOMAIN_ID로 사용하지 않는다. ROS domain을 바꾸면 두 bridge의 endpoint와 재시작 상태를 함께 확인한다.

Dashboard 경계

Dashboard는 상태를 표시하고 사용자 입력을 소유자에게 전달하는 adapter다. package, OTA, Autoware lifecycle을 직접 소유하지 않는다. HTTP는 localhost 18082에 bind하며 control token을 요구한다.

package dashboard는 main Dashboard와 같은 권한으로 실행하지 않는다. package frame은 allow-same-origin 없는 opaque-origin sandbox에서 열고, package별 capability에 허용된 method/path/query만 proxy한다. package HTML을 top-level 문서로 직접 열어 이 경계를 우회하지 않는다. 로컬 HDMI capture viewer는 media API 때문에 control API가 없는 별도 loopback origin을 사용한다.

Furive OS 전체 데이터 교환 정책

[!TIP] 운용 절차는 **토픽 명령어 표와 흐름도**에서 확인합니다. 아래는 중앙 전달 설정과 패키지 기능을 수정할 때 따르는 구현 계약입니다.

Furive OS는 ECU 사이의 통신 설정을 소유하고, 독립 패키지는 기능과 업무 상태를 소유한다. **DLS에 기록 토픽을 추가해도 통신 허용 범위가 늘지 않는다.** DLS는 Furive OS 없이도 자신의 ROS 환경에서 토픽을 조회하고 녹화한다. Furive OS에 연결했을 때 중앙 관리는 ECU 간 전달에만 적용된다.

중앙 설정과 실제 전달

ROS 연결 정의는 `config/capability-profiles.yaml`의 `connections.ros2` 한 곳에 있다. 공통 기본값과 플랫폼별 `connections`를 기존 플랫폼 해석기가 병합한다. 사용자가 `furive topic add` 또는 통신 토픽 화면에서 바꾼 값은 중앙 PC의 기존 `config/master.yaml`에 차이만 저장한다. 수신 PC에 같은 정책을 편집하는 파일을 만들지 않는다.

원본 값	뜻	적용 대상
<code>exclude</code>	전달에서 제외할 이름 패턴	Domain Bridge 경로 생성
<code>topics</code>	전달할 토픽 이름 또는 이름 패턴, 메시지 타입, 방향과 필요한 QoS	두 bridge의 경로와 자동 발견 필터
<code>services</code>	정확한 서비스 이름과 타입	기존 서비스 proxy와 Zenoh 필터

기본 설정은 와일드카드 없이 실제 토픽을 `topics`에 한 번씩 명시한다. 같은 경로 아래에 새 토픽이 생겨도 자동으로 전달하지 않으며, `furive topic add /이름`으로 추가한다. `type: auto`는 적힌 이름의 메시지 타입만 발견하며 전달 범위를 넓히지 않는다. 타입을 명시하면 발행 전에도 경로를 준비한다. 기본 방향은 차량에서 수신 PC로 전달이며, 역방향에는 `reversed: true`, 양방향에는 `bidirectional: true`와 정확한 타입을 명시한다. `receive`라는 별도 목록은 없다.

YAML · CONFIG

```
connections:
  ros2:
    topics:
      /planning/scenario_planning/current_max_velocity:
        type: tier4_planning_msgs/msg/VelocityLimit
      /tf:
        type: auto
```

토픽·서비스의 `null`은 상속된 항목을 회수한다. 이름 패턴은 정확한 이름 또는 `/경로/.*`로 제한해 두 bridge가 같은 문법을 해석한다. 역방향·양방향 차량 명령은 플랫폼 계약에서 관리하며 녹화용 `topic add/remove`로 바꿀 수 없다. 새로운 메시지 타입이 설치되어 있지 않으면 패키지 업데이트가 먼저 필요하다. 기존 타입의 토픽 추가에는 이미지 재빌드나 설치 묶음 재생성이 필요 없다.

`core/capability_profile.py`가 정책 버전과 두 bridge의 설정을 함께 생성한다. PackageManager가 실행 환경으로 전달하고, bridge는 이를 받아 설정 파일을 만든다. 이전 package config의 토픽 목록은 새 권한을 부여하지 않는다. 중앙 설정이 누락됐을 때 전체 허용으로 대체하지 않는다.



목록에서 녹화까지

1. `furive topic list`는 중앙 설정 토픽만 보여준다. `furive topic list --all`은 차량 원본 ROS 영역에서 발행 중인 미등록·미수신 토픽도 포함한다. 타입, 중앙 등록 여부, 수신 PC의 publisher 발견 상태와 조회 시각을 표시한다. 꺼진 프로그램의 미등록 토픽은 발견할 수 없으며, 목록 조회는 payload 구독이나 전달 설정 변경을 하지 않는다. 이 관측은 실제 데이터 수신이나 파일 저장의 증거가 아니다.
2. `furive topic add /이름`은 발행 중인 토픽의 타입을 조회해 중앙 설정을 만든다. 양쪽 PC의 타입 지원과 설정 계약을 검사한다. Autoware나 DLS가 실행·녹화 중이면 설정만 저장하고 **적용 대기**로 표시한다. 토픽이 사라진 뒤 다시 이름과 타입을 입력할 필요가 없다.
3. 정비 시 Autoware와 녹화를 중지하고 `furive topic apply`로 반영한다. 이미 중지되어 있으면 add/remove가 바로 적용한다. 재시작은 PackageManager가 소유하며, 실행 중인 bridge의 설정 버전을 확인한다. 부분 실패 시 기존 양쪽 전달 설정으로 복구를 시도하고 복구 실패도 응답에 포함한다.
4. 수신 PC에서 토픽이 조회되면 DLS의 기록 목록에서 선택한다. DLS 화면의 “토픽이 목록에 없나요?”는 Furive OS 중앙 화면을 열어 주는 탐색 기능이다. DLS iframe에는 중앙 정책 변경 API를 허용하지 않는다. 독립 실행에는 이 안내를 표시하지 않는다.
5. `furive topic record /이름`은 기존 DLS 기록 선택에 토픽을 추가해 수동 녹화를 시작한다. `furive topic stop`은 DLS 녹화를 멈춘다. 통신 설정은 바꾸지 않으며 DLS 선택·세션이 유일한 녹화 상태다. DLS에서 선택 해제하거나 녹화를 멈춰도 다른 소비자가 사용하는 중앙 전달 경로는 유지된다.
6. 녹화 종료 후 DLS가 파일 검사를 접수한다. `furive topic sessions`로 세션을 찾고 `furive topic inspect <세션> --status`로 진행·무결성·토픽별 누락과 시간 공백을 확인한다. 접수·녹화 종료·검사 완료는 구분하며, 검사와 저장된 결과는 DLS가 소유한다. **녹화 검사 안내**의 지원 범위와 판정 한계를 따른다.

현재 중앙 전달 변경과 녹화·검사 CLI의 대상은 설치된 플랫폼의 중앙 PC와 연결된 수신 PC 한 대다. 1PC에 배정된 DLS의 로컬 녹화는 DLS 화면에서 사용한다. 재연결 시 중앙 설정 버전을 기존 인증된 node 연결로 전달한다. 구버전 bridge의 무응답이나 한쪽 PC의 연결 단절을 적용 성공으로 표시하지 않는다. 현재 구현은 토픽별 전송량 제한이나 실제 DDS payload 전달 완료로 증명하지 않는다. 실차에서는 양방향 제어 유지, QoS, 재연결, 대용량 부하와 실제 기록 파일을 확인한다.

명령과 기능 상태의 소유권

CLI와 Dashboard는 같은 `core/operator_control.py` 입력 adapter를 사용한다. 이 adapter는 기존 node 프로세스 안에 있으며 별도 서비스나 daemon을 만들지 않는다. PackageManager·launcher·OTA·DLS가 각자 실행·업데이트·녹화를 처리한다. main 화면이 Launcher뿐이어도 CLI는 Dashboard 실행에 의존하지 않는다.

기존 launcher/OTA 전용 socket은 node의 패키지와 host 기능을 다루지 못하므로, node에 checkout과 사용자별 private Unix socket을 추가했다. 새 지속 상태는 없다. 사용자는 `furive` 하위 명령만 배우며 socket 경로나 PC 역할을 다시 지정하지 않는다. 다른 PC의 Autoware·토픽·패키지 요청은 기존 mTLS node 연결에서 명시적으로 정의한 운용 명령만 전달한다. 임의 shell이나 업무 메시지 전달 API는 없다. 수신 PC에서 중앙 PC로 전달할 명령의 허용 목록은 `core/operator_control.py`가 한 번 정의하고, 양쪽 node가 같은 목록으로 송신·수신을 검사한다. 로컬 명령 지원과 원격 전달 허용은 구분한다.

중앙 설정 필드는 두 bridge의 중복 정책을 대체하며 플랫폼 해석기가 단독 소유한다. 실행 환경의 JSON과 bridge 설정 파일은 파생 산출물이다. 사용자는 `topic list → add → 수신 확인 → DLS 선택`을 사용하고 내부 필드를 반복 편집하지 않는다. 다른 전송 방식의 공통 schema는 실제 adapter 구현과 함께 확장한다.

현재 강제할 수 있는 경계

인증된 node와 개별 package는 다른 경계다. 같은 ROS domain을 공유하는 프로세스는 같은 데이터를 볼 수 있다. 로컬 제어 socket은 사용자 경계이며 같은 사용자로 실행한 모든 프로세스를 서로 격리하지 않는다. Domain Bridge의 제외 규칙이 로컬 ROS 접근이나 NAS 전송 권한까지 제한한다고 설명하지 않는다. 기록·재생·보존·NAS 정책은 DLS, package lifecycle은 PackageManager, OTA 정책은 OTA 소유자에 둔다. 화면·영상·기록 파일과 외부 전송에는 각 경로의 기존 인증·surface·egress 계약을 적용하며 이번 ROS 중앙 설정으로 모든 외부 전송을 통제한다고 주장하지 않는다.

5. 업데이트와 복구

사용자가 보는 정책

운용자는 `normal` 과 `test` 만 선택한다.

공개 이름	용도	내부 배포 표현
<code>normal</code>	여러 차량이 쓰는 공용판	<code>main channel</code>
<code>test</code>	지정한 차량 한 대의 임시 시험판	<code>preview/<vehicle_id></code>

차량별 목표 version은 fleet assignment 한 곳이 소유한다. 운용자가 Git branch, 내부 호환판이나 구버전 설정 변환을 직접 고르지 않는다.

내부 적용 흐름

1. `ota-updater` 가 현재 channel의 최신 canonical tag 또는 차량의 exact version을 최종 목표로 선택한다.
2. 현재판에서 목표판으로 직접 갈 수 없으면 release manifest에 결박된 bounded compatibility route를 검증한다.
3. 현재 실행본을 유지한 채 `/data/safe_staging/<checkout-key>/work` 에 source와 image를 준비한다.
4. source commit, release manifest, architecture별 image digest와 설치 산출물을 검증한다.
5. 후보를 `finalized` 로 만든 뒤 다음 시작에서 `ReleaseActivation` 이 switching journal을 먼저 쓰고 active와 candidate를 원자적으로 교환한다.
6. 호환판에 구버전 설정 변환이 있으면 패키지 시작 전에 실행한다. 설정 준비와 필수 package health가 성공하면 그 단계를 확정하고 최종 목표를 다시 계산한다.
7. 실패하면 직전 정상판과 이번 설정 준비가 바꾼 값만 복구한다. 동시에 사람이 바꾼 값은 CAS conflict로 보존한다.

staging 실패는 현재 실행본을 바꾸지 않는다. 교체 상태가 모호하거나 manifest·commit·digest가 일치하지 않으면 실행하지 않고 fail closed한다. 파일명 marker나 process exit code 하나만으로 release 성공을 판단하지 않는다.

한 설치로 운영하는 1PC에서는 main의 OTA가 로컬 배포판을 관리한다. 분산 구성에서는 main이 exact 목표를 고정하고 edge를 먼저 준비한다. edge의 fresh signed heartbeat가 목표 protocol, release tag, 전체 commit과 확정 상태를 보고한 뒤에만 main이 자기 후보를 준비한다. updater가 임의로 다른 PC를 재시작하거나 edge의 영구 channel/version 설정을 덮어쓰지 않는다.

호환판 선택, 설정 journal과 rollback의 전체 불변식은 업데이트 호환 정책과 이전 정상 버전 기준이 소유한다. release와 bundle 중 무엇이 필요한지는 CI/CD 릴리즈 판단을 따른다.

6. 저장소와 런타임 위치

소스 폴더 배치는 루트 지도에 모아 둔다. 공통 소스 안내에서 구현·설정 파일을, 스크립트 안내에서 검사·배포 내부 도구를 찾는다. `deploy-repo-template/` 은 차량용 생성 저장소의 템플릿이며 source 구현과 구분한다.

차량의 지속 runtime asset은 `/data/furive-os` 아래에 둔다. package별 config와 data는 가능하면 `/data/furive-os/<package>` 아래에서 그 package가 소유한다. source repo와 runtime data를 같은 상태 저장소처럼 사용하지 않는다.

찾으려는 정보	위치
현재 역할과 node package 목록	내부 <code>config/master.yaml</code> , <code>config/agent.yaml</code>
플랫폼별 package 조합	<code>config/capability-profiles.yaml</code>
Autoware OS/GPU/CUDA/driver/image 호환	<code>config/autoware-compatibility.yaml</code>
차량별 플랫폼 channel-version	<code>config/fleet-vehicles.yaml</code>
실제 발견된 package 목록	<code>logs/effective-packages-*.json</code>
package lifecycle 로그	<code>logs/<package>.log</code> , <code>logs/<package>.build.log</code>
차량 runtime asset과 data	<code>/data/furive-os/</code>
OTA staging	<code>/data/safe_staging/</code>

7. 현재 패키지 보기

패키지 목록을 문서에 복사하면 저장소가 추가·삭제될 때 바로 오래된 정보가 된다. 현재 checkout에서 Furive가 실제로 발견한 목록은 다음 명령이 한 번만 계산한다.

```

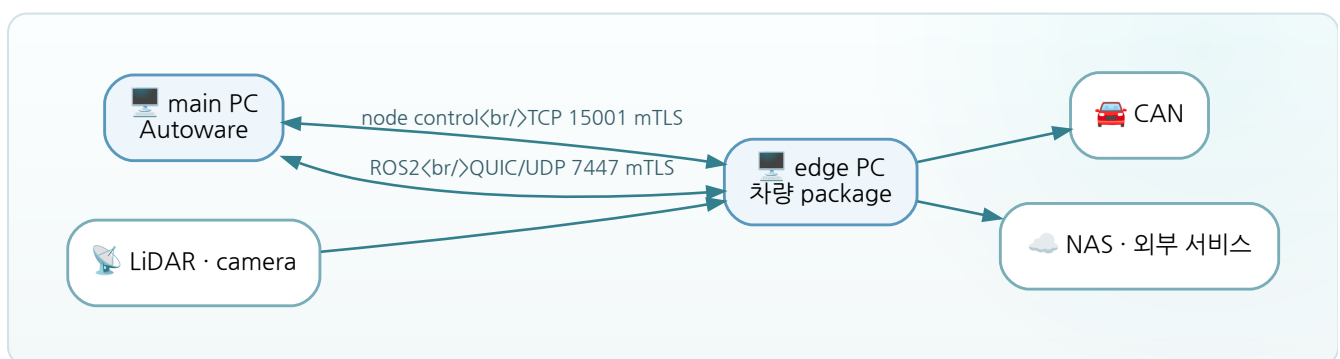
TERMINAL

./furive package list

```

차량에서 실행할 목록은 플랫폼과 PC 역할이 선택한다. 주요 운영 패키지의 역할은 **실행 구성요소**, 모든 `package.yaml` 필드는 **프로그램 연결 설정**을 기준으로 한다.

8. 차량 네트워크와 주요 포트



경로	현재 값	범위
main/edge node control	TCP 15001	main의 차량망 interface; mTLS

경로	현재 값	범위
node discovery	UDP 15002	차량망
package integration gateway	TCP 127.0.0.1:15003	각 PC의 loopback
Dashboard	HTTP 127.0.0.1:18082	loopback
Zenoh ROS2 bridge	QUIC/UDP 7447	차량망; mTLS와 ACL
RS80 LiDAR	UDP 6699/7788, multicast 239.192.1.2	sensor network
FLIR camera	UDP 50000, multicast 239.192.1.1	sensor network
NAS upload	HTTPS 443	외부 연결

표준 2PC 주소는 main 192.168.1.102, edge 192.168.1.103 이다. 내부 config의 local_ip 은 NIC 주소를 직접 바꾸지 않고 자동 탐색의 우선값으로만 사용한다. discovery가 실제 main 주소를 찾으면 edge node 연결과 Zenoh endpoint를 동기화한다. 시작·재연결 시 main 후보가 여러 개면 운영체제의 실제 송신 경로가 이더넷인 주소를 Wi-Fi 주소보다 먼저 선택한다. 이더넷으로 탐색된 후보가 없으면 Wi-Fi를 사용하며, 같은 경로 종류 안에서는 기존 선호 대역·저장된 주소 우선순위를 유지한다. 응답이 max(15초, heartbeat 간격 × 3) 동안 없으면 기존 연결을 정리하고 다시 탐색한다. 발견된 후보는 순위대로 연결·인증을 시도한다. Wi-Fi 대체 연결 중에도 같은 간격으로 더 좋은 경로를 탐색하고 새 경로의 상대 인증이 성공한 뒤 전환한다. 같은 등급의 경로 사이에서는 전환하지 않는다. 호스트 라우팅 테이블을 변경하지 않으며 정확한 복구 시간에는 탐색·연결·패키지 시작 시간이 더해진다.

이 판단은 node control 한 곳이 소유한다. 연결 후 endpoint를 동기화하고 기존 PackageManager의 node_connection_changed lifecycle을 적용한다. Zenoh는 새 경로로 세션을 열고 DLS 등 로컬 소비자는 계속 실행한다. 패키지의 독립 실행, 공통 정책, 목적별 전송 adapter의 경계와 모든 패키지에 적용하는 기준은 통신·복구 계약을 따른다.

Gateway 연결 주소는 현재 127.0.0.1:15003 하나다. 이전 package별 endpoint와 tcp_port 설정은 구버전 설정 변환 대상이며 현재 사용자가 선택하는 항목이 아니다. 저장소의 과거 HMI bundle에는 N-1 release 전환 검사용 5000 listener가 남을 수 있지만 현재 Unity source와 관리 실행의 통신 구조로 설명하거나 재사용하지 않는다.

실차에서 ROS2나 sensor가 보이지 않으면 application 재시작 전에 다음을 함께 확인한다.

- main/edge의 차량 ID, platform, ROS domain과 node identity
- NIC 주소, subnet, multicast routing, IGMP, MTU와 ring buffer
- zenoh-bridge 와 ros2-domain-bridge 의 endpoint와 health
- package가 mount한 CycloneDDS config와 shared message ABI
- sensor IP, serial, UDP port와 실제 link speed

9. 유지보수와 검증

장애를 좁히는 순서

1. furive status 에서 node와 package health를 확인한다.
2. furive service logs 에서 Furive lifecycle 오류를 확인한다.
3. 문제 package의 logs/<package>.log 와 build log를 확인한다.
4. Docker package이면 container 상태와 해당 package 로그를 확인한다.
5. 통신 문제는 node control, integration gateway, Zenoh 중 어느 경로인지 먼저 구분한다.

6. ROS2 문제는 domain, CycloneDDS config, shared message ABI를 확인한다.
7. 장치 문제는 process restart보다 NIC, sensor, CAN, mount와 권한을 먼저 확인한다.
8. 업데이트 문제는 target identity, staging, activation journal과 직전 정상판 상태를 확인한다.

로그에 `ERROR` 라는 문자열이 있다는 이유만으로 package를 unhealthy로 판단하지 않는다. process 또는 container 생존 여부 밖의 판정이 필요하면 package manifest가 health pattern과 startup grace를 소유한다. `pending` 상태는 준비 중이라는 뜻이며 restart 실패 횟수를 소비하지 않는다.

변경 단계에 맞는 검증

단계	필요한 검증
일상 개발	변경 범위의 작은 test, 관련 package test
차량 한 대 시험	<code>furive check</code> , 한 차량 <code>test</code> , 변경 기능의 실행과 복구
고객용 <code>normal</code>	전체 검증, 운영 증거, 실제 권한·데이터·외부 전송 변경에 필요한 별도 검토

작은 test를 먼저 실행하고 마지막에 다음 공통 검사를 실행한다.



systemd, Docker, GUI, 장치나 실차를 실제로 실행하지 않았다면 검증 결과에 그 사실을 남긴다. 미래 인증 로드맵이나 변경과 무관한 운영 자동화를 일상 개발의 완료 조건으로 추가하지 않는다.

10. 상세 reference

더 자세히 볼 내용	기준 문서
프로그램 하나를 연결하는 기본 절차	프로그램 연결 가이드
<code>package.yaml</code> 선택 항목과 권한·health 규칙	프로그램 연결 설정
새 차량과 Autoware 구성 연결	차량 플랫폼 만들기
설치가 host에 미치는 영향	호스트 영향
차량 설치	차량 PC 설치
시험판과 공용판 업데이트	차량 업데이트
업데이트 호환판과 구버전 설정 변환	업데이트 호환 정책
직전 정상판 판정과 복구	이전 정상 버전 기준
<code>release·bundle·CI</code> 산출물	CI/CD 기준
<code>control</code> , <code>credential</code> , <code>package</code> 권한	보안·개인정보 기준

더 자세히 볼 내용	기준 문서
변경 단계별 검사 범위	변경 검증

과거 조사와 계획은 [docs/reports/](#)에 있다. 현재 동작과 사용 절차는 README, guides, 현재 code와 이 reference를 우선한다.

11. 실행 구성요소

이 장의 목적 · runtime 장애 조사자

표시·전달 구성요소와 실제 상태 소유자를 구분합니다.

완료 판단

상태·로그·통신 문제를 올바른 구성요소에서 조사합니다.

이 문서는 Furive OS 안에서 어떤 프로그램이 무슨 일을 맡는지 설명한다. 이름을 모두 외울 필요는 없다. 평소에는 차량이 부팅되면 서비스와 운영 화면이 자동으로 시작되고, 문제가 있을 때만 이 문서를 찾으면 된다.

전체 흐름

PC 한 대에 자율주행·운영·부가 패키지를 함께 배정할 수 있다. A-ECU·HMI-ECU 분산 구성은 부하를 나누는 권장 배치다. 두 방식 모두 아래의 기능 소유자와 공통 lifecycle을 사용한다.

PC 한 대부터, 필요하면 분산

플랫폼이 차량 · 센서 · 패키지와 실행 PC를 배정

1PC 한 PC에 함께 설치

Autoware · 운영 화면
자율주행 · 기본 운영

HMI · DLS 등 부가 패키지
함께 배정한 기능

한 설치 · 공통 실행 관리 · 로컬 ROS/DDS와 메시지

분산 엣지 컴퓨팅

자율주행과 부가 기능의 부하 분리 권장

A-ECU

Autoware · 기본 운영
자율주행 PC

HMI-ECU

데이터 로깅 · 부가 기능
부하를 분담하는 별도 PC

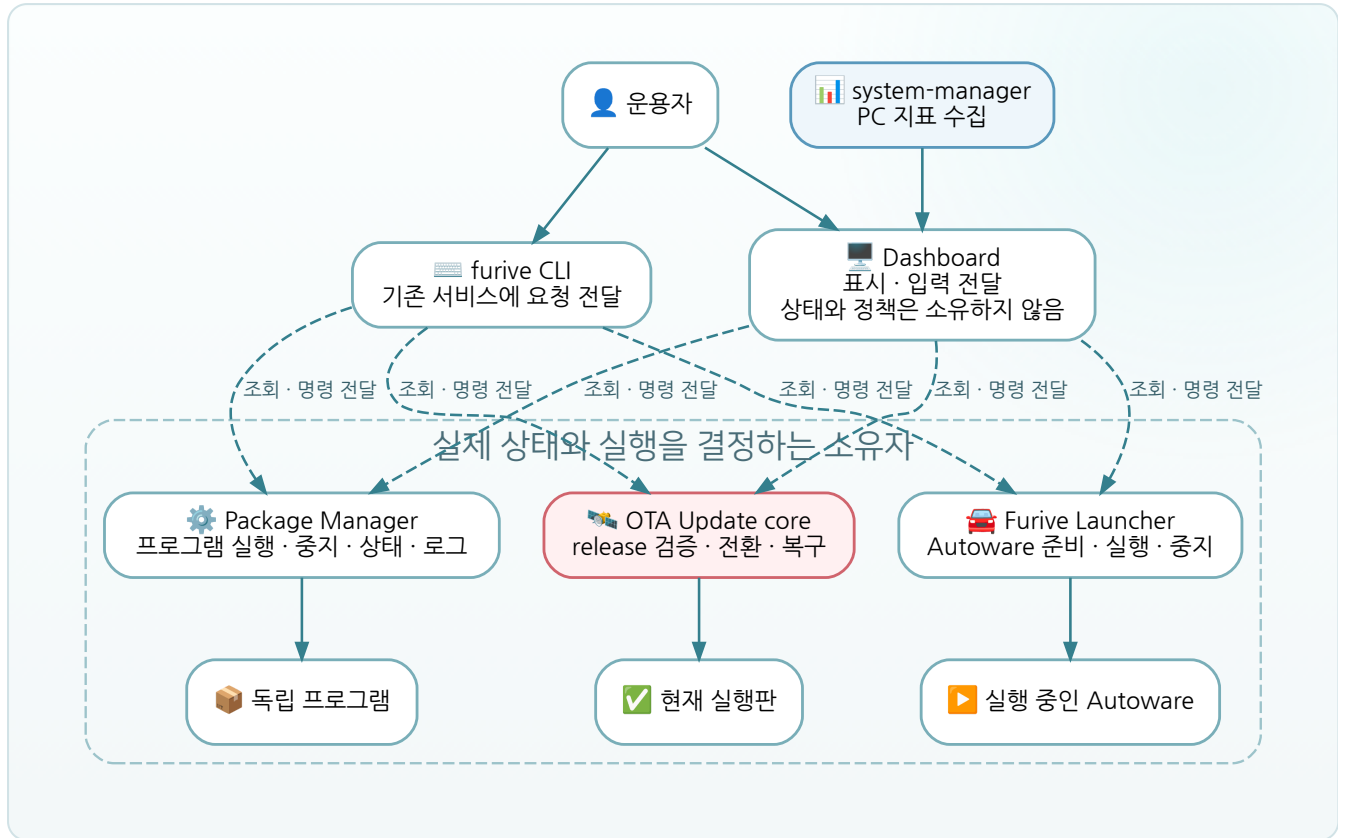


PC 간 연결: 등록 · 상태 · 운용 요청 / ROS 데이터 전달

두 구성 모두 공통 설치 · 실행 · 상태 · 업데이트와 복구

현재 기본 배정과 1PC 구성 준비는 ECU 설치 설계를 따른다.

운영 명령과 상태를 맡는 프로그램

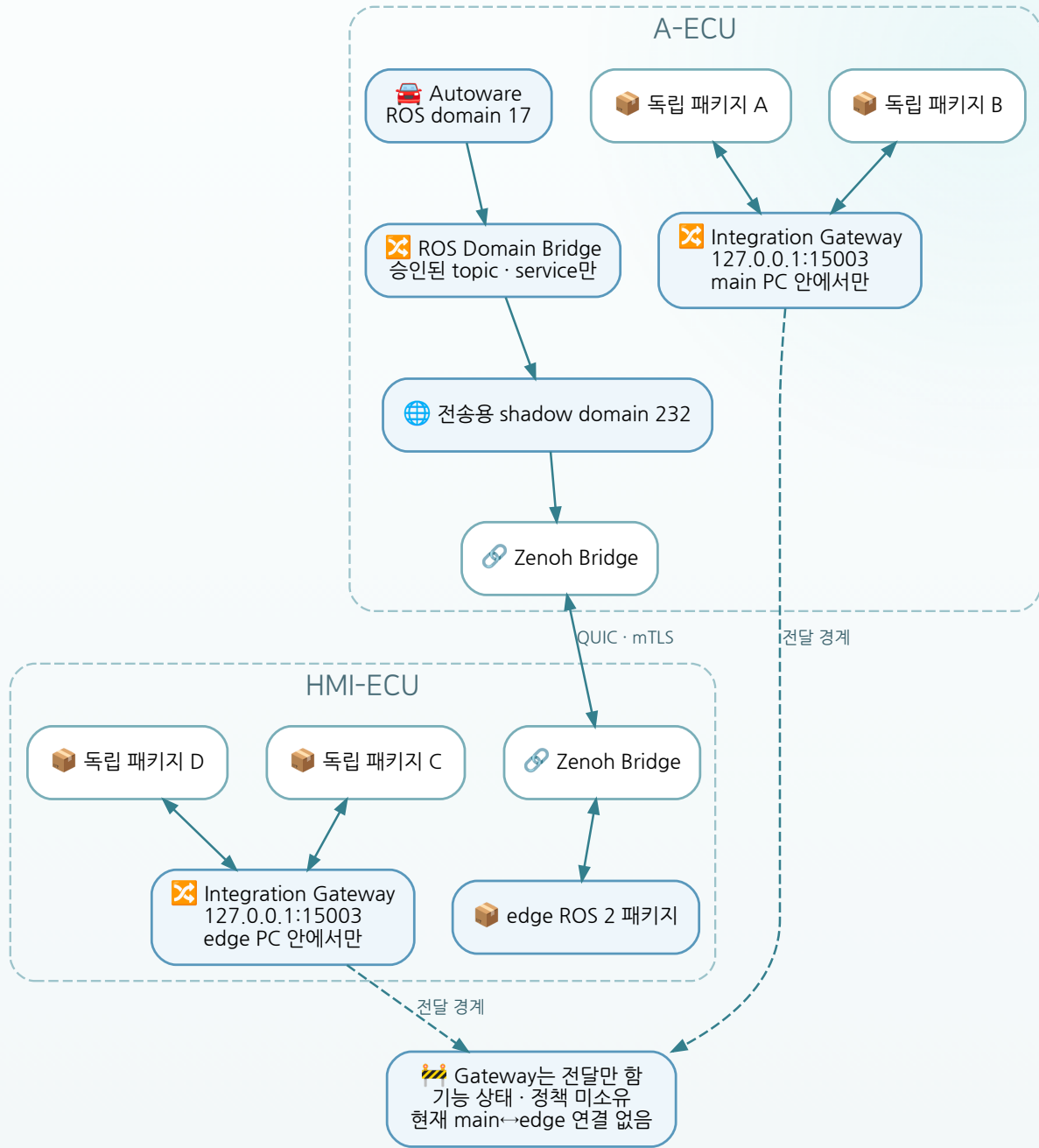


운영 화면(Dashboard)은 여러 프로그램의 상태를 한곳에 보여주고 명령을 전달한다. 프로그램 실행, Autoware 실행, 업데이트 규칙은 각각의 담당 프로그램이 처리한다.

CLI도 같은 담당 기능을 호출하며 운영 화면 실행에 의존하지 않는다. 설치된 PC 배경과 기존 node 프로세스를 사용한다. 운영자는 CLI 명령어 표, 구현 담당자는 공통 소스 찾기를 참고한다.

PC 사이 ROS와 같은 PC 안의 메시지

1PC에서는 그 PC의 ROS/DDS 환경과 로컬 Integration Gateway를 사용한다. 아래는 PC를 분리한 경우의 데이터 전달 경로다. domain 번호는 기본 구성 예이며 실제 값은 플랫폼과 설치 설정을 따른다.



PC 사이 ROS 2 데이터는 ROS Domain Bridge와 Zenoh Bridge가 맡는다. Integration Gateway는 각 PC 안의 요청·응답·이벤트만 전달하며, 기능 상태나 정책을 저장하거나 두 PC를 연결하지 않는다.

경로별 목적·방향·동작의 허용 기준과 구현 한계는 [Furive OS 전체 데이터 교환 정책](#)이 소유한다. 아래 연결도는 현재 전달 경로를 보여주며, 모든 package의 데이터 접근이 개별 인증·격리된다는 뜻은 아니다.

HMI·MGMT·DLS가 함께 보이지만 독립적으로 동작하는 구조

Furive OS 차량 화면·데이터 경로

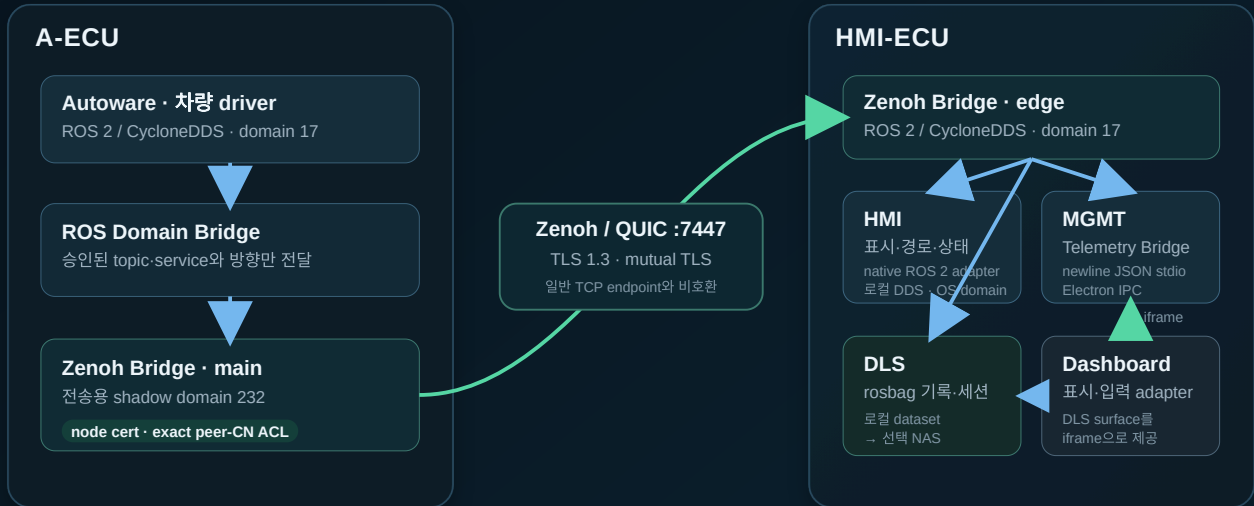
독립 패키지는 같은 ROS 데이터에 연결되지만 lifecycle-상태-설정은 각 패키지가 따로 소유합니다.

1PC · 한 PC에 함께 설치·실행

Autoware · 기본 운영 — 로컬 ROS / DDS — **배정된 HMI · MGMT · DLS**

플랫폼의 1PC 배정 · 한 Package Manager · 같은 PC의 Gateway와 화면 · 자원 공유

분산 구성 · 자율주행과 로깅·부가 기능의 부하 분리 권장



Furive OS가 공통으로 맡는 것

Package Manager

의존성·시작/중지·health·로그

main ↔ edge 제어면

JSON object stream · TCP 15001
TLS 1.3 mutual TLS

Integration Gateway

같은 PC의 요청-응답
loopback · 127.0.0.1

독립성 경계

패키지 사이에 상태-정책을
복제하지 않음

보안 요약: PC 간 ROS-제어면은 각각 mTLS로 연결합니다. HMI는 로컬 DDS를 사용하며, Dashboard와 Gateway는 loopback 경계에 듭니다.

세 화면은 하나의 큰 프로그램이 아닙니다. Furive OS Package Manager가 의존 순서와 실행·중지·health·로그를 공통으로 관리하지만 HMI, MGMT, Data Logging System(DLS)은 각자의 package와 프로세스로 실행된다. MGMT의 기록 탭도 DLS가 제공한 화면을 iframe으로 보여주는 adapter일 뿐 녹화·세션·전송 정책을 복제하지 않는다.

구간	실제 프로토콜과 경계	맡는 일
Autoware → main Zenoh	ROS 2/CycloneDDS domain 17 → ROS Domain Bridge → shadow domain 232	허용된 topic-service와 방향만 전송망으로 전달
main ↔ edge ROS	Zenoh over QUIC/UDP :7447, TLS 1.3 mutual TLS	설치가 배포한 node certificate를 검증하고 exact peer-CN ACL을 적용해 ROS 경로 전달
main ↔ edge 제어면	UTF-8 JSON object stream over TCP :15001, TLS 1.3 mutual TLS	edge 등록·heartbeat·package 상태와 승인된 명령/응답; ROS 데이터 경로와 별도 연결
edge ROS → HMI	HMI에 포함된 ROS 2 runtime과 native ROS adapter → 로컬 DDS	OS가 정한 ROS domain에서 표시용 topic을 Unity HMI에 제공
edge ROS → MGMT	MGMT Telemetry Bridge → newline-delimited JSON over process stdio → Electron IPC	최신 상태 snapshot과 승인된 차량 명령; rosbridge는 명시적 fallback일 때만 사용

구간	실제 프로토콜과 경계	맡는 일
edge ROS → DLS	ROS 2/CycloneDDS → rosbag recorder	선택 topic을 로컬 dataset에 기록하고 세션·재생·선택 NAS 전송을 관리
Dashboard/DLS → MGMT	같은 edge의 loopback HTTP와 server-issued iframe URL	DLS 화면만 MGMT 안에 표시; DLS 상태와 설정은 DLS가 소유

HMI는 로컬 DDS에 직접 참여하고 별도 rosbridge 서버를 요구하지 않는다. ROS domain·DDS 설정과 복구 기준은 [패키지 통신·복구 계약](#)을 따른다. rosbridge를 사용하는 경로는 자체 client 인증이 없으므로 127.0.0.1 바인딩, topic allowlist와 subscribe-only 경계를 적용한다. Dashboard surface와 Integration Gateway도 같은 PC의 loopback 경계다. 반대로 PC 사이를 건너는 Zenoh와 Furive node 제어면은 각각 별도의 TLS 1.3 mTLS 연결이며, 일반 TCP Zenoh endpoint로 낮춰 연결하지 않는다. Telemetry Bridge의 stdio와 Electron IPC는 네트워크 연결이 아닌 같은 MGMT process tree 내부 경계이므로 mTLS를 적용하는 구간이 아니다. renderer는 ROS 2나 Zenoh socket에 직접 연결하지 않는다.

상태와 로그를 보는 기준

`furive status`는 전체 운용 문제와 서비스·업데이트·플랫폼 상태를 요약한다. 기존 패키지·PC 상태 보고를 사용하며 최신 전체 상태를 받지 못하면 확인 불가로 표시한다.

패키지별 상태와 로그는 `furive package status`, `furive package logs <name>` 또는 전체 Dashboard의 Package Manager에서 확인한다. 명령별 범위는 [CLI 명령어 표](#)를 따른다. 차량 PC에 로그인하면 플랫폼에 배정된 화면이 열린다. Lotte Ohmio처럼 `furive-launcher` 전용 화면이 먼저 열리는 구성에서 전체 진단 화면이 필요할 때 다음 명령을 사용한다.

```

● ● ●  TERMINAL

furive dashboard --web

```

표준 패키지 실행 로그는 저장소의 `logs/<package>.log`, 빌드 로그는 `logs/<package>.build.log`에 남는다. 파일 이름에서는 하이픈이 밑줄로 바뀐다. 예를 들어 `ota-updater`의 로그는 `logs/ota_updater.log`다. 전체 서비스 기록은 `furive service logs`로 본다.

Dashboard — 하나의 운영 화면

목적: 여러 프로그램의 상태와 명령을 한 화면에 모은다.

- 담당: 화면, HTTP API 연결, 사용자 입력 전달
- 담당하지 않음: 패키지 실행·중지, 업데이트 규칙, Autoware 실행·중지
- 시작: 화면 뒤의 서버는 서비스와 함께 시작하고, main 화면은 데스크톱 로그인 뒤 자동으로 열린다.
- 로그: `logs/dashboard.log`

사용자는 평소 설정 파일을 직접 만지지 않고 화면에서 제공하는 설정만 바꾼다. 개발자는 새 기능의 규칙을 운영 화면에 만들지 말고, 실제 담당 프로그램의 API를 연결하고 결과를 표시한다.

`desktop_surface`가 지정된 플랫폼은 로그인 시 전체 Dashboard 대신 해당 패키지 화면을 연다. 차량 플랫폼의 기본 화면은 Furive Launcher다. 이는 backend나 Package Manager가 빠졌다는 뜻이 아니다.

Package Manager — 모든 패키지의 공통 실행 관리자

목적: 언어와 실행 방식이 다른 프로그램을 같은 방법으로 실행하고 관리한다.

- 담당: 패키지 발견, 실행 순서, 빌드, 실행·중지, 정상 동작 확인, 재시작 판단, 로그
- 입력: 각 독립 저장소의 `package.yaml` 과 실행 진입점
- 출력: 운영 화면과 main/edge PC가 사용하는 패키지 상태
- 로그: `logs/<package>.log`, `logs/<package>.build.log`

사용자가 Package Manager 자체를 설정할 일은 없다. 개발자는 프로그램 안에 별도 재시작 반복문이나 다른 패키지 제어를 넣지 말고, 필요한 차이만 `package.yaml` 에 선언한다. 기본 패키지는 이름과 버전, `run.sh` 만으로 시작할 수 있다. 자세한 필드는 [프로그램 연결 설정](#)을 따른다.

Furive Launcher — Autoware 실행 준비

목적: 선택된 차량·센서 모델과 지도에 맞춰 Autoware를 준비하고 시작·중지한다.

- 담당: Autoware 빌드, 실행·중지, 시작 방식과 실행 상태
- 실행 위치: main PC
- 화면: 차량 플랫폼에서 로그인 뒤 기본으로 열리는 주행 준비 화면
- 로그: `logs/furive_launcher.log`
- Autoware 실행 로그: `logs/furive_launcher_autoware.log`
- Autoware 빌드 로그: `logs/furive_autoware_build.log`

사용자는 주행 준비 화면에서 제공하는 플랫폼·지도·실행 동작만 사용한다. 개발자는 플랫폼 정의와 외부 Autoware 소스 버전을 Furive Launcher에 복사하지 않고 각각 `config/capability-profiles.yaml`, `config/external-workspaces.yaml` 에서 바꾼다.

OTA Update — 원격으로 안전하게 계속 업데이트

목적: 차량이 새 배포판을 자동으로 찾고, 서명과 호환성을 확인한 뒤 안전하게 전환하도록 한다.

- 화면·명령 연결: `package/core/ota-updater`
- 업데이트 규칙과 실행: `core/ota_policy.py`, `core/self_updater.py`, `core/release_activation.py`
- 업데이트 흐름: 확인 → 내려받기와 검증 → 다음 재시작에 활성화 → 실패 시 이전 판 복구
- 채널: 한 차량 시험은 `test`, 고객용 공용판은 `normal`
- 로그: `logs/ota_updater.log`

차량 사용자는 보통 확인 주기나 내부 branch를 바꾸지 않는다. 문제가 있으면 Dashboard의 OTA 화면과 `furive status` 로 현재 판과 진행 상태를 본다. 개발자는 한 차량 `test` 에서 실행과 복구를 확인한 뒤 `normal` 로 승격한다. 배포 버전과 설치 묶음의 판단은 [CI/CD 기준](#)을 따른다.

PC 상태 수집기 (`system-manager`)

호스트 시간 상태는 `core/time_sync.py` 가 관측해 공통 시스템 지표에 포함한다. 시각 보정은 chrony가 소유하고 `furive system status` 와 `furive status` 에서 확인한다. 준비되지 않은 호스트를 동기화 완료로 표시하지 않는다. [PC 시간 동기화](#)에 설치·보정·연결 복구 기준을 정리한다.

목적: main PC의 자원 사용량을 가볍게 수집해 Dashboard에 제공한다.

- 수집: CPU, 메모리, uptime, OS, 네트워크 사용량, NVIDIA GPU 정보

- 출력: 내부 상태 파일 `logs/.system_manager_state.json`
- 로그: `logs/system_manager.log`

이 패키지는 차량 상태, 디스크 관리, 전원 제어, 서비스 시작·종료를 담당하지 않는다. 사용자가 설정할 항목도 없다. 개발자는 새 운영 정책을 이 수집기에 넣지 않고, 필요한 지표만 추가한다.

Zenoh Bridge — main과 edge의 ROS 2 연결

목적: 서로 다른 PC의 필요한 ROS 2 데이터를 차량 네트워크로 전달한다.

- 흐름: main Zenoh Bridge ↔ QUIC/TLS 1.3 상호 인증 ↔ edge Zenoh Bridge
- 연결 주소: 설치가 정한 main 주소에서 자동으로 계산
- 보안: 설치가 배포한 PC 인증서로 상대 PC 확인
- 로그: `logs/zenoh_bridge.log`

사용자는 연결 주소나 인증 방식을 반복 설정하지 않는다. 개발자는 전달할 ROS 계약이나 네트워크 보안 경계를 바꿀 때만 이 패키지를 수정한다. 일반 TCP Zenoh 연결 주소와는 호환되지 않는다.

ROS Domain Bridge — Autoware와 전송망 사이의 경계

목적: main PC에서 Autoware의 ROS 2 영역과 Zenoh 전송용 영역을 분리하고, 승인된 경로만 잇는다.

FLOW / OUTPUT

Autoware domain 17 → ROS Domain Bridge → 전송용 shadow domain 232 → Zenoh Bridge

- 담당: 허용된 ROS topic·service와 전달 방향
- 실행 위치: main PC
- 로그: `logs/ros2_domain_bridge.log`

232는 제품 내부 전송용 예약값이므로 일반 사용자가 바꾸지 않는다. Autoware domain은 PC 설정에서 정한다. 통합 개발자는 HMI나 edge가 새 ROS 경로를 실제로 필요로 할 때만 허용 목록과 메시지 형식을 수정하고 main→edge 전체 경로를 검증한다.

Integration Gateway — 같은 PC의 패키지 통합 버스

목적: 독립 개발된 패키지가 서로의 내부 코드에 의존하지 않고 요청·응답·이벤트를 교환하게 한다.

- 범위: 현재 **같은 PC 안에서만** 동작
- 연결: `127.0.0.1:15003`
- 시작: 필요한 패키지가 의존 프로그램으로 지정하면 Package Manager가 자동으로 먼저 시작
- 메시지: 패키지 이름을 대상으로 하는 요청·응답과 주제(topic)별 이벤트
- 로그: `logs/integration_gateway.log`

사용자가 주소나 별도 서비스를 설정할 일은 없다. 패키지 개발자는 상대 패키지 프로세스에 직접 붙는 대신 공용 통로의 간단한 메시지 형식을 사용한다. 일반 메시지는 내용 해석 없이 전달하며 업무 원본 상태와 정책은 기능 패키지가 소유한다. 구버전 응답 변환 예외는 **패키지 계약**에 한정한다. 현재 main과 edge 사이의 gateway 연결은 없다. 상세 메시지 계약은 [Integration Gateway README](#)에 있다.

변경 위치 요약

바꾸려는 것	담당 프로그램
화면 배치와 표시	Dashboard
패키지 실행·정상 동작 확인·로그 규칙	Package Manager
Autoware 시작·중지·빌드	Furive Launcher
원격 업데이트 정책과 활성화	OTA Update core
PC 지표 수집	<code>system-manager</code>
main↔edge ROS 전송	Zenoh Bridge
Autoware↔전송망 ROS 경로	ROS Domain Bridge
같은 PC 패키지 간 메시지	Integration Gateway

전체 PC와 배포 버전 구조까지 필요하면 [기술 가이드](#)를 이어서 본다.

12. 프로그램 연결 설정 (package.yaml)

이 장의 목적 · 고급 패키지 개발자

최소 기본값을 벗어날 때만 필요한 필드를 선택합니다.

완료 판단

발견·실행·종료·health·권한 계약이 한 manifest에서
검증됩니다.

이 문서는 프로그램을 Furive에 연결할 때 쓰는 `package.yaml`의 상세 설정입니다. 일반 프로그램은 먼저 [프로그램 연결 가이드](#)의 `name`, `version`, `run.sh`만 사용합니다.

기본값으로 동작한다면 필드를 더 쓰지 않습니다. 새 필드를 추가할 때는 “기본값으로 불가능한 이유”와 “이 값을 담당하는 한 곳”을 설명할 수 있어야 합니다.

 독립 프로그램

 package.yaml

 Furive가 발견

 실행 · 상태 확인 · 중지

필요한 기능	찾아볼 절
최소 설정	최소 필드와 기본값
로그와 상태 확인	로그와 정상 상태 기준
장치·권한	책임·권한 경계
개발 중 소스 실행	승인된 소스 실행(Source Unlock)
시작·종료·재시작	시작·중지 규칙, 다시 시작 조건
패키지 자체 테스트	패키지가 담당하는 테스트

아래부터는 코드와 설정에서 쓰는 내부 필드 이름을 그대로 설명합니다.

개발 경계

Furive 패키지는 독립 저장소에서 이미 개발된 프로그램을 Furive의 발견·실행·상태 확인·로그·업데이트에 연결하는 운용 단위다. Autoware package나 ROS package와 다른 개념이며 `furive package` 명령은 프로그램 소스, Dockerfile 또는 Autoware 작업 공간을 생성하지 않는다.

프로그램 내부의 언어, 프레임워크, 빌드 및 테스트 방식은 독립 저장소가 담당한다. Furive OS는 `package.yaml`과 실행·중지·정상 상태 확인만 담당한다. Autoware 소스는 별도 작업 공간과 저장소에서 개발하고, Furive OS에서는 검증된 소스 버전과 Autoware 시작·종료 방식만 연결한다.

패키지 발견 기준

프로그램 실행 관리자와 검사 도구는 기본적으로 다음 `package.yaml` 을 찾는다.

FLOW / OUTPUT

```
package/core/*/package.yaml
package/plugin/*/package.yaml
```

공식 패키지는 최상위 저장소가 추적하거나 `.gitmodules` 에 등록된 연결 저장소입니다. 승인된 소스 실행 표시가 있는 작업 폴더도 포함합니다. 등록되지 않은 중첩 Git 저장소는 제외해 예전 작업 폴더나 백업이 검사와 차량 실행에 섞이지 않게 합니다.

외부 패키지를 실행할 때만 `FURIVE_OS_EXTERNAL_PACKAGE_DIRS` 에 별도 Git 저장소 경로를 지정합니다.

최소 필드와 기본값

일반 script 패키지의 필수 필드는 `name` 과 `version` 뿐이다.

필드	기준
<code>name</code>	Furive OS가 인식하는 패키지명. 플랫폼 설정에는 이 이름을 쓴다.
<code>version</code>	사람이 확인할 수 있는 패키지 버전.
<code>type</code>	생략 시 <code>script</code> . 필요한 경우 <code>docker</code> , <code>colcon</code> , <code>binary</code> 를 명시한다.
<code>nodes</code>	생략 시 <code>edge</code> 를 뜻하는 내부 값 <code>[agent]</code> . 설정 파일에서만 <code>master</code> , <code>agent</code> 를 사용한다.
<code>run.script</code>	생략 시 <code>run.sh</code> . 다른 이름을 사용할 때만 명시한다.

YAML · CONFIG

```
name: hello-sensor
version: 0.1.0
```

Docker 패키지는 `type: docker` 와 `run.script` 가 필요합니다. Docker 이미지 이름을 생략하면 패키지 이름을 사용합니다. 빌드가 필요하면 `build.script` 또는 `Dockerfile` 을 두고, 실행 파일 패키지는 `binary.path` 를 지정합니다.

실행 프로그램 없이 운영 화면 파일만 제공하는 패키지는 `dashboard.yaml` 과 `dashboard/index.html` 을 함께 둡니다. `run.sh` 나 상태 검사를 역지로 추가하지 않습니다.

`docker` 형식은 컨테이너로, 나머지는 일반 프로그램으로 실행 관리자가 직접 감시합니다. `run.sh` 는 준비를 끝낸 뒤 실제 프로그램을 `exec` 해야 합니다. 자체 재시작이나 `kill` 을 넣지 않습니다.

호스트의 로그인 X 세션을 실제로 사용하는 패키지만 아래 필드를 선언한다. `edge/main` 백엔드는 GUI를 기다리지 않고 먼저 시작하며, PackageManager는 사용자 `systemd` 세션이 개시한 정확한 `DISPLAY` 와 `XAUTHORITY`, X 소켓 접근을 확인한 뒤 해당 패키지만 시작한다. 기다리는 동안 재시작 실패 횟수를 소비하지 않는다. 컨테이너 안에서 자체 X 서버만 사용하는 패키지에는 선언하지 않는다.

YAML · CONFIG

```
runtime:
  requires_graphical_session: true
```

공식 release에서 GUI를 host process로 실행해야 하지만 package source 접근 권한을 배포하면 안 되는 Docker package는 준비된 runtime을 같은 release image 안에 넣고 다음처럼 선언한다.

YAML · CONFIG

```
runtime:
  requires_graphical_session: true
  host_from_image:
    source: /app
    entrypoint: run.sh
    startup_probe_seconds: 2
```

PackageManager는 release manifest가 선택한 architecture별 로컬 image를 사용하고 없을 때만 pull한다. 임시 network-disabled container에서 지정 payload만 사용자 전용 임시 디렉터리로 materialize한 뒤 로그인 세션 환경으로 실행한다. 상태·중지·재시작·health는 기존 PackageManager process lifecycle이 그대로 소유한다. 별도 user service나 autostart owner를 만들지 않는다. 이 필드는 Git key를 설치하거나 source checkout을 허용하지 않으며 `source_unlock`의 접근제어 의미를 바꾸지 않는다. Image 안의 backend는 필요하면 같은 digest image를 headless sidecar로 실행할 수 있다.

차량 플랫폼이 선택하는 Docker package의 release image는 원본 구현 source를 실행 파일처럼 그대로 실행하지 않는다. 해당 Dockerfile은 build stage와 runtime final stage를 분리하고, final stage에는 native binary, bundle/minified JavaScript, bytecode, ROS install tree처럼 실행에 필요한 결과와 config만 복사한다. 최종 image에는 다음 내부 metadata를 둔다.

DOCKERFILE

```
LABEL io.furive.runtime.source-confidential="1"
```

Package manifest 검사는 이 label과 final stage의 직접적인 source copy를 정적으로 확인하고, release workflow는 실제로 빌드한 local image의 label을 Harbor push 전에 다시 확인한다. 이 계약은 `docker run`, `docker save`로 원본 프로젝트 source를 그대로 열람하는 경로를 막기 위한 것이다. 실행 결과는 역공학될 수 있으므로 암호화나 DRM, 완전한 비밀성을 뜻하지 않는다. 실행에 필요한 공개 OSS dependency, 생성 code, ROS/Python runtime helper와 config도 image에 남을 수 있다.

Source Unlock은 이 경계의 예외가 아니라 별도의 의도적인 source 제공 경로다. 운영자가 package를 명시적으로 승인하면 Git checkout을 전달하며, 그 checkout과 Autoware source workspace에는 위 image 은닉 계약을 적용하지 않는다.

로그와 health 경계

패키지는 stdout/stderr에 로그를 쓰고 메시지 내용과 내부 logger 이름을 소유한다. Furive는 서로 다른 언어와 framework의 출력 본문을 다시 해석하거나 재작성하지 않는다. 대신 Docker runtime을 파일과 Dashboard에서 읽을 때 Docker의 RFC3339 timestamp를 붙이고, PackageManager가 생성하는 build·start lifecycle 행은 다음 공통 형식을 사용한다.

FLOW / OUTPUT

2026-08-10T16:30:00+0900 [INFO] [example] 빌드 완료

로그에 `ERROR`, `failed`, `traceback` 같은 문자열이 있다는 이유만으로 package health를 실패로 판정하지 않는다. 프로세스·컨테이너 생존 여부 외에 로그 기반 판정이 필요하면 그 package의 `health_check.healthy_log_patterns`, `health_check.pending_log_patterns` 또는 `health_check.unhealthy_log_patterns`에 명시한다. `pending_log_patterns`는 장치 연결을 기다리는 살아 있는 runtime처럼 재시작하면 안 되는 미준비 상태에만 사용한다. 이 규칙으로 Chromium/D-Bus 진단처럼 무해할 수 있는 `stderr`와 실제 lifecycle 실패의 소유자가 섞이지 않는다.

`health_check.startup_grace_seconds` 동안 살아 있는 runtime은 healthy가 아니라 `pending`이다. OTA candidate gate는 이 시간이 끝나기 전에 package를 확정하면 안 된다. `health_check.max_failures`는 누적 실패 횟수가 아니라 연속된 확정 unhealthy sample의 threshold이며 healthy sample이 나오면 counter를 초기화한다. observer 자체 오류나 indeterminate 결과는 runtime crash로 간주하지 않는다. `pending`도 연속 실패 횟수와 재시작 횟수를 소비하지 않으며 다른 독립 package의 lifecycle에 영향을 주지 않는다. 다만 준비되지 않은 장치를 healthy로 표시하지는 않는다.

`update.rollback_on_failure`의 기본값은 `true`다. 장치가 빠진 정비·개발 환경에서도 나머지 기능이 계속 동작해야 하는 하드웨어 package만 이를 `false`로 둔다. 이 package도 평소처럼 시작·health 표시·재시작 감독을 받지만, 장치 미연결이 OTA 후보 전체의 rollback 사유가 되지는 않는다. 특정 차량 시험에서 그 장치를 반드시 확인해야 하면 `node의 self_update.candidate_required_packages`에 명시해 해당 시험에서만 release gate로 승격한다. `ota-updater`는 복구 경로를 준비하는 core package이므로 실행 대상으로 선택된 node에서는 이 예외를 적용하지 않으며 항상 후보 release gate에 포함된다.

책임·권한 경계

`dependencies`는 문서용 메타데이터가 아니라 실행 순서 계약이다. PackageManager는 의존 패키지를 먼저 시작하고, 재시작·종료 때는 역순으로 정리하며 OTA 후보 검증에도 같은 dependency closure를 사용한다. 없는 의존성이나 순환 의존성은 시작 실패이며, 실행 중인 dependent가 있으면 개별 dependency 중지를 거부한다. 따라서 로컬 메시지가 필요한 기능 패키지가 `integration-gateway`를 의존하면 사용자가 gateway를 패키지 목록에 다시 선택하거나 새 설정을 배울 필요가 없다.

제어 토큰은 기본적으로 package의 build/run 환경과 Docker container에 전달하지 않는다. Master 제어면을 호출해야 하는 신뢰 package만 다음처럼 명시적으로 opt-in한다.

YAML · CONFIG

```
control:
  credentials: true
```

이 설정은 전역 관리자 권한이 필요한 core package에만 허용한다. 일반 plugin은 자체 로컬 서비스나 제한된 Master command를 사용하고 reusable token을 보유하지 않는다.

Docker package는 `privileged: true`와 `pid: host`를 기본 수단으로 쓰지 않는다. 필요한 Linux capability와 장치를 최소 단위로 선언한다. 선택 장치는 host에 없을 때 생략할 수 있다.

YAML · CONFIG

```
docker:
  cap_drop: [ALL]
  cap_add: [NET_ADMIN, NET_RAW]
  security_opt: [no-new-privileges:true]
  devices:
    - path: /dev/bus/usb
      optional: true
```

기능마다 지속 상태와 lifecycle의 소유자는 하나뿐이다.

책임	단일 소유자
패키지 실행·중지·health	PackageManager
main/edge 등록·호스트 상태	main/edge 노드
각 PC 안의 패키지 사이 요청·응답·이벤트 전달	integration-gateway
녹화·재생·세션·NAS 전송	data-logging-system
Autoware 시작·종료	furive-launcher
ROS2 데이터 전달	zenoh-bridge
OTA 정책·목표·준비·적용·복구	하나의 OTA 흐름; 구현 분리는 기술 구조 참조
표시와 사용자 입력	Dashboard·CLI; 다른 항목의 상태는 소유하지 않음

integration-gateway의 일반 v1 경로는 크기가 제한된 메시지를 지정된 수신자에게 전달할 뿐 업무 상태를 소유하지 않는다. 현재 hmi_dls_legacy.py 만 N-1 응답 호환을 위해 DLS 명령을 분류하고 작은 휘발성 상태 projection을 둔다. 지속 녹화·재생·세션·NAS 상태와 정책은 계속 data-logging-system이 소유하고, gateway 재시작 뒤 다시 동기화한다. 새 호환 상태를 gateway에 추가하지 않으며 HMI/DLS가 v1 envelope를 직접 쓰면 이 adapter를 삭제한다.

패키지의 데이터 요구와 플랫폼의 허용 범위, 조화·수신·명령·기록·외부 전송의 구분은 Furive OS 전체 데이터 교환 정책을 따른다. 중앙 ROS 전달 설정과 명시적인 furive topic 적용은 구현되어 있다. DLS 기록 선택은 전달 허용 범위를 늘리지 않으며, manifest에 임의의 권한 필드를 추가해 허용된 것으로 취급하지 않는다. 운용 절차는 토픽 명령어 표를 본다.

통신·복구 계약

패키지의 독립 개발·실행과 공통 연결 정책은 함께 유지한다. 플랫폼의 기존 연결 정의가 허용하는 데이터 흐름을 한 번 결정하고, 각 PC의 전송 adapter가 실행한다. 패키지마다 상대 PC 주소, 이더넷/Wi-Fi 우선순위, 탐색·경로 전환 정책을 복제하지 않는다. 같은 목적의 데이터에 새 전송 프로토콜을 추가하지 않으며 기술적으로 분리가 필요할 때만 기존 소유자 안에서 처리한다.

책임	단일 소유자와 패키지 계약
데이터 연결 허용 범위	기존 플랫폼 연결 정의. 프로토콜별 설정은 이 정의의 projection이며 독립 정책이 아니다.

책임	단일 소유자와 패키지 계약
상대 PC 선택·복구	Furive OS node control과 <code>core/network_selection.py</code> . 인증된 이더넷 경로 우선, Wi-Fi 대체, 응답 단절 후 재탐색, 더 좋은 경로 복귀 시 인증 후 전환한다.
PC 간 ROS 데이터	Zenoh가 전송하고 소비 패키지는 로컬 ROS 영역을 사용한다. 소비 패키지는 상대 PC의 NIC/IP를 선택하지 않는다.
같은 PC의 패키지 메시지	기존 loopback integration gateway를 사용한다. PC 간 gateway routing은 현재 구현되어 있지 않다.
연결 재생성이 필요한 패키지	기존 <code>lifecycle.restart_on: [node_connection_changed]</code> 로 선언한다. PackageManager가 실행 중·시작 중·오류 상태인 선언 패키지만 재시작하며 수동 중지는 유지한다.
로컬 기록·처리	DLS를 포함한 소비 패키지는 네트워크 전환만으로 재시작하지 않는다. 입력이 돌아오면 기존 구독으로 수신을 재개하고, 단절 중 생성되지 않은 데이터까지 복구했다고 보고하지 않는다.
NAS 등 별도 목적지	해당 기능이 기존 목적지·인증·미완료 작업을 소유한다. 네트워크 오류에는 제한된 대기 간격으로 다시 연결하고 pending 작업을 보존한다. 차량 PC 탐색 정책을 외부 서버 선택에 재사용하지 않는다.

`node_connection_changed`는 인증된 node 연결이 수립된 뒤 endpoint projection을 갱신하고 HMI-ECU의 기존 PackageManager에 적용하는 내부 lifecycle 이름이다. 같은 IP로 복구되어도 발생한다. 별도 버스·서비스·설정 파일을 추가하지 않는다. 기본값인 무재시작으로는 오래된 QUIC 세션을 새 경로로 재생성할 수 있어야 한다는 요구를 충족하지 못하므로 Zenoh만 이 event를 선언한다. 운용자는 새 명령을 배울 필요가 없고, 패키지 작성자는 연결 재생성이 필요할 때만 이 선언을 사용한다.

ROS2 HMI는 프로세스 안의 native ROS adapter로 로컬 DDS에 참여한다. 별도 rosbridge 서버나 WebSocket endpoint를 추가하지 않으며 배포물에 ROS runtime을 포함한다. MGMT의 기존 native ROS adapter와 같은 ROS/Zenoh 경로를 사용한다. MGMT의 로컬 stdout/IPC는 같은 프로세스 묶음의 UI adapter이며 새 PC 간 프로토콜이 아니다. HMI domain과 DDS 설정은 OS 실행 환경만 소유한다. HMI 메시지 변환은 기존 IL2CPP 빌드에 맞춰 정적 코드로 생성하고 UI 상태는 최신 값 하나로 전달한다. 변경 시에만 발행하는 latched 상태에 주기적 telemetry의 시간 초과 기준을 적용하지 않는다. MGMT는 ROS executor 한 개에서 상태를 갱신하고, 출력 주기가 지나면 마지막 값을 내보내므로 burst의 마지막 상태를 버리지 않는다.

네트워크 오류 뒤에는 상태 조회·구독을 다시 구성할 수 있지만 응답을 잃은 차량 명령을 자동으로 재실행하지 않는다. 명령 결과가 불명확하면 현재 상태를 확인한다. 새 연결의 상대 인증이 실패하면 기존 정상 경로를 유지하고, 연결·정책 적용 실패를 성공으로 표시하지 않는다.

이 계약을 바꾸는 테스트는 무응답 단절, 대체 경로 복구, 이더넷 복귀, 새 후보 인증 실패, 같은 IP의 재연결, 로컬 기록 유지, pending 작업 보존 중 해당 범위를 검증한다. 센서 전용망·외부 전송 등 실제 장치를 사용하지 않은 범위는 명시하며 모든 패키지의 실장비 복구가 검증됐다고 간주하지 않는다.

Source Unlock 실행 모드

Docker package는 기본적으로 deploy repo에서 Harbor image로 실행된다. Source unlock으로 실제 package repo가 `package/core/<name>` 또는 `package/plugin/<name>`에 checkout되더라도, 명시 설정이 없으면 PackageManager는 기존 Docker 실행 방식을 유지한다.

Source checkout에서 host `run.sh`를 직접 실행해야 하는 Docker package는 다음처럼 opt-in한다.

YAML · CONFIG

```
source_unlock:
  run_mode: source
  build_script: scripts/prebuild-source.sh
  startup_probe_seconds: 2
```

이 설정이 있고 source unlock checkout이 감지되면 PackageManager는 `source_unlock.build_script` (생략 시 `build.script`) 를 완료한 뒤 `run.sh` 를 subprocess로 실행한다. 사전 빌드를 수행한 경우 실행 환경에 `FURIVE_OS_BUILD_PREPARED=1` 이 전달되므로 package는 같은 빌드를 반복하지 않을 수 있다. `startup_probe_seconds` 를 지정하면 그 시간 안에 프로세스가 종료되는 경우 시작 성공으로 보고하지 않는다. 상태 조회, 중지, health check도 process 기준으로 처리한다. Docker image 실행과 source 직접 실행을 모두 지원하는 GUI package, 예를 들면 `lotte-ohmio-mgmt` , 에서 사용한다.

Deploy source unlock 중 active config의 `self_update.package_updates.<package-name>: false` 인 package만 실제 Git checkout을 `/data/furive-os/dev-sources/package/<core|plugin>/<name>` 에 영구 보관하고, release tree의 원래 package 경로에는 symlink로 노출한다. 따라서 whole-release atomic swap이 일어나도 개발 workspace의 tracked/untracked 파일과 Git HEAD는 바뀌지 않는다. Source Unlock key는 updater의 clone/fetch에만 사용하고 repo-local SSH 명령으로 고정하지 않는다. 기존 package 경로에서 commit/push할 때는 쓰기 권한이 있는 사용자의 호스트 GitHub 인증 또는 SSH key가 그대로 적용된다. 기존 workspace에는 updater가 fetch만 수행하며 stash/reset, release ref checkout 또는 submodule update를 수행하지 않는다. detached HEAD에는 같은 commit의 로컬 작업 branch만 연결할 수 있다. `FURIVE_OS_DEV_SOURCE_ROOT` 로 경로를 바꿀 수 있지만 updater와 launcher를 포함한 모든 실행 환경에 같은 값을 제공해야 한다.

기본 managed checkout을 독립 개발 저장소로 전환하려면 package 경로에서 `.git` 을 제거하고 `git init` 한다. 다음 source sync는 원래 Source Unlock remote가 없는 새 Git 저장소를 사용자 takeover로 판정하고 기존 `self_update.package_updates.<name>: false` 정책을 기록한 뒤 영구 workspace로 옮긴다. deploy 루트의 `git status`에서는 installer-owned overlay를 숨기지만, package 경로의 `git status`에는 개발자의 tracked/untracked 변경이 그대로 보인다.

Deploy image package의 source checkout은 소스 저장소의 `package.yaml` 을 포함하지만, runtime 정책은 release가 생성한 `release/package-manifests.yaml` 사본을 사용한다. 따라서 source-unlock이 package 경로를 checkout 또는 symlink로 바뀌도 Harbor digest, architecture별 image와 `distribution` lock은 바뀌지 않는다. launcher는 source sync 전후에 artifact integrity를 각각 검사하며, sync 뒤 release manifest와 package lock이 어긋나면 package를 시작하지 않는다.

설정이 없거나 `true` 이면 package OTA가 기본 동작이다. `false` 에서 `true` 로 되돌리면 외부 workspace를 원래 package 경로로 이동하고, Source Unlock key가 있으면 dirty 파일을 stash한 뒤 manifest의 release ref로 정렬한다.

`next_restart` OTA는 현재 release를 실행한 채 후보 checkout에서 source package를 사전 빌드한다. 기본적으로 non-Docker `build.script` 와 source mode의 `source_unlock.build_script` 가 대상이다. 사전 빌드가 외부 장치나 실행 중 서비스에 영향을 줄 수 있는 package는 다음처럼 제외한다.

OTA가 중단된 영구 source-unlock workspace는 개발자가 소유한 작업트리이므로 release image pull과 후보 사전 빌드에서 자동 제외한다. 후보 활성화 후 PackageManager가 기존 symlink를 통해 현재 개발 source를 실행하고 health gate는 그대로 적용한다.

YAML · CONFIG

```
update:
  prebuild: false
```

사전 빌드 script는 `FURIVE_OS_PREPARE_ONLY=1` , `FURIVE_OS_CANDIDATE_ROOT=<staging-root>` 환경을 받을 수 있으며 실행 프로세스를 띄우지 않고 후보 artifact만 완성해야 한다. 성공하면 updater가 release HEAD와 build script가 기록된 일회성 marker를 남긴다. 후보의 첫 PackageManager 시작은 marker를 소비하고 동일 build를 건너뛰며, 이후 일반 package

재시작은 기존 build/hash 정책을 다시 따른다. `prebuild: false` 인 host build package는 후보 시작 직전에 이전 정상 release의 build 자산을 승계할 수 있다. PackageManager는 실행 전에 package의 build script를 호출하므로 `.build_hash` 가 현재 source와 다르면 재빌드하고, 같을 때만 기존 결과를 사용한다.

Release CI에서 만든 결과를 차량이 다시 빌드하면 안 되는 source package는 `deploy.built_assets` 에 상대 경로를 선언한다. deploy exporter는 전역적으로 제외하는 `dist/` , build hash 같은 생성물 중 선언된 항목만 포함하고 deploy용 `.gitignore` 예외를 만든다. release 검증은 각 항목이 실제 deploy commit에 추적되는지도 확인하며, 누락되면 release를 실패시킨다. 따라서 차량 OTA는 검증된 결과를 그대로 사용하고 npm 같은 개발 의존성을 내려받지 않는다.

YAML · CONFIG

```
build:
  script: build.sh
deploy:
  built_assets:
    - web/dist
    - .build_hash
```

시작·중지 규칙 (lifecycle)

lifecycle 은 PackageManager가 stop/restart를 어떻게 다룰지 정하는 운영 정책이다.

YAML · CONFIG

```
lifecycle:
  stop_timeout_seconds: 30
  restart_on_aware_stop: true
```

필드	의미
<code>stop_timeout_seconds</code>	Docker stop, subprocess SIGTERM 대기, lifecycle command timeout의 기본값.
<code>pre_stop</code>	패키지 stop 전에 실행할 argv 또는 argv 목록. 문자열 호환 입력도 shell 없이 argv로만 해석한다.
<code>post_stop</code>	패키지 stop 후 실행할 argv 또는 argv 목록. pipeline·redirect·substitution 같은 shell 문법은 실행하지 않는다.
<code>restart_on_aware_stop</code>	Autoware 종료 후 잔여 ROS2 graph/cache 정리를 위해 재시작할지 여부. 미지정 기본값은 <code>false</code> . 자동 재시작이 필요한 패키지만 <code>true</code> 로 명시한다.
<code>restart_on</code>	lifecycle event별 재시작 정책을 mapping/list/string으로 표현할 때 사용.

Autoware 종료는 독립 `furive-launcher` 가 감지해 master에 `autoware_stopped` lifecycle event를 보낸다. master는 로컬 PackageManager와 연결된 agent에 `lifecycle_event: autoware_stop` 을 generation과 함께 전파하고 마지막 generation을 저장한다. 끊겼다가 재접속한 agent는 최신 generation을 한 번만 적용한다. PackageManager는 실행 중인 패키지 중 `lifecycle.restart_on_aware_stop: true` 를 명시한 패키지만 재시작하며, 이 값이 없거나 `false` 이면 재시작하지 않는다. dashboard는 이 lifecycle 경로에 관여하지 않는다.

다시 시작 조건 (restart_triggers)

restart_on top-level 필드는 설정 변경에 반응하는 재시작 조건이다.

YAML · CONFIG

```
restart_on:  
  - ros_domain_id
```

예를 들어 ROS domain 변경 시 `ros_domain_id`를 선언한 패키지 또는 manifest 안에 `ROS_DOMAIN_ID`를 명시적으로 참조하는 구형 패키지는 재시작 대상이 된다.

패키지가 소유하는 테스트

root pytest는 의도적으로 `package/`를 수집하지 않는다. 대신 CI 내부의 `run-package-tests`가 runtime과 같은 discovery로 `package/core/*`, `package/plugin/*`를 inventory하고 각 package의 test를 그 package cwd에서 실행한다. 독립 서비스 gitlink는 해당 package 저장소 revision을, root가 직접 소유하는 얇은 통합 package는 root revision을 증거로 사용한다. 개발 검사는 보이는 direct package도 실행하지만 release 검사는 root 전체가 clean하고 package가 추적된 상태여야 통과한다. `FURIVE_OS_EXTERNAL_PACKAGE_DIRS`로 명시한 독립 저장소도 manifest 검증과 같은 경로로 test한다.

현재 runner가 발견하는 형식은 다음과 같다.

- package root의 `test_*.py`, `*_test.py`, `test/`·`tests/`의 Python test
- `src/<ros-package>/test`·`tests`의 Python test
- `package.json`에 선언된 `scripts.test`
- `test/`·`tests/`·`scripts/` 아래 `test-*.sh`

official package에는 이 중 적어도 하나의 실행 가능한 suite가 있어야 한다. 발견된 test가 없거나 manifest를 읽을 수 없거나 suite가 실패·timeout되면 root/PR gate 전체가 실패한다. ROS나 장치가 없는 사무실 CI에서도 판정해야 하는 logic은 module shim, fixture와 fake adapter로 실제 분기를 실행한다. 장치가 필수인 test는 조용히 skip해 이 lane을 통과시키지 않고 별도 bench/HIL lane에서 필수 증거로 관리한다.

현재 runner는 no-tests·timeout을 fail-closed 처리하고 source 종류, 기대 revision, 실제 HEAD, 시작·종료 dirty status, 실행 worktree hash, 명령과 stdout/stderr가 포함된 JSON report를 만든다. 개발 PC에서는 아직 commit하지 않은 수정도 그대로 시험하되 그 사실을 report에 표시한다. trusted CI/release에서는 actual HEAD가 기대 revision과 다르거나 test 전후 source repository가 clean하지 않으면 실패한다. Python suite는 JUnit을 생성하며 skip 수가 하나라도 있으면 성공 코드를 실패로 승격한다. TAP Node suite도 test/pass/fail과 skip/todo/cancelled를 파싱해 exit 0 안의 필수 case 누락을 실패시킨다. trusted CI는 package JSON/JUnit과 root pytest JUnit을 함께 보존한다.

각 package는 위험에 맞는 세 경계를 소유한다. 계산·파싱·상태 전이는 production 함수를 직접 호출하는 unit test로, process/signal/filesystem/ROS adapter는 실제 endpoint와 격리된 fake dependency를 잇는 integration test로 검사한다. 사용자가 보는 핵심 결과는 release image/endpoint를 기동해 API, ROS message, 파일 또는 화면으로 확인하는 semantic E2E를 둔다. 장치가 필요한 E2E는 source test의 mock 통과로 대체하지 않고 bench/HIL lane으로 분리한다.

Native test는 기능 분기를 실제로 호출해 입력과 관측 결과를 assert해야 한다. 다음은 단독으로 의미 동작의 증거가 아니다.

- 파일·함수명·source 문자열이 존재한다는 검사
- PID, port 또는 Docker container가 살아 있다는 검사
- 아무 일도 하지 않는 `run.sh`, sleeper, 항상 성공하는 health
- mock 호출만 확인하고 외부에 보이는 payload·파일·ROS message를 확인하지 않는 검사

Release 증거에는 root SHA, 독립 package gitlink SHA, JUnit과 전체 log를 묶는다. RC/시험 배포까지 승격할 때는 같은 묶음에 exact artifact/image digest, 플랫폼/Autoware SHA와 ROS/API/파일 probe도 추가한다. Source test 통과는 RC semantic smoke나 차량 OTA/HIL 통과를 대신하지 않는다.

`package/shared/mgs` 는 package가 아니라 공용 ROS interface source다. Release base image는 그 저장소의 prebuilt `install` 을 ABI 증거로 신뢰하지 않고 target architecture에서 source를 clean build한다. Release export는 architecture별 `install-<arch>` 를 같은 `/data/furive-os/mgs/install` target의 상호 배타적인 runtime asset으로 기록한다. 차량 시작과 OTA activation은 해당 asset을 refresh하고 architecture/source digest가 일치하지 않으면 package를 시작하지 않는다. 개발 PC 설치도 architecture와 source digest가 모두 맞는 install만 재사용하며 먼저 build를 확인한 뒤 `/data` 로 동기화한다.

검증

Furive 연결 검증에는 다음을 실행한다.

```

● ● ● TERMINAL

./furive package check
```

이 명령과 release/CI는 중복 package name도 실패 처리한다. 내부 validator를 직접 호출할 필요는 없다.

```

● ● ● TERMINAL

./furive package list
```

발견된 package는 위 명령으로 빠르게 보고, 상세 런타임 discovery 결과는 다음 파일에서 확인한다.

```

FLOW / OUTPUT

logs/effective-packages-master.json
logs/effective-packages-agent.json
```

로그 디렉터리를 만들 수 없는 비정상 환경에서는 PackageManager가 `/tmp/furive-os/logs` 로 fallback하고 실행을 계속한다.

13. Autoware 플랫폼 탐색

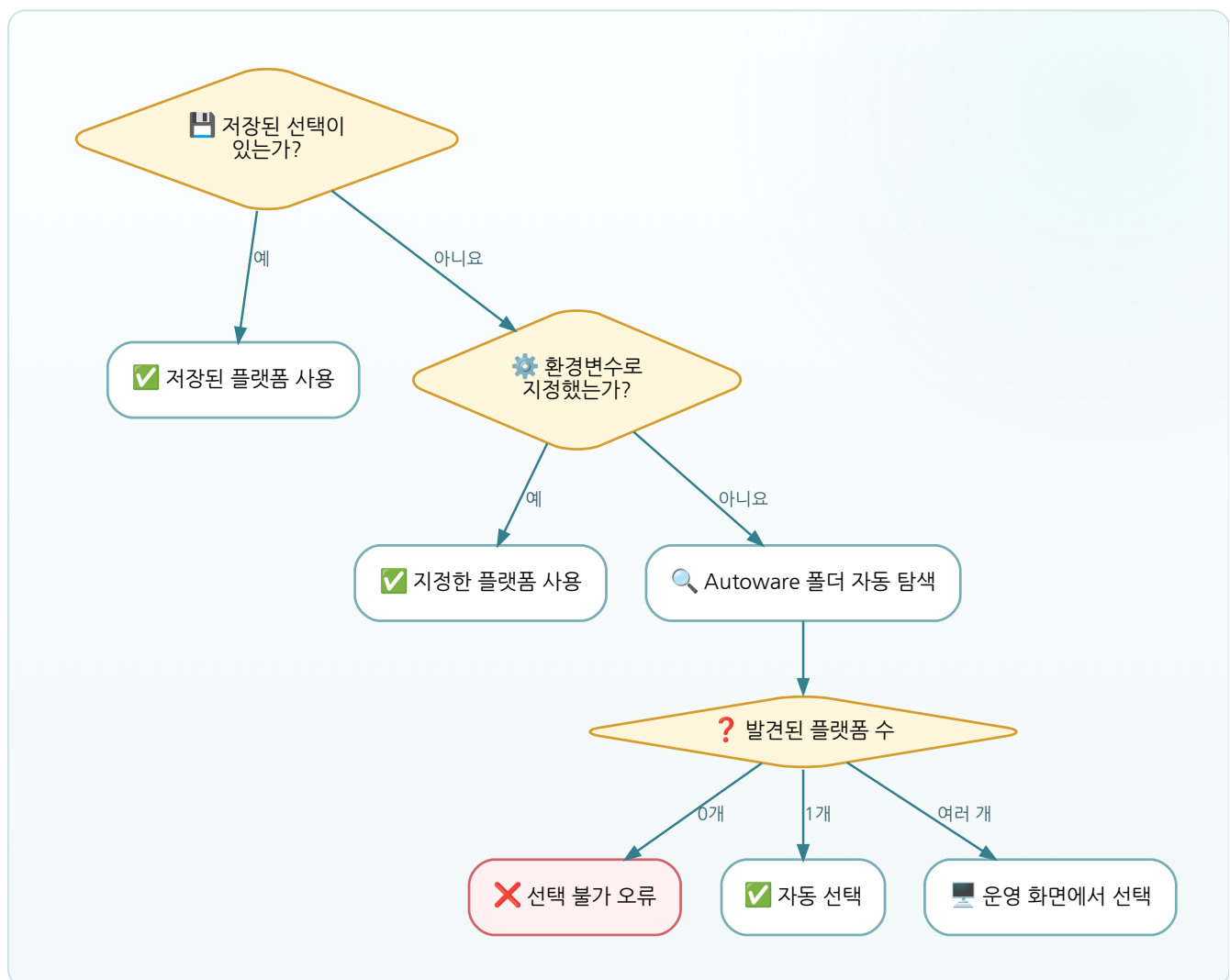
이 장의 목적 · Launcher · 플랫폼 유지보수자
Furive 플랫폼과 Autoware 모델 이름을 혼동하지 않고
탐색을 진단합니다.

완료 판단
0개·1개·여러 모델 발견 조건의 결과를 구분합니다.

이 문서는 main의 Autoware 실행 화면이 차량 모델과 센서 모델을 찾는 순서를 설명합니다. 일반적인 플랫폼 추가 절차는 [차량 플랫폼 만들기](#)를 따릅니다.

정상 경로에서는 사용자가 탐색 위치를 설정하지 않습니다. 같은 이름의 <플랫폼>_vehicle 과 <플랫폼>_sensor_kit 이 하나씩 있으면 자동 선택됩니다. 아래 환경변수와 선택 파일은 자동 탐색이 실패하거나 여러 플랫폼이 함께 발견됐을 때만 확인합니다.

선택 순서



정확한 우선순위는 다음과 같습니다.

1. `FURIVE_SELECTED_PLATFORM_FILE` 이 가리키는 파일의 값
2. `FURIVE_AUTOWARE_PLATFORM` 환경변수
3. Autoware 폴더 자동 탐색 결과

기본 선택 파일은 `$FURIVE_AUTOWARE_DIR/scripts/selected_platform.txt` 입니다.

자동 탐색 규칙

다음 경로를 순서대로 찾습니다.

FLOW / OUTPUT

```
$FURIVE_AUTOWARE_DIR/furive_autoware_main/src  
$FURIVE_AUTOWARE_DIR/src  
$FURIVE_AUTOWARE_DIR
```

전체 탐색 경로를 직접 지정할 때는 `FURIVE_AUTOWARE_PLATFORM_DIRS` 에 `:` 로 구분해 넣습니다. 이 값이 있으면 기본 목록을 대체하므로 기본 경로도 계속 찾으려면 함께 포함해야 합니다.

TERMINAL

```
export FURIVE_AUTOWARE_PLATFORM_DIRS="$HOME/Documents/Autoware/furive_autoware_main/src:$HOME/extra_autoware/src"
```

같은 이름의 차량 모델과 센서 모델이 모두 있어야 플랫폼 하나로 인식합니다.

FLOW / OUTPUT

```
<플랫폼>_vehicle  
<플랫폼>_sensor_kit
```

예를 들어 다음 두 폴더가 있으면 플랫폼 이름은 `ioniq5` 입니다.

FLOW / OUTPUT

```
furive_autoware_main/src/vehicle/ioniq5_vehicle  
furive_autoware_main/src/sensor/ioniq5_sensor_kit
```

Autoware 실행에는 다음 모델 이름이 전달됩니다.

FLOW / OUTPUT

```
vehicle_model:=ioniq5_vehicle  
sensor_model:=ioniq5_sensor_kit
```

여러 플랫폼이 발견된 경우

운영 화면에서 하나를 선택합니다. 선택한 이름은 `selected_platform.txt`에 저장되어 다음 실행에도 사용됩니다.

지도 선택

지도는 `$FURIVE_AUTOWARE_DIR/furive_map` 아래 폴더를 찾습니다. 선택한 지도는 기본적으로 `$FURIVE_AUTOWARE_DIR/scripts/selected_map.txt`에 저장됩니다.

14. 이전 정상 버전 복구 기준

이 장의 목적 · OTA 장애 조사자

업데이트 중 어느 상태에서 무엇이 보존되고 복구되는지
확인합니다.

완료 판단

실패 후보·정상판·설정 snapshot의 의미를 구분합니다.

이 문서는 준비된 새 버전을 언제 적용하고, 실패했을 때 어떤 상태로 돌아가는지 설명하는 내부 자료입니다. 운용자는 [차량 업데이트](#)의 화면 안내를 따릅니다.

버전 사이의 호환성과 중간 버전 선택은 [업데이트 호환 정책](#)에서 설명합니다.

이 문서에서 찾을 것

질문	담당 코드
어떤 배포 버전을 받을지, 언제 준비할지	<code>core/self_updater.py</code>
새 버전을 적용·확정하거나 직전 상태로 복구하기	<code>core/release_activation.py</code>
전환 중에도 멈추지 않는 시작 프로그램	최상위 <code>run.sh</code> 와 설치된 서비스
상태 표시와 사용자 입력	운영 화면과 <code>furive status</code>

각 패키지는 자신의 시작 방법과 정상 상태만 알려줍니다. 패키지마다 별도 업데이트·복구 프로그램을 만들지 않습니다. 아래 파일명과 설정명은 코드를 찾을 때만 쓰는 내부 이름입니다.

Furive는 새 버전을 발견해도 실행 중인 버전을 바로 덮어쓰지 않습니다. 새 버전을 별도 위치에 준비한 뒤 운영자가 차량의 안전 상태를 확인하고 재시작할 때 전환합니다. updater가 운행 안전 여부를 대신 판단하지는 않습니다.

한눈에 보는 동작 순서

FLOW / OUTPUT

현재 배포 버전 실행

- 현재 Git 폴더를 별도 후보 폴더로 복사
- 후보 폴더만 새 Git tag · commit에 맞춤
- 연결 저장소 · 대용량 파일 · Docker 이미지 · 빌드 준비
- 현재 버전은 계속 실행
- 다음 재시작에서 현재 폴더와 후보 폴더를 한 번에 교환
- 모든 패키지를 시작하고 정상 상태 확인
- 성공: 새 버전 유지, 직전 상태 1개 보관
- 실패: 직전 코드 · 설정 · Docker 이미지로 복구

Git 내려받기, Docker 이미지 받기, 소스 빌드 중 하나라도 실패하면 후보 폴더만 버립니다. 현재 버전과 실행 중인 패키지는 멈추지 않습니다.

`ota-updater` 하나만 새 버전 확인과 준비를 맡습니다. 운영 화면은 상태를 보여주고 사용자의 명령만 전달합니다.

재시작 때 전환하고 실패하면 복구하기

`run.sh`는 완성된 후보가 있으면 Linux의 디렉터리 교환 기능으로 현재 폴더와 후보 폴더를 한 번에 바꿉니다. 교환 직전에 서명, 목표 commit, Git 파일과 배포 정보를 다시 확인합니다. 두 폴더는 같은 디스크에 있어야 하므로 시스템이 설치 위치에 맞는 준비 폴더를 자동 선택합니다. 다른 개발 작업 폴더와 업데이트 상태를 공유하지 않습니다. 교환 뒤 새 코드가 모든 패키지를 시작합니다.

`furive setup --repair`는 기존 업데이트 기록이 현재 설치 폴더의 것인지 확인한 뒤에만 이어 씁니다. 경로가 남아 있다는 이유만으로 다른 설치의 기록을 사용하지 않습니다.

모든 패키지가 정해진 시간 동안 계속 정상이어야 새 버전을 확정합니다. 제한 시간 안에 하나라도 정상화되지 않으면 새 패키지를 먼저 종료하고 이전 버전으로 돌아갑니다. CAN 패키지는 프로그램 실행 여부뿐 아니라 두 CAN 장치가 실제로 준비됐는지도 확인합니다.

새 버전이 확정되기 전에 프로그램이 죽거나 다시 시작돼도 다음 시작에서 이전 버전으로 돌아갑니다. 실패한 같은 commit은 반복 적용하지 않으며, 더 새 버전이 올라오면 다시 준비합니다. 운영자가 직접 고정한 이전 버전은 자동으로 해제하지 않습니다.

정상 확인 뒤에도 업데이트 직전 상태를 바로 지우지 않는다. 대시보드의 **이전 정상** 또는 `updater`의 `previous` 복구를 선택하면 해당 시도 직전에 보존한 release tree, mutable Docker tag, `/data/furive-os` 아래 실행 설정을 함께 복구한다. 설정 snapshot은 이름이 `config`인 디렉터리와 루트 설정 파일만 포함하며 logs, datasets, binaries, version state는 포함하지 않는다. 따라서 주행 기록은 롤백 때문에 과거 상태로 되감기지 않는다. 새 후보를 준비할 때 복구 슬롯은 그 시점의 실제 현재 상태로 교체된다. 그 사이 Furive OS 업데이트가 여러 번 있었더라도 release에 미리 적힌 과거 버전을 찾지 않으며, 항상 이번 업데이트 시도 직전 상태 하나를 기준으로 한다. 대시보드는 이 즉시 복구 항목과 현재 channel에 보존된 과거 signed release 전체를 함께 보여준다. `preview/<vehicle_id>`는 그 차량의 test tag만, `main`은 suffix 없는 normal tag만 목록에 포함한다. **이전 정상**은 이미 보존한 전체 상태를 즉시 복구하고, 그보다 오래된 버전은 일반 업데이트와 같은 별도 준비·호환성 검증·다음 재시작 절차를 거친다. 목록 포함은 적용 가능 판정이 아니다. 후자는 같은 identity가 같고 설정 변환이 필요 없는 controlled return만 허용하며 검증에 실패하면 현재 release를 유지한다.

후보 staging이 완료되면 `updater`는 candidate marker, host supervisor, active/finalized 동일 filesystem, 현재 release에 결박된 Docker image snapshot을 읽기 전용으로 다시 검증한다. 이 복구 사전 확인 결과는 `rollback_readiness`로 상태에 포함되며, `apply_update`는 `ready`가 아니면 재시작 예약을 거부한다. Persistent config snapshot은 실제 값의 변경 경쟁을 줄이기 위해 release 전환 직전에 만들고 검증한 뒤, 성공한 경우에만 원자 교환한다.

Credential bundle은 과거 ZIP을 장기 rollback artifact로 삼지 않는다. 기존 설치에 bundle을 적용하는 경로도 candidate activation을 사용해 설치 직전 release를 retained previous로 남긴다. 완전히 새 PC에는 pre-install release가 없지만, 원격에 보존된 호환 과거 signed release는 일반 controlled return으로 준비할 수 있다. 오프라인 전달이 필요하면 fleet assignment의 정확한 version을 선택해 현재 credential로 bundle을 다시 만든다. Bundle이 release history나 rollback 정책을 별도로 소유하지 않는다.

상태 파일과 보존 디렉터리는 다음과 같다.

경로	의미
<code><staging>/finalized</code>	다음 재시작에 적용할 완성 후보
<code><staging>/old_furive-os</code>	다음 정상판을 준비하기 전까지 보존하는 이전 정상 release

경로	의미
<code><staging>/failed-furive-os</code>	자동 복구 후 보존하는 실패 후보
<code><staging>/persistent-config-snapshot</code>	직전 정상판의 실행 설정 복구본
<code><staging>/release-image-snapshot.json</code>	mutable Docker tag의 이전 image ID
<code><staging>/release-activation.json</code>	<code>idle</code> , 교환·롤백 <code>journal</code> , <code>guarded candidate</code> , <code>good</code> , <code>rolled back</code> , <code>error</code> 상태와 <code>commit/health</code> 기록
<code><repo>/rollback_pin</code>	같은 실패 후보 재적용 또는 수동 rollback 자동 업데이트 방지

설정

YAML · CONFIG

```
self_update:
  auto_apply_on_detect: false
  candidate_health_timeout_seconds: 180
  candidate_stable_seconds: 5
  # 생략/true는 일반 OTA. false인 Source Unlock package만 작업공간을 보호한다.
  package_updates:
    lotte-ohmio-mgmt: false
  # 생략하면 config의 packages 전체를 검사한다.
  # candidate_required_packages: [dashboard, lotte-ohmio-can-ros2]
```

기본값 `auto_apply_on_detect: false`에서는 업데이트 탐지만 주기적으로 수행하고, Dashboard에서 적용을 선택할 때 후보를 준비한다. 따라서 평상시 service 장애 재시작이 아직 승인하지 않은 일반 OTA를 우연히 활성화하지 않는다. `true`는 탐지 시 후보 준비까지 자동화해야 하는 통제된 환경에서만 사용한다. fleet의 exact target 수렴은 이 일반 채널 기본값과 별도 정책이며, OTA apply와 rollback은 모두 같은 staged `ReleaseActivation` 경로를 사용한다. `run.sh`의 시작 전 인플레이스 git sync도 기본적으로 꺼져 있으며, 유지보수 목적으로 꼭 필요할 때만 `FURIVE_OS_ALLOW_INPLACE_SYNC=1`을 명시한다.

후보 release에서 `ota-updater`가 실행 대상으로 선택됐다면 이 package는 `candidate_required_packages`나 `update.rollback_on_failure`로 후보 검증에서 제외할 수 없다. 후보의 프로세스 생존뿐 아니라 OTA 제어 응답과 polling thread를 확인한 뒤에만 정상 release로 확정한다. 응답하지 않거나 polling이 중단된 후보는 정상판이 되지 않으며, 후보 코드가 아닌 안정된 host launcher가 이전 release로 되돌린다. 단순 상태 관찰 timeout은 기존 복구 담당 프로세스를 즉시 종료하는 근거로 쓰지 않아, 오래 걸리는 검증 중 updater를 스스로 끊지 않는다.

상태는 다음 명령으로 확인한다.

TERMINAL

```
furive status
furive status --json
```

CI와 test 배포 검증기는 내부적으로 strict OTA 검사까지 수행한다. 사용자는 해당 구현 script를 직접 호출하지 않고 위 공개 상태 명령과 Dashboard를 사용한다.

업데이트할 프로그램 준비

- 일반 시작은 설치된 소스와 로컬 이미지를 사용하며 원격 소스 동기화나 이미지 일괄 pull을 기다리지 않는다. 이미지 배포 package의 기본 pull 정책은 `missing` 이다. 필요한 이미지가 없는 package만 다운로드가 필요하다.
- Docker image 배포 package는 `release/package-images.tsv` 의 선택 package 및 공용 base image를 staging 중에 준비한다. 정확한 digest의 이미지가 이미 있으면 재사용한다. 변경 가능한 태그는 staging에서 pull에 성공해야 하며, 통신 실패 시 이전 태그 캐시를 새 후보로 간주하지 않는다. 다운로드·사전 빌드가 실패하거나 지연되어도 현재 release를 교체하지 않고, 준비가 완료된 후보만 다음 재시작에 적용한다.
- `script`, `colcon`, `binary` source package의 `build.script` 는 staging checkout에서 실행한다.
- `source_unlock.run_mode: source` package는 `source_unlock.build_script` 를 staging에서 실행한다. 성공한 후보에는 release HEAD가 묶인 일회성 prebuild marker를 남겨 첫 PackageManager 시작이 같은 script를 다시 실행하지 않는다. MGMT는 build hash와 last-known-good runtime cache도 사용해 변경 없는 telemetry/UI 빌드를 건너뛴다.
- 차량에서 `self_update.package_updates.<name>: false` 로 OTA를 중단한 source unlock 개발 package만 실제 Git checkout을 `/data/furive-os/dev-sources` 에 영구 보관한다. OTA는 이 workspace를 stash/reset/checkout하거나 사전 빌드하지 않고 후보 release의 원래 package 경로에 symlink만 재연결한다. 해당 경로에서 개발자가 수행한 commit/push와 dirty 작업은 whole-release atomic swap 뒤에도 유지된다.
- master에 존재하는 외부 `furive_aware_main` Git checkout은 `release/external-workspaces.yaml` 에서 active `capability_profile` 에 고정된 40자리 commit SHA도 release 구성요소로 취급한다. 이 선택은 Furive OS branch/tag와 독립적이므로 동일 release에서도 `lotte-ohmio`, `ipg-g80`, `furive-niro` 가 서로 다른 Autoware SHA를 적용할 수 있다. staging에서는 target SHA를 fetch만 하고 atomic swap callback에서 checkout한다. 적용 직전에 루트/하위 repo의 실제 HEAD와 branch, tracked 변경 및 build/install을 한 묶음으로 캡처한다. candidate 또는 수동 release rollback은 release에 적힌 임의의 이전 commit이 아니라 바로 그 업데이트 시도 직전 상태를 재빌드 없이 복구한다. branch 값은 fetch/audit metadata이고 실제 적용 identity는 commit SHA이다. 정상 확인 뒤에도 이 복구 묶음 하나는 유지하고, 다음 Autoware OTA가 시작될 때 당시의 실제 상태로 교체한다. 각 snapshot은 이를 만든 Furive 후보 HEAD와 결합되어 다른 Furive-only update의 rollback에는 사용되지 않는다. `repos_manifest` 하위 repository는 release에 기록된 개별 40자리 SHA를 사용한다. 일반 release는 플랫폼별 빌드 fingerprint나 receipt를 요구하지 않고 exact root-하위 SHA만 고정하며 artifact에 `build_validation: not_performed` 를 기록한다. 실제 적용 build와 rollback은 외부 Autoware 업데이트를 명시적으로 켜 차량의 다음 경로가 소유한다.
- 외부 Autoware OTA on/off는 default-first/local-override 방식의 `self_update.external_updates.furive-aware-main` 으로 결정한다. 현재 default는 `false` 이며 local `true` 를 명시한 차량만 업데이트한다. `false` 이면 fetch와 checkout을 모두 생략해 source 작업공간을 hold한다. 이 경로는 Docker image pull/build를 포함하지 않고, 최초 설치의 Autoware image 준비와 분리한다.
- 외부 Autoware checkout 적용 직후에는 commit diff를 `package.xml` 에 매핑한 선택 colcon build를 실행하고 매핑할 package가 없으면 전체 build한다. 로그와 상태는 각각 `logs/furive_aware_build.log`, `logs/furive_aware_build.status.json` 에 보존한다. build 실패 시 candidate와 외부 source/build/install을 업데이트 시도 직전 상태로 함께 rollback한다. external update hold이면 snapshot, checkout, build, rollback을 모두 실행하지 않는다.
- staging 중 apt/colcon처럼 host를 오래 점유하는 package는 `update.prebuild: false` 를 사용한다. RS80은 이전 정상 release의 build/install/hash를 후보 시작 직전에 승계한 뒤, PackageManager가 현재 source hash를 다시 검사한다. source가 같으면 즉시 재사용하고 다르면 실행 전에 재빌드한다.
- 후보 준비 시 사전 빌드하면 안 되는 package는 `update.prebuild: false` 를 명시한다.
- `health_check` 는 단순 프로세스 존재보다 실제 사용 가능 조건을 표현해야 한다. 하드웨어가 필요한 package는 interface/device/topic 준비 상태를 검증해야 한다.

보장 범위

이 구조는 소프트웨어 update/build 실패 때문에 현재 정상 release가 사라지지 않도록 하고, 직전 정상판 한 개로 코드·이미지·실행 설정을 함께 복구한다. 과거 차량별 업데이트 경로의 1회 전환판은 이전 설정을 먼저 보존한 뒤 후보 기능 시작 직전에 설정을 보정하고, 보정 완료와 health가 모두 확인될 때만 확정한다. 어느 쪽이든 실패하면 과거 경로까지 복구한다. 기존 구버전 설정 변환 완료 이력도 전환판에 이어받아 끝난 작업을 불필요하게 다시 실행하지 않는다.

자동 legacy 인계의 최저 기준은 `v2026.06.17` 차량별 배포판이다. active tree의 구형 평문 marker만 신뢰하지 않고 marker의 이전/후보 commit, 실제 retained checkout, Git ancestry와 변경 수, 전환판의 deploy metadata·branch·tag·deferred migration을 함께 검증한다. 기존 candidate state가 있는 `v2026.07.07` 이후 배포판은 snapshot schema를 현재 journal로 보수적으로 승격한다. 첫 bridge가 확정된 뒤 host launcher에 supervisor contract가 없거나 후보가 요구하는 최소 contract보다 낮고, unit/token 연결도 구 계약이면 다음 staging은 `furive setup --repair` 완료까지 fail-closed한다. launcher byte hash 차이만으로는 막지 않는다. repair는 `candidate / switching` 상태에서는 시작하지 않고, 누락 unit을 다시 쓰며, 기존에 실행 중이던 service를 끝에서 재시작해 영구 token을 현재 updater에도 적용한다. 이보다 오래된 설치 또는 marker-backup 증거가 모호한 상태는 `setup --repair` 대상이 아니다. 최신 credential bundle의 `bootstrap.sh --mode <main|edge> --target-dir <현재 설치 경로>`로 known-good 배포를 복구한다.

후보 전환 순간에는 package 재시작에 필요한 짧은 중단이 있다. CAN 장치 자체 고장, 전원 단절, 디스크 손상처럼 이전 release에서도 해결할 수 없는 하드웨어 장애는 복구할 수 없다. logs/datasets/database 자체는 자동 snapshot하지 않으므로 이를 변경하는 migration은 반드시 이전판도 읽을 수 있는 방식으로 작성해야 한다. 파괴적 데이터 변환이 필요한 release는 이 일반 OTA 경로에 넣지 말고 별도 백업·복구 절차를 먼저 설계한다.

15. 업데이트 호환판과 구버전 설정 변환

이 장의 목적 · OTA 호환성 유지보수자

직접 이동할 수 없는 호환 세대의 중간 경로를 이해합니다.

완료 판단

변환 실패가 직전 정상 버전 복구로 끝나는지 확인합니다.

이 문서는 오래된 차량이 최신 버전으로 바로 갈 수 없을 때 Furive가 중간 호환판을 선택하고 복구하는 내부 계약을 설명합니다. 운용 절차는 [차량 업데이트](#)를 따릅니다.

운용자가 알아야 할 것은 **현재 버전**, **목표 버전**, **이전 정상 버전**뿐입니다. 내부 호환 번호와 설정 변환 방식은 Furive가 자동으로 선택합니다.

한눈에 보기



호환판은 오래된 버전과 최신 버전 사이를 잇는 중간 업데이트입니다. 코드에는 호환을 위해 `kind: bridge` 라는 이름이 남아 있지만 네트워크 Bridge와는 관계없습니다.

일반 `normal` 업데이트는 설정을 변환하지 않습니다. 현재는 구버전에서 사용자가 바꾼 설정만 골라 새 기본값에 옮기는 일회성 변환 하나만 있습니다.

하나의 OTA 흐름

원격 업데이트 흐름은 하나이며, 코드는 다음처럼 나뉩니다.

책임	구현 위치
허용할 업데이트 정책	<code>core/ota_policy.py</code>
목표 선택, Git 내려받기와 별도 폴더 준비	<code>core/self_updater.py</code>
폴더 교환, 설정 변경 기록, 확정과 복구	<code>core/release_activation.py</code>
주기 확인과 API 연결	<code>package/core/ota-updater</code>
호환 경로와 배포 버전 확인	<code>core/release_identity.py</code>

개별 패키지는 자신의 시작 방법과 정상 상태만 알려줍니다. 별도 업데이트·설정 변환·복구 프로그램을 만들지 않습니다. 운영 화면도 상태 표시와 사용자 입력만 담당합니다.

내부 호환 선언

배포 설명 파일(`release/manifest.json`)에는 현재 버전에서 다음 버전으로 안전하게 갈 수 있는 조건이 들어 있습니다.

사람이 이해할 의미	기존 내부 필드
일반판인가 중간 호환판인가	<code>kind: normal bridge</code>
받을 수 있는 이전 호환 세대	<code>source epoch/contract/config_schema/control_protocol</code>
적용 뒤의 호환 세대	<code>target identity</code>
PC의 시작 프로그램에 필요한 최소 기능	<code>minimum supervisor contract</code>
필요한 설정 변환 목록	<code>migration catalog</code>

`artifact_contract_version`은 배포 파일 형식 번호이고, OTA `epoch`는 업데이트 호환 세대입니다.

일반판은 `source`와 `target identity`가 같고 설정 변환이 0개여야 합니다. 호환 세대를 넘는 버전만 내부 `bridge`가 될 수 있습니다. 설정 `schema`까지 달라질 때만 설정 변환 파일 하나를 포함하고, `control protocol`만 달라지면 변환 파일은 없습니다.

현재 artifact v2는 일반판에도 빈 `migration catalog`와 `no-op checkpoint`를 기록합니다. 이는 게시된 `preview` 호환 경로를 위한 기존 wire 계약이지 새 개발자가 배워야 할 제품 개념이 아닙니다. 다음 artifact contract에서는 일반판의 빈 `catalog`를 없애고, 호환판에만 선택적인 단일 설정 변환 정보를 두는 것이 목표입니다.

목표까지 가는 경로

Furive는 서명과 배포 설명 파일을 확인했고 내용을 바꿀 수 없게 고정된 Git tag만 사용합니다.

1. 목표가 현재 상태를 직접 받을 수 있으면 목표를 준비합니다.
2. 직접 갈 수 없으면 목표 버전에 적힌 호환판 중 가장 짧은 경로를 고릅니다.
3. 한 번에 다음 버전 하나만 준비·적용·확정합니다.
4. 호환판이 확정되면 같은 최종 목표로 경로를 다시 계산합니다.
5. 필요한 호환판이 없거나 검증에 실패하면 현재 정상판을 계속 실행합니다.

경로는 최대 네 단계이며 중간 호환판은 최대 세 개입니다. 순환하거나 더 오래된 호환 세대로 가는 경로, 허용된 Git 이력 밖의 tag, 발행 뒤 내용이 바뀐 버전은 거부합니다.

`release_pin`은 최종 목표만 뜻합니다. 중간 호환판을 거치더라도 차량의 영구 목표 설정은 바꾸지 않습니다.

과거 버전으로 직접 돌아가는 요청은 현재판과 목표판의 호환 조건이 같고 설정 변환이 없는 일반판일 때만 허용합니다. 그 밖의 복귀는 직전 정상판 자동 복구 또는 새 설치 묶음을 사용한 정비 설치로 처리합니다.

임시 구버전 설정 변환

현재 `source`에서 실행 가능한 설정 변환은 `migrations/compatibility/epoch-1-to-2.py` 하나뿐입니다. 일반 확장점이나 DB migration 체계가 아닙니다.

- 새 설치와 일반판에는 변환 파일이 없습니다.
- 이전 버전의 기본값과 현재 설정을 비교해 사용자가 바꾼 값만 보존합니다.
- PC 명령, 네트워크 다운로드, 의존 프로그램 설치, 패키지 빌드·실행을 하지 않습니다.
- 변경 전 설정을 보관하고 변경 전후 기록을 남깁니다.
- 패키지를 시작하기 전에 끝나며, 실패하면 직전 정상 버전으로 돌아갑니다.
- 파일명이나 `.done` 파일만으로 완료를 인정하지 않습니다.

현재 `config/deploy-targets.yaml` 의 구버전 전환 gate가 열려 있으므로 이 파일의 경로와 기존 wire 값을 바꾸지 않습니다. 전환판 게시와 대상 fleet의 수렴이 확인된 뒤 source의 임시 변환기와 validator를 제거합니다. 게시된 immutable 호환판은 필요한 차량이 남아 있는 동안 보존합니다.

되돌릴 수 있는 설정만 변경

설정 변환 전에 복구본을 만들고 변경 내용을 기록합니다. 복구할 때는 이번 업데이트가 바꾼 값만 되돌리며, 그사이 운영자나 다른 프로그램이 바꾼 값은 덮어쓰지 않습니다.

새 필드는 먼저 이전판도 읽을 수 있게 추가하고, 바로 다음 버전에서 과거 형식을 제거하지 않습니다. 데이터베이스, 데이터셋, 주행 기록처럼 설정이 아닌 데이터를 되감는 작업은 이 경로에 넣지 않습니다. 그런 데이터는 해당 패키지가 별도 백업·복구 절차를 담당합니다.

발행과 보존

개별 `test` 채널에서는 호환판을 먼저 발행해 한 차량의 적용·전원 중단·health·rollback을 검증할 수 있습니다. 공식 공용 전환은 필요한 main과 기존 차량 channel의 호환판을 같은 source commit으로 검증한 뒤 한 묶음으로 게시합니다. 일부만 공용 최신판으로 먼저 노출하지 않습니다.

게시 뒤에도 곧바로 구버전 지원을 닫지 않습니다. 대상 차량이 호환판에 도달했거나 새 bundle로 복구됐다는 fleet inventory가 확인될 때까지 다음 `normal` 발행 gate를 유지합니다. 지원 차량이 참조하는 호환판 tag, deploy artifact와 image digest도 삭제하지 않습니다.

서명·보존·publication gate의 세부 운영은 [CI/CD 기준](#)을 따릅니다.

화면에 보이는 말

운영 화면은 내부 호환 번호나 wire 값을 보여주지 않고 다음 상태만 사용합니다.

- 최신
- 업데이트 있음
- 준비 중
- 적용 준비 완료
- 적용·확인 중
- 이전 정상 버전으로 복구됨
- 설치 복구 필요
- 오류

내부 identity와 호환판 tag는 장애 분석용 JSON과 이 reference에서만 확인합니다.

16. CI/CD 기준

이 장의 목적 · 릴리즈 엔지니어

코드 변경, release, bundle 중 이번 변경에 필요한 산출물을
판정합니다.

완료 판단

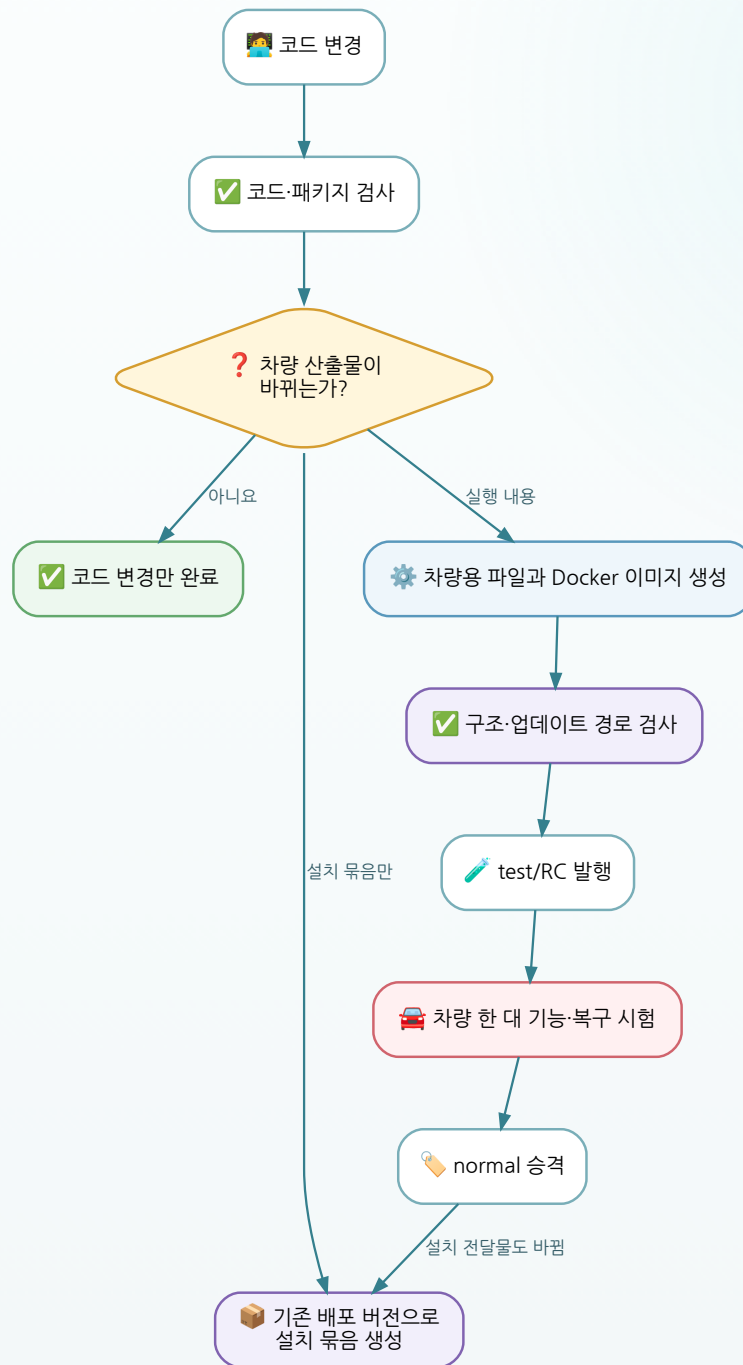
차량 실행 결과물이 바뀔 때만 release가 생성됩니다.

이 문서는 배포 자동화의 세부 입력·산출물과 현재 보장 범위를 조사할 때 보는 자료입니다. 평상시 발행 절차는 [차량 업데이트](#)를 따릅니다.

처음부터 끝까지 읽을 필요는 없습니다.

알고 싶은 것	볼 곳
지금 변경에 배포 버전이나 설치 묶음이 필요한가?	릴리즈 여부 결정
일상 개발·차량 시험·고객 배포에서 무엇을 확인하는가?	변경 검증
CI가 자동으로 검사·생성·수정하는 것은 무엇인가?	한눈에 보기부터 필요한 절
workflow 구현을 어디서 읽고 수정하는가?	워크플로 코드 위치
차량에서 버전을 발행·적용하는 명령은 무엇인가?	차량 업데이트

차량 실행 내용이 바뀌었는지는 `scripts/release/deploy_impact.py` 가 판단하고, 차량용 파일은 배포 자동화가 만듭니다.
패키지나 설치 묶음에 같은 판단 규칙을 다시 만들지 않습니다.



CI는 코드·패키지 검사와 배포 파일 구조를 자동 확인합니다. 실제 ROS/API/장치 기능, 보호된 시험 차량 배포와 HIL은 아직 모두 자동화되지 않았습니다. 따라서 CI 통과만으로 실차 기능까지 검증됐다고 판단하지 않습니다.

[!TIP] 관련 기준

운영/품질 기준 전체는 [변경 검증](#)을 따릅니다.

이 문서는 그중 GitHub Actions가 자동으로 해주는 일만 정리합니다.

워크플로 코드 위치

릴리즈 흐름은 `deploy-repo-release.yml`에서 읽습니다. 이 파일은 trigger, 작업 순서, 조건, runner, 권한과 단계 간 입출력을 소유합니다. 긴 실행 구현은 기존 `scripts/release/` 안에서 책임별로 관리하며, source 저장소 루트를 `cwd`로 사용합니다.

수정할 책임	구현 위치
source 검증 증거 선택·검증·보존	<code>ci-locate-source-evidence.sh</code> , <code>ci-verify-source-evidence.sh</code> , <code>ci-save-source-evidence.sh</code>
architecture·기존 deploy 상태	<code>ci-resolve-architectures.sh</code> , <code>ci-prepare-deploy.sh</code>
공통·package image build	<code>ci-build-base-image.sh</code> , <code>ci-build-package-images.sh</code>
deploy 산출물과 서명 전달물 준비	<code>ci-stage-deploy.sh</code> , <code>ci-prepare-signing-handoff.sh</code>
실패한 OTA 검증 재개	<code>ci-locate-staged-checkpoint.sh</code> , <code>ci-resume-staged-release.sh</code>
발행 대상·완료 증거 검증	<code>ci-validate-publication.sh</code> , <code>ci-validate-transition.sh</code>
보호된 서명·발행	<code>deploy-repo-sign.yml</code>

서명 workflow는 호출한 workflow와 같은 commit에서 실행되는 내부 `workflow_call`입니다. 직렬 실행과 선행 검증 조건은 호출자가, `production-signing` Environment와 서명 key 사용은 호출된 job이 소유합니다. 등록된 secret 이름과 기존 전달물 identity만 연결하며 새 운영자 입력은 없습니다. [GitHub의 재사용 workflow 규칙](#)에 따라 Environment secret은 호출된 job에서 해석됩니다.

서명 코드는 보호된 runner가 source를 checkout하거나 전달물 안의 실행 코드를 사용하지 않도록 전용 YAML 안에 유지합니다. 짧은 signer 사전 검사와 source 쓰기 권한을 갖는 transition 기록도 기존 checkout 없는 경계를 유지합니다. 일반 build·검증 구현을 이 경계로 옮기지 않습니다.

검증·build 구현 또는 서명 workflow가 바뀌면 이전 package 증거를 자동 재사용하지 않습니다. checkpoint 재개는 검증 구현의 제한된 변경만 허용하며 image-deploy 생성 코드 변경은 새 실행이 필요합니다. 코드 위치가 바뀌어도 증거를 다시 확인해야 하는 조건은 유지합니다.

릴리즈 여부 결정

배포 버전은 Git commit을 보관하기 위해 만드는 것이 아닙니다. 어떤 파일을 수정했는지가 아니라 차량이 실행할 내용과 설치 때 전달할 내용이 바뀌었는지로 판단합니다.

바뀐 산출물	해야 할 일	대표 사례
차량의 실행·설치 내용	<code>furive release test ...</code>	설치 코드, 업데이트 코드, 패키지·Docker 이미지 버전, 플랫폼·Autoware 버전, 차량 설정 변경
설치 묶음의 전달 내용만	<code>furive bundle <차량-ID></code>	접근 정보 교체, 지도 추가, 기존 배포 버전 선택, 같은 Docker 이미지 파일의 재포장
둘 다	먼저 <code>release</code> , 성공 후 <code>bundle</code>	새 실행 내용을 새 차량 설치 묶음으로 전달
둘 다 아님	코드 검사와 merge만 수행	문서·테스트, 검사 속도 개선, 개발 도구 내부 개선

판단 기준은 차량 실행 파일, Docker 이미지, 패키지과 Autoware 버전이 달라지는지입니다. Git commit이 달라졌다는 이유만으로 새 배포 버전을 만들지 않습니다. 반대로 차량 실행 결과가 달라지면 생성 도구를 직접 수정하지 않았어도 새 배포 버전이 필요합니다. 이미 발행한 버전의 내용은 바꾸지 않습니다.

설치 묶음이 설치 결과에 영향을 줄 때

설치 묶음은 기존의 서명된 배포 버전을 선택하고 접근 정보, 지도와 Docker 이미지 파일을 전달합니다. 이 전달 방식만 바뀌면 설치 묶음만 다시 만듭니다.

다음 경우에는 설치 묶음만 바꾸면 안 됩니다.

- `bootstrap.sh` 가 패키지·플랫폼·Autoware 버전을 새로 선택한다.
- 설치 묶음이 설치 옵션을 넘겨 계속 유지되는 실행 상태를 바꾼다.
- 전달한 Docker 파일이 배포 버전에 적힌 이미지와 다른 내용을 선택한다.
- 같은 배포 버전인데 설치 묶음에 따라 서비스나 설정이 달라진다.

이때는 설치와 실행 결정을 기존 설치·배포 설정으로 옮깁니다. 차량 실행 내용이 달라지면 배포 버전을 먼저 발행하고, 그 버전을 선택한 설치 묶음을 만듭니다.

실행 전 네 가지 질문

1. 차량이 실행할 코드·패키지·Docker 이미지·Autoware 버전이 달라지는가? 배포 버전을 만듭니다.
2. 같은 배포 버전에서 접근 정보·지도·전달 방식만 달라지는가? 설치 묶음만 만듭니다.
3. 설치 묶음이 계속 유지될 실행 설정을 경하는가? 그 설정을 배포 버전으로 옮긴 뒤 둘 다 만듭니다.
4. 두 산출물이 모두 같은가? release와 bundle을 만들지 않는다.

`release + bundle` 인 경우 순서는 반드시 signed deploy release 발행·검증 완료 후 bundle 생성이다. bundle은 성공한 기존 release만 pin하며, 아직 게시되지 않은 source commit을 release처럼 전달하지 않는다.

이 판단은 `scripts/release/deploy_impact.py` 한 곳이 소유한다. 알려진 문서·테스트·bundle-only 경로만 명시적으로 제외하고, exporter 입력·image build 입력·package gitlink·release workflow와 새로 생긴 미분류 경로는 기본적으로 deploy 영향으로 판정한다. 따라서 새 경로를 잘못 제외해 release가 빠지는 방향이 아니라, 소유권을 확인할 때까지 release가 필요하다고 보는 방향으로 실패한다.

- `furive release ... --run` 은 직전 같은 channel source tag 이후 deploy 영향 경로가 없으면 발행을 거부한다.
- `furive bundle` 은 선택한 signed deploy manifest의 `source_ref` 부터 현재 clean HEAD까지 검사한다.
- bundle-only·문서·테스트 변경만 있으면 기존 signed deploy release를 허용한다.
- 미릴리즈 deploy 영향 변경, 분기된 source history, dirty tracked source는 bundle 생성 전에 실패한다.

빠른 PR 검증과 전체 릴리즈 검증

변경	PR-merge queue에서 실행하는 검사
한 package의 코드·gitlink	전체 root 검사 + 변경 package와 manifest <code>dependencies</code> 의 간접 의존 package
root 문서 Markdown·Python test만	전체 root 검사; package suite 생략
공통 runtime·설치·설정·shared image 입력·CI 정책	전체 root + 전체 package
package 삭제·이름 변경, 미분류 경로, 비교 실패·dirty checkout	전체 검사로 복귀

변경	PR-merge queue에서 실행하는 검사
기존 대상의 정확한 Autoware SHA lock만	기존 Autoware lock 전용 검사

선택 정책은 기존 `scripts/checks/run-package-tests.py` 가 소유한다. root package 통합 계약은 계속 전부 실행하므로 checkout과 Python 공통 의존성 설치의 유지한다. Node 의존성은 선택된 package에만 설치한다. 새 commit이 들어온 PR의 이전 검사는 취소한다. merge queue와 수동 실행은 실행 도중 취소하지 않는다.

`furive check` 와 `release`는 전체 검증을 유지한다. CI 내부 비교 옵션은 검증 대상 선택에만 사용하며 새로운 사용자 절차는 없다. 부분 검사 JSON에는 전체 inventory, 선택 경로와 base/head를 기록하고, 전체 release 검증의 PASS로 재사용할 수 없다. 기존의 exact release evidence 재사용 정책은 유지한다.

Docker 이미지는 이미 package revision + 공통 base content + architecture로 재사용한다. root commit이나 다른 package만 바뀌었다고 동일 component를 다시 빌드하지 않는다. 공통 base가 바뀌면 이를 소비하는 package image는 다시 필요하며, release alias/digest 검증은 빌드 생략 여부와 관계없이 수행한다. 빌드 분기 회귀는 workflow의 실제 Bash를 실행해 cache hit/miss와 실패 후 발행 차단을 확인한다. 실제 registry-Docker 실행 시간 단축 수치는 CI 실행 후 비교해야 한다.

로컬 E2E의 비용과 실행 기준

E2E는 실제 프로그램들을 연결해 입력부터 API 응답·상태·파일·화면 결과까지 확인한다. 차량에 가지 않고 개발 PC의 격리 프로세스·컨테이너에서도 수행한다. 차량·센서 입력을 시험용 발행자나 재생 데이터로 공급할 수 있지만, 검증하려는 package·통신·저장 구현은 실제로 실행한다. 단위 테스트나 mock handler의 PASS를 해당 package E2E로 집계하지 않는다.

상황	실행·재사용 기준
일반 PR	위의 root + 영향 package 검사. 수초 내 끝나는 실제 gateway 프로세스 계약도 root에 포함한다. ROS·Unity 전체 E2E를 모든 PR에 추가하지 않는다.
통신 정책·메시지 ·HMI/MGMT·Zenoh·node 경로 복구·해당 E2E 변경	준비된 개발 PC에서 기존 통신 E2E를 실행한다. 변경 package가 직접 참조하는 공통 입력과 소비 경로도 함께 확인한다.
설치·OTA 변경	기존 설치·서명·activation 회귀는 PR에서 실행한다. 실제 설치본·업데이트 후보의 기능과 복구를 확인하는 E2E는 임시 VM/컨테이너에서 수행하며, 합성 health 회귀와 구분한다. 이 경로의 자동화는 아래 lane 2에 남은 작업이다.
다른 코드·문서만 변경	무관한 긴 E2E는 반복하지 않는다. 영향 경계를 판정할 수 없으면 관련 E2E 범위를 넓힌다.
실패·시간 초과·의존성 없음	실패 또는 미검증으로 남긴다. 자동 재시도로 PASS를 얻거나 다른 입력의 이전 성공을 가져오지 않는다.

현재 통신 E2E는 개발 PC 실행 경로이며 GitHub 필수 job이나 nightly로 연결되어 있지 않다. 자동 gate에 연결할 때는 package revision, 공통 통신 코드·정책·메시지, test fixture, 실행 image digest와 HMI 산출물, 실행 환경이 같은 완료 증거만 재사용한다. source SHA나 image tag 하나만으로 동등성을 판단하지 않는다. 부분 E2E 통과를 전체 release 검증으로 승격하지 않는다.

기존 통신 runner의 단계 상한은 메시지 입력 확인 30초, HMI 시험 코드 빌드 60초, MGMT 빌드 300초, DDS/HMI/MGMT/QUIC 실행 180초, node 제어면 복구 150초다. 빌드 포함 합계 12분에 제한된 정리 시간이 추가된다. 각 단계의 PASS/FAIL과 실제 시간을 로그에 남기고 실패·취소 시 자신이 만든 프로세스와 컨테이너를 정리한다. 이 값은 대기 상한이며 CI 완료 시간의 보장이나 측정된 p95가 아니다. 실제 E2E 실행 상한은 데이터면 180초 + 제어면 150초다. MGMT source 빌드는 2 CPU·16 GiB로 제한하며 swap을 사용하지 않는다. ROS 메시지·Unity 플레이어·image의 최초 준비 비용은 따로 기록한다. 현재 입력과 일치하는 기존 산출물을 사용하며, 매 E2E마다 전체 개발 환경을 다시 빌드하지 않는다.

검증 lane과 현재 상태

Release 승격은 다음 네 lane의 증거를 서로 대체하지 않고 누적해야 한다. 구현됨 은 저장소에 실행 workflow와 fail-closed 계약이 있는 범위만 뜻한다.

lane	필수 판정	현재 자동화 상태
1. PR root + tracked package executable tests	검토 대상 root SHA와 그 index의 <code>package/core</code> , <code>package/plugin</code> 에서 실행 가능 test를 수행한다. PR은 위 변경 범위를, release는 전체 inventory를 사용한다. no-tests, 실패, timeout, 필수 pytest skip과 Node TAP의 0건 ·skip·todo·cancel은 모두 실패다.	<code>.github/workflows/quality-checks.yml</code> 의 trusted job과 release source gate가 root 검사 및 내부 <code>run-package-tests</code> 를 실행한다. runner는 Python/ROS Python, TAP Node test와 <code>test · tests · scripts</code> 아래 shell <code>test-*.sh</code> 를 package별 cwd·임시 경로에서 최대 네 개 병렬 실행한다. CI에서는 package HEAD가 gitlink와 다르거나 시작·종료 worktree가 dirty이면 실패한다. 개발 검사는 큰 payload를 다시 읽지 않고 actual HEAD만 기록한다. root/package JUnit과 package JSON report를 artifact로 보존한다.
2. RC exact artifact/image semantic smoke	export된 RC commit/artifact와 <code>repository@sha256:...</code> image만 기동해 정상·고장 입력의 기능 결과와 cleanup을 검사한다.	정책/필수 후속. 현재 deploy artifact 검사, registry digest 확인, recovery dry-run은 구조 검증이다. updater 검증도 합성 health를 사용하며 실제 package semantic smoke가 아니다.
3. 보호된 test 배포	normal 기준판의 등록 차량을 <code>preview/<vehicle_id></code> test release로 올려 실제 ROS/API/장치 probe를 통과시키고 normal 기준판으로 rollback한다. Autoware revision 변경도 적용·복원 SHA로 입증한다.	정책/필수 후속. preview route, candidate/rollback 코드와 합성 회귀는 구현됐지만 보호된 시험 차량에서 이 순서와 증거 수집을 수행하는 workflow는 없다.
4. nightly/bench HIL	CAN, 센서, GPU/display, NAS 등을 연결한 bench에서 장시간·재부팅·단절·복구 시나리오를 실행한다.	정책/필수 후속. bench inventory, 예약, probe/JUnit 수집과 release 차단 연결이 필요하다.

보호된 test 배포 host의 내부 post-apply gate는 다음과 같이 호출한다. `--expected-version` 과 `--expected-source-ref` 를 항상 함께 넘겨야 형식이 올바른 임의 release가 아니라 CI가 선택한 exact 대상을 증명한다.

```

 TERMINAL

python3 scripts/checks/verify-ota-update.py --strict-runtime \
  --expected-version <release-tag> \
  --expected-source-ref <40-character-lowercase-source-commit> \
  --json

```

이 gate는 최신이며 실행 중인 OTA 상태, canonical manifest identity의 형식과 exact match, active systemd service, 실행 가능한 package entrypoint/binary, release lock의 Docker image와 그 image ID로 실행 중인 container를 확인한다. 상태를 조회할 수 없거나 증거가 없으면 warning으로 통과시키지 않고 nonzero로 종료한다. 다만 이는 runtime liveness/identity gate이며 ROS/API/장치의 semantic probe는 아니다. Release source gate는 release branch가 선택한 Autoware 플랫폼의 root commit과 `.repos` 하위 저장소가 모두 source-owned 40자리 SHA로 고정됐지만 확인한다. Generic release runner는 플랫폼마다 다른 의존성·container·장치를 재현할 수 있다고 가정하지 않으며 Autoware를 build하지 않는다. 생성된 `release/external-workspaces.yaml` 에는 `build_validation: not_performed` 가 명시된다. 이미 존재하는 deploy tag도 source에서 다시 생성한 external-workspace lock의 hash가 실제 tag blob 및 manifest hash와 같을 때만 up-to-date로 처리한다. 따라서

사람이 계산한 fingerprint나 이전 tag의 source SHA만으로 release를 통과시킬 수 없다. `furive check --platform <플랫폼>`은 해당 플랫폼을 build할 수 있는 PC에서 쓰는 선택적 로컬 진단이며 release receipt를 만들지 않는다. 고객용 `normal`의 실행 증거는 시험 차량에서 launcher가 고정 revision을 실제 기동한 SHA와 launch/ROS probe로 남긴다.

기존 `ipg-g80`과 `furive-niro`는 root SHA만 고정돼 차량 적용 때 manifest branch tip을 다시 해석하던 상태였다. 이번 source lock부터 각 pinned root manifest의 현재 해석 결과를 nested SHA로 직접 고정해 같은 release가 이후 다른 source를 받지 않게 했다. 이 결정성 보강은 build 검증을 뜻하지 않으며 세 플랫폼 모두 artifact의 `build_validation: not_performed` 계약을 따른다.

fork PR은 private submodule credential을 받지 않으므로 gitlink fixture로 gate 자체의 fail-closed 동작만 검사한다. 실제 package source를 포함하는 lane 1은 같은 저장소 PR, merge queue 또는 승인된 수동 실행의 trusted job에서 수행한다. tracked gitlink checkout이 누락되거나 다른 SHA이면 이 job은 먼저 실패한다.

개발 속도와 실행 경계

테스트 개수나 전체 build 반복을 안전성의 대리 지표로 쓰지 않는다. 설치 script 변경은 source 문자열 존재만 확인하지 않고 installer 함수의 실제 분기와 외부 명령 경계를 짚은 fake host/Docker로 실행한다. 예를 들어 Autoware bootstrap 계약은 pin된 Dockerfile/Compose의 최종 bundle seed 선택, registry pull 금지, tracked `devcontainer.json` Compose 목록과 checkout 무변경, 선택 `BASE_IMAGE`만 `--build-arg`로 넘긴 `.devcontainer/Dockerfile` 파생 image build, identity-bound local tag, ROS setup 후 그 image의 `docker run`과 install 결과 확인을 한 흐름으로 검증한다. 이 계약은 실제 차량 장치 시험을 대체하지 않지만 shell 환경 누락을 full Autoware build 전에 잡는 빠른 실패 경계다.

`furive check`의 root suite와 package suite는 서로 상태를 소유하지 않는 독립 lane이므로 동시에 실행한다. 어느 한쪽도 다른 쪽의 성공을 대신하지 않으며 두 종류 코드가 모두 성공해야 다음 artifact 검사로 간다. 플랫폼 전체 Autoware build는 일상적인 root script·문서·테스트 변경마다 반복하지 않고, Autoware revision, image 또는 build recipe가 바뀔 경우와 명시적인 플랫폼 진단에서 수행한다. 재사용 증거는 exact source/image identity에 결박하며 identity가 다르거나 판단할 수 없으면 재실행한다.

PR 또는 release delta가 `config/external-workspaces.yaml` 한 파일뿐이고 YAML 의미 비교 결과가 기존 target의 root `commit`과 `repositories` exact SHA 값 변경으로만 증명되면 `autoware-lock-only` lane을 사용한다. 이 lane은 private package submodule·LFS·Node 의존성을 받지 않고 Autoware source 선택·render·credential·release 계약만 실행한다. target 추가, branch·workspace·build metadata 변경, 다른 파일 변경, 잘못된 SHA 또는 base revision을 확인할 수 없는 경우에는 자동으로 전체 root/package 검증으로 돌아간다. 이 최적화는 deploy release 필요 여부를 바꾸지 않으며 generic runner가 Autoware build 성공을 주장하게 만들지 않는다.

bundle 생성도 같은 원칙을 따른다. 서로 독립인 main/edge bootstrap smoke는 같은 read-only deploy object cache를 shared lock으로 읽으며 병렬 실행하고, cache 갱신만 exclusive lock이 소유한다. 실제 차량 bootstrap은 branch와 exact signed tag를 한 fetch transaction으로 받고, desired와 landing release identity가 같은 fresh install에서는 같은 서명 검사를 반복하지 않는다. 어느 최적화도 exact commit/tag 검증, 원격 branch 재확인 또는 ZIP byte/mode 전수검증을 생략하지 않는다.

승격 증거 묶음

증거	결박할 대상
source identity	root SHA와 모든 대상 package gitlink SHA
deploy identity	deploy commit/artifact hash와 실제 실행 OCI image digest
Autoware identity	플랫폼 이름, Autoware root SHA와 관리 대상 하위 저장소 SHA
자동 test	root/package JUnit, package test report, 종료 코드와 전체 log

증거	결박할 대상
runtime semantic probe	입력·기대값·관측값이 담긴 ROS topic/service, CAN, API 또는 파일 probe 원문

각 파일은 release ID, workflow run, lane과 시간을 함께 기록한다. PID/container 생존이나 source 문자열 검사는 보조 구조 증거일 뿐 semantic PASS가 아니다. 필수 test 또는 probe의 skip은 PASS나 조건부 성공이 아니라 FAIL이다. 현재 PR job은 package gitlink와 stdout/stderr를 담은 JSON report, package Python suite별 JUnit과 root pytest JUnit을 함께 보존한다.

한눈에 보기

순서	CI/CD가 해주는 일	결과
1	runner host 의존성·Docker·vcan을 read-only preflight한다	긴 다운로드 전에 잘못 준비된 runner를 즉시 차단
2	독립 source-contract job이 exact submodule/LFS와 root/package 검사를 수행한다	build 실패 재시도에서 이미 통과한 QA를 반복하지 않음
3	HEAD의 <code>v*</code> tag를 deploy branch와 매칭하고 기존 deploy 상태를 확인한다	처리할 release 목록 결정, 동일 release면 private hydration도 생략
4	pending release에만 source/submodule/LFS를 준비한다	image/export 입력 확보
5	dashboard asset을 build한다	deploy repo에 넣을 dashboard 산출물 생성
6	Harbor에 로그인한다	image push 준비
7	content identity가 같은 base image는 재사용하고, 없을 때만 build/push한다	공통 runtime image 생성
8	shared ROS runtime을 위 exact base image에서 추출한다	같은 colcon build의 이중 실행 방지
9	없는 Docker package component image만 build/push한다	package별 실행 image 생성
10	deploy repo 파일을 export한다	차량이 받을 release repo 생성
11	Harbor image tag를 registry digest로 고정한다	tag 재지정에도 실행 image 불변
12	SPDX SBOM·SLSA provenance와 hash index를 생성한다	component/image subject의 release 증적 생성
13	deploy artifact와 공급망 증적을 검증한다	manifest/image/secret/recovery/evidence 확인
14	아직 push하지 않은 local tag/branch를 checkpoint로 보존한 뒤 OTA upgrade path를 검증한다	이전 release에서 새 release 감지/prepare 확인, 검증기 수정 시 동일 staging 재사용
15	격리된 job에서 tag를 서명하고 exact lease로 atomic push한다	검증·서명된 차량 OTA 대상을 한 번에 공개
16	legacy transition 완료를 read-only job에서 검증한 뒤 checkout 없는 write job이 고정 receipt tag만 기록한다	deploy read credential과 source write token 분리

언제 실행되나

방식	조건
자동 실행	<code>main</code> branch push. 기본은 job skip이며 release 작업은 하지 않음
자동 실행	canonical <code>vYYYY.MM.DD.N</code> tag push 한 건
수동 실행	GitHub Actions <code>workflow_dispatch</code>

branch push release job은 기본적으로 skip되어 source submodule checkout, LFS hydrate, Docker build, deploy repo export를 수행하지 않는다. 따라서 `scripts/release/release-sync.py` 기본 실행은 `main` 과 canonical tag 한 건만 발행한다. 운영상 branch push에서도 release 작업을 허용해야 하는 경우에만 repository variable `ENABLE_BRANCH_PUSH_RELEASES=true`를 설정한다.

일회성 전환처럼 source release tag가 여러 개인 경우에는 alias tag를 먼저 한 번의 atomic push로 발행하고 canonical `main` tag를 마지막 단독 push로 발행한다. GitHub는 한 push에서 tag를 3개 넘게 만들면 push event를 생성하지 않으므로, 마지막 canonical tag가 전체 tag 집합의 공개 barrier이자 workflow trigger다. Resolver는 canonical event에서 같은 source commit의 alias가 하나라도 빠지면 실패한다. 따라서 자동 발행에는 source repo Git SSH 쓰기 권한만 필요하며 로컬 `gh` API token은 필요하지 않다. 수동 `workflow_dispatch` 와 CLI로 run 상태를 조회할 때는 유효한 `gh` 인증 또는 GitHub Actions UI가 별도로 필요하다. 자세한 event 제한은 [GitHub Actions의 push event 문서](#)를 따른다.

수동 실행 입력값:

입력	하는 일
<code>version</code>	특정 release tag만 처리
<code>deploy_branch</code>	기본 <code>main</code> . 과거 branch 복구 작업에서만 명시
<code>staged_release_run_id</code> , <code>staged_release_run_attempt</code>	내부 복구 전용. 실패 run의 exact staging checkpoint에서 OTA 검증만 다시 시작
<code>target_arch</code>	build 대상 architecture 선택: <code>auto</code> , <code>amd64</code> , <code>arm64</code> , <code>all</code>

OTA upgrade 검증이 실패해도 이미 통과한 source/package 검증, image build/push, deploy export를 다시 실행하지 않는다. 실패 run이 서명 전 업로드한 7일 보존 checkpoint를 지정하면 현재 main의 OTA verifier와 release test 변경만 허용해 재검증하고, 동일 source SHA·release set·staged commit을 그대로 서명 경계에 전달한다. 그 밖의 source·package·config 변경이 섞이면 checkpoint 재사용은 실패한다.

같은 source SHA의 앞선 run에서 clean source/package 증적 artifact가 남아 있으면 preview와 normal 재시도 모두 그 exact artifact를 사용한다. 이때 Python/Node setup, private package/LFS hydration과 11분 source/package matrix를 다시 실행하지 않는다. 새 source SHA이거나 artifact가 만료·불완전하면 자동으로 전체 gate로 돌아간다. Release image는 component content identity의 registry manifest가 이미 있으면 다시 build/push하지 않으므로, 24분 image 작업도 동일 입력에서 반복하지 않는다.

새 test tag에서 이전 package 증적을 상속할 수 있는 변경은 다음과 같다. 선택은 기존 `ci-locate-source-evidence.sh` 가 소유하며 재사용 전 checksum·revision·전체 inventory를 검증한다.

변경	다시 실행할 검사
명시된 deploy/OTA 검증 도구·bundle/export 도구와 <code>tests/release/</code> 만	기존 release 회귀 묶음

변경	다시 실행할 검사
root Markdown 문서·Python test만	전체 root pytest와 보안·layout·문서 링크·설정 변환·manifest 검사
정확한 Autoware SHA lock만	기존 lock 전용 검사
공통 runtime·설치·설정·package source·검증 정책·미분류 입력	전체 source/package 검사

앞의 세 경우도 gitlink와 검증 입력이 일치하고 완전한 성공 증적이 있을 때만 package suite를 생략한다. Package test는 공통 core와 설정을 소비할 수 있으므로 gitlink 일치만으로 충분하지 않다. 특히 계약 테스트 변경을 release 회귀 묶음으로 대신하지 않는다. 증적이 없거나 만료·불완전하면 전체 검사로 돌아간다. 이미지 content cache와 동일 source SHA 재시도 정책은 위 검사 선택과 별개다.

Full package lane은 서로 격리된 네 worker로 실행한다. `scripts/check.py` log에는 gate별·전체 시간, 가장 느린 root test 10개와 package/suite별 시간을 남긴다. Package JSON/JUnit도 함께 보존한다. 병렬 suite 시간의 합은 경과 시간과 다르므로 비교할 때 같은 runner·변경 범위·cache 조건을 사용한다. 재실행한 Python suite는 자기 JUnit을 먼저 제거해 이전 PASS를 새 실행의 증거로 읽지 않는다.

release 선택 방식

CI/CD는 canonical release tag를 공통 `main` channel에 발행한다.

tag	deploy branch
<code>v2026.08.04.1</code>	<code>main</code>

[!WARNING] branch/tag 매칭

차량 ID나 고객명 suffix를 release tag에 붙이지 않는다. 차량 assignment는 provisioning config의 책임이다.

동시 실행과 최신성 보호

canonical `vYYYY.MM.DD.N` tag 한 건에서 source 계약과 build/publication은 서로 다른 job이다. Source job이 성공하고 뒤 build·stage·publish job만 실패했다면 GitHub의 **Re-run failed jobs**를 사용한다. 이 경우 성공한 source job은 다시 실행하지 않는다. self-hosted runner는 private worktree를 매번 exact gitlink로 다시 만들지만 LFS object는 repository URL과 OID로 구분한 0700 content cache에서 먼저 복원하고 `git lfs fsck` 후 사용한다. 따라서 첫 실행만 큰 binary를 내려받으며, cache 손상은 exact ref를 다시 fetch해 복구한다. 이 cache의 hardlink와 checkout object가 같으면 검증한 521MB payload를 cache로 다시 복사하지 않는다. deploy repo의 Git object도 전용 release runner의 0700 bare cache에서 재사용하되, branch와 tag는 매 run read-only key로 강제 fetch한다. 빈 cache는 SHA256과 Git bundle 검사를 통과한 직전 local staging checkpoint로 seed할 수 있으며 checkpoint 자체를 최신 ref의 권위로 사용하지 않는다. 같은 SSH credential 경계 안의 GitHub 연결은 private control socket으로만 재사용하고 key cleanup 전에 master 연결을 닫는다. pip와 npm도 lock file 기반 cache를 사용한다. branch push release job은 기본 skip한다. workflow 전체는 전역 `concurrency` 와 `cancel-in-progress: false` 로 직렬화한다.

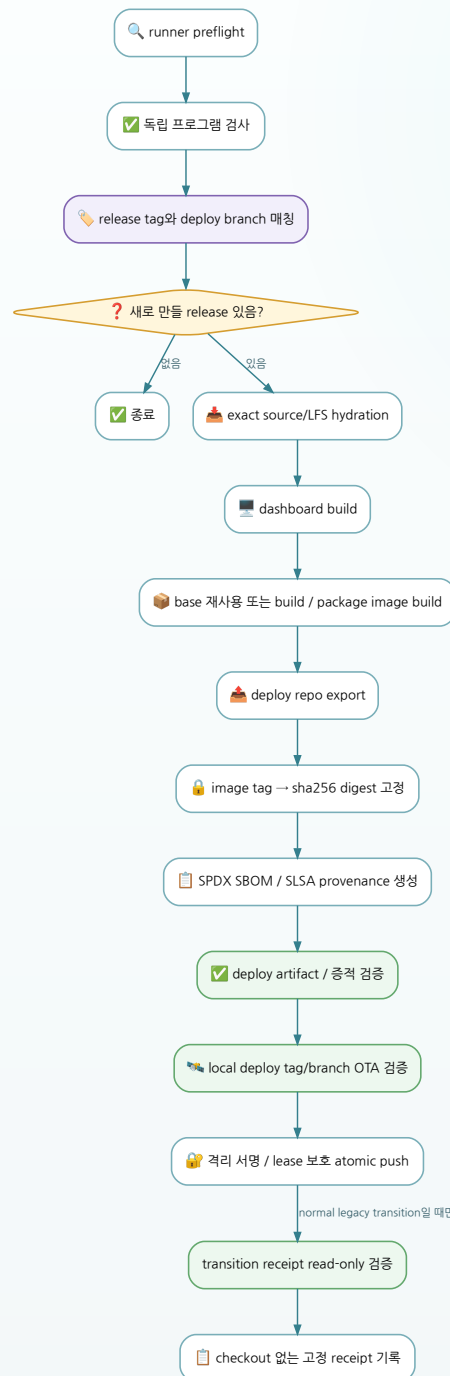
Harbor의 package component 조회와 release alias 생성은 package별 독립 작업을 최대 네 개 병렬로 수행한다. 최종 deploy artifact에서 서로 독립적인 image manifest 존재 확인도 최대 여섯 개를 병렬로 조회한다. build가 실제 필요한 경우에도 이 상한을 유지해 release runner와 registry를 무제한 점유하지 않으며, 하나라도 실패하면 전체 단계가 실패한다.

source tag 목록은 private package hydration과 분리된 짧은 read-key step에서 run-scoped cache로 한 번 조회하며, lightweight/annotated tag를 모두 commit SHA로 정규화한다. 이 조회가 실패하면 no-op으로 간주하지 않고 즉시 실패한다. deploy repo 상태 확인 단계에서는 source repo의 현재 release tag가 아직 이 run의 GITHUB_SHA를 가리키는 release pair만 pending list에 남긴다. release image alias 생성과 deploy repo atomic push 직전에도 cache를 갱신해 같은 조건을 다시 확인한다.

자동 push 이벤트에서 이미 더 새 source commit으로 tag가 이동했으면 해당 release pair는 notice를 남기고 건너뛴다. 이 guard는 오래된 run이 늦게 끝나면서 불필요한 build를 계속하거나 Harbor release image tag, deploy repo branch/tag를 이전 source 기준으로 덮는 상황을 막기 위한 장치다. 수동 workflow_dispatch는 과거 release 재생성이 의도될 수 있으므로 이 최신성 guard를 강제하지 않는다.

최신성 guard의 공통 구현은 scripts/release/ci-release-helpers.sh의 source_release_is_current_or_skip이다. tag 없음·이동은 stale event로 건너뛰지만 tag inventory 조회 오류는 workflow 오류로 분리해 fail-closed한다.

전체 흐름



Record completed legacy transition receipt 가 test /preview release에서 skipped인 것은 정상이다. 이 one-time 기록은 과거 normal 배포 경로를 새 서명 계약으로 완전히 전환한 normal release에만 적용하며, 일반 preview 발행의 성공 조건이 아니다. 서명 직전의 작은 revalidation은 private key/Environment 경계에서 staging bytes와 lease가 그대로인지 확인하는 보안 검사라 생략하지 않지만 source build나 package test를 반복하지는 않는다.

자동으로 하는 검증

1. 루트와 source 패키지 검증

PR trusted job과 release image/build 전 gate에서 root test와 source package 기본 규칙을 확인한다.

상세 기준은 [프로그램 연결 설정](#)을 따른다.

검증	확인 내용
root regression	root pytest, 개발 보안, script/test layout, migration과 문서 link 계약
documentation	<code>scripts/checks/check-doc-links.py</code> 로 root repo가 소유한 Markdown 상대 링크 확인
package manifest	<code>scripts/checks/validate-package-manifests.py --strict-duplicate-names</code> 로 <code>package.yaml</code> 구조와 중복 package name 확인
tracked package test	root index의 gitlink SHA에 checkout된 각 package의 선언된 Python/Node/shell suite를 격리된 package cwd에서 실행. 일부 shell suite는 production native code를 compile/run하며 no-tests, 실패와 timeout은 전체 gate 실패
Autoware source lock	release 대상 플랫폼만 export하고 root와 모든 nested repository를 source config의 40자리 SHA로 고정. CI build/receipt 검증은 하지 않으며 artifact에 <code>build_validation: not_performed</code> 명시
shared ROS message ABI	base image를 checked-in prebuilt가 아닌 <code>package/shared/msgs/src</code> 에서 architecture별로 clean build한다. Deploy runtime asset은 같은 source를 다시 build하지 않고 content identity가 일치하는 exact base component image의 <code>/opt/furive-os/msgs/install</code> 에서 추출한다. Export는 <code>install-<arch></code> 와 source digest marker를 묶고 verifier가 target architecture 전부를 확인한다. 차량 시작/OTA는 선택된 asset을 <code>/data/furive-os/msgs/install</code> 에 강제 refresh한 뒤 arch/digest가 다르면 시작하지 않는다. 개발 PC도 architecture와 source digest가 모두 맞는 install만 재사용한다.
source built asset	<code>deploy.built_assets</code> 로 선언한 CI 빌드 결과가 deploy export에 존재하는지 확인; 누락 시 release 중단
OTA image 확인	deploy artifact 단계에서 digest와 registry 존재를 확인하고, OTA 적용-롤백 시뮬레이션에서는 같은 image의 중복 pull을 생략
package metadata	<code>name</code> , <code>version</code> , <code>type</code> , <code>nodes</code> , <code>run.script</code>

검증	확인 내용
package type/node	허용된 type과 master / agent node인지
script	run.sh, setup.sh, build.sh 존재와 shell 문법
Docker package	docker.image_name, build.script 또는 Dockerfile; 차량 플랫폼 대상은 final stage source-confidential label과 원본 source 직접 복사 금지
architecture	target architecture에서 실행 가능한 package인지
ROS2 launch 참조	ros2 launch 대상 launch file이 image/package/shared install에서 해소되는지
root image bootstrap	Root와 공용 base가 검토한 공식 ROS Humble multi-arch index sha256:7bea3d9aa2483d3ca34c8e30d921b79273b0913bd7dc64bebf51d082b5d107e4 를 사용하고, root Docker apt key의 primary fingerprint가 9DC858229FC7DD38854AE2D88D81803C0EBFCD88 인지

검증 스크립트는 package/core/*, package/plugin/* 아래 manifest를 스캔하되 .gitmodules 에 등록되지 않은 nested git repo는 제외한다. self-hosted runner workspace에 남은 예전 checkout이나 로컬 백업 패키지가 release 검증을 오염시키지 않게 하기 위한 기준이다. Package test runner의 JSON report에는 package path, 기대 gitlink와 actual HEAD/dirty hash, suite command, 종료 상태와 제한된 stdout/stderr가 들어가며 PR trusted job이 artifact로 보존한다.

현재 runner는 suite가 없거나 package 시간 예산을 넘으면 실패한다. Python은 JUnit의 0건·실패·skip과 background thread·unraisable 오류를 검사한다. Node는 TAP summary가 있는 runner만 허용하고 0건·실패·skip·todo·cancel을 검사한다. build·lint만 실행하는 Node script와 fail-fast·동작 assertion이 없는 shell script는 test로 발견하지 않는다. trusted/release root gate도 JUnit skip을 실패시킨다. release self-hosted gate는 FURIVE_PACKAGE_NATIVE_FULL=1 로 CAN clean colcon/vcan의 필수 SocketCAN 10 case와 checksum 1 case를 skip 없이 실행하고 RS80 upstream native suite를 추가 실행한다. trusted job은 package JSON report, suite별 JUnit과 root pytest JUnit을 보존한다.

Release runner의 host 의존성은 CI가 sudo 로 설치하지 않는다. 러너 계정으로 다음 내부 setup을 한 번 실행하면 Ubuntu/architecture를 확인하고 이미 설치된 apt package는 건너뛰고, 빠진 ROS Humble·native build package만 한 번의 운영자 sudo 승인으로 설치한다. Python requirements와 공식 Node.js 22.23.2도 /opt/furive-release-runner 에 checksum으로 고정해 한 번만 설치하며 workflow 실행마다 setup-python / setup-node 와 pip 설치를 반복하지 않는다. CAN software test용 vcan0 는 systemd가 부팅 때 준비한다. 비밀번호를 GitHub Secret에 저장하지 않는다.

```

TERMINAL

bash scripts/release/setup-release-runner.sh
bash scripts/release/setup-release-runner.sh --check

```

Release source gate는 서로 독립된 package를 두 개씩 실행하고 private submodule checkout을 네 개씩 병렬화한다. Source 검증 job과 Harbor build job의 private hydration은 서로 다른 credential·실행 경계이므로 각각 fail-closed로

수행하지만, self-hosted runner의 content-addressed LFS cache를 공유해 동일 대용량 object를 다시 내려받지는 않는다. 동일 source SHA 재시도는 보존된 source-contract artifact를 검증해 전체 package/native gate를 다시 실행하지 않는다.

Workflow 내부 구현(사용자 기본 실행 경로가 아님):

```
python3 scripts/checks/check-doc-links.py
python3 scripts/checks/validate-package-manifests.py --strict-duplicate-names
python3 scripts/checks/run-package-tests.py --report ...
scripts/checks/verify-deploy-artifact.py --source-only ...
```

2. Deploy Artifact 검증

deploy repo export 후 차량에 배포할 형태가 맞는지 확인한다.

검증	확인 내용
필수 파일	<code>run.sh</code> , <code>install.sh</code> , <code>uninstall.sh</code> , <code>recovery/source-unlock</code> script
release metadata	<code>release/manifest.json</code> , package catalog/lock, asset metadata
metadata 일치	version, deploy branch, source ref, Harbor registry/project, architecture
package 배포 방식	source package와 image package 구분이 맞는지
image package	source/build payload가 deploy repo에 새지 않았는지
Docker image	package-lock에 기록된 image가 Harbor에 있고 arch가 맞는지
image 불변성	runtime image가 모두 <code>repository@sha256:... </code> 이고 digest lock/hash와 일치하는지
공급망 증적	SPDX 2.3 SBOM·SLSA provenance v1-index hash가 manifest/source ref와 immutable image subject에 정확히 결박됐는지
secret scan	private key, token, <code>.env</code> , credential file/content 포함 여부
recovery smoke	<code>recover-deploy.sh</code> dry-run 수준의 기본 동작

이 검사는 exact deploy metadata와 image digest가 서로 맞는지를 확인하지만 그 image 안의 서비스에 업무 입력을 넣어 결과를 확인하지 않는다. 따라서 lane 2 semantic smoke PASS로 승격하지 않는다.

Workflow 내부 구현(사용자 기본 실행 경로가 아님):

```
scripts/checks/verify-deploy-artifact.py --deploy-dir ... --require-supply-chain
```

3. OTA Upgrade Path 검증

deploy repo release를 원격에 공개하기 전에 partial clone을 재패킹하지 않고 현재 local staged checkout을 임시 local origin으로 노출해, staged ref와 LFS object를 대상으로 실제 updater의 fetch/detect/prepare 경로를 확인한다. 검증 후 origin URL은 즉시 복원한다. 모든 차량 release가 통과한 뒤에만 branch/tag 전체를 한 번의 atomic push로 공개한다.

검증	확인 내용
target release	target tag의 manifest version/branch/source ref
branch 연결성	target tag가 target deploy branch에서 도달 가능한지
이전 release	같은 branch의 이전 release 탐색
update 감지	이전 release 기준 updater가 새 release를 감지하는지
staging prepare	새 release를 staging 영역에 prepare할 수 있는지
candidate lifecycle	합성 stable/delayed-crash health로 confirm과 rollback 전이가 작동하는지

Workflow 내부 구현(사용자 기본 실행 경로가 아님):

```
scripts/checks/verify-deploy-ota-upgrade.py ...
```

이 단계는 격리 checkout에서 `actual_package_health: false` 인 합성 health를 사용하고 release job에서는 이미 검증한 digest의 중복 image pull을 생각한다. 실제 차량의 normal → test/preview → normal rollback, 실제 package health, Autoware launch/data flow를 검증했다는 뜻이 아니다. 그 검증은 lane 3의 **정책/필수** 후속이다.

v2026.08.12.19-preview-lotte-ohmio-1 의 updater는 signed annotated successor tag를 commit이 아닌 tag object로 해석하는 immutable 결함이 있어 다음 release의 OTA 시작점으로 사용할 수 없다. 이 exact release에서 후속 test release로 이동할 때는 자동 OTA 증적을 N/A 로 기록하고 새 credential bundle로 다시 설치한다. 명시적으로 이 release를 `--baseline` 으로 요구한 검증은 계속 실패하며, 다른 release를 넓게 예외 처리하지 않는다.

자동으로 만드는 산출물

산출물	예
base image	<code>\${HARBOR_REGISTRY}/\${HARBOR_PROJECT}/furive-os-base:<hash>-<arch></code>
base release alias	<code>\${HARBOR_REGISTRY}/\${HARBOR_PROJECT}/furive-os-base:<version>-<arch></code>
package image	<code>\${HARBOR_REGISTRY}/\${HARBOR_PROJECT}/<image_name>:<hash(package_commit,base_hash)>-<arch></code>
package release alias	<code>\${HARBOR_REGISTRY}/\${HARBOR_PROJECT}/<image_name>:<version>-<arch></code>
multi-arch image manifest	<code><image>:<version></code>
deploy repo branch	<code><deploy_branch></code>

산출물	예
deploy repo tag	<version>
release metadata	release/manifest.json, release/package-lock.yaml, release/package-catalog.yaml
image digest lock	release/image-digests.yaml 의 tag→digest 대응과 manifest 내 SHA-256 fingerprint
공급망 증거 index	release/supply-chain.json 과 manifest에 결박된 SHA-256
component SBOM	release/sbom.spdx.json (SPDX 2.3)
build provenance	release/provenance.intoto.json (in-toto Statement, SLSA provenance v1 predicate)

SBOM·provenance 보증 범위

scripts/release/generate-supply-chain-evidence.py 는 package-lock.yaml 의 Furive package/component와 image-digests.yaml 의 모든 immutable OCI image를 release subject로 만든다. Provenance에는 source commit, workflow ref, run ID, runner environment와 image digest가 들어간다. supply-chain.json 은 각 증거의 SHA-256을, manifest.json 은 다시 index의 SHA-256을 고정한다. Build job의 artifact 검증과 sign_and_publish job의 source-free stdlib validator가 descriptor hash, unique immutable image subject, SPDX documentDescribes·container download/checksum map과 in-toto/SLSA source·release·branch·architecture·workflow·runner·builder invocation을 다시 확인하므로 누락되거나 바뀐 증거는 게시하지 않는다. Plan의 repository/source SHA/run ID/run attempt도 protected handoff에 결박한다. 다만 run_attempt 는 provenance schema 자체에 아직 들어가지 않고 generator와 byte-for-byte deterministic export 비교도 하지 않는다. 따라서 “signer가 generator의 모든 의미를 독립 재현한다”고 주장하지 않는다.

이 SBOM은 release가 선택한 **component와 image digest 수준**이다. OCI image 항목은 filesAnalyzed: false 이고 index는 다음 범위를 명시한다.

JSON
<pre>{ "deep_image_file_inventory": false, "deep_image_inventory_owner": "registry-scanner" }</pre>

따라서 image 내부 OS package·language dependency·파일 목록과 CVE 판정은 현재 증거에 포함되지 않는다. Harbor scanner가 각 digest의 deep SBOM/SCA 결과를 만들고 severity 차단, 예외 승인, scanner DB 시각과 결과 보존을 release run에 연결해야 이 잔여 위험을 달을 수 있다. 현재 provenance도 pipeline이 자체 생성한 build 증거이며 독립 제3자 attestation이나 특정 SLSA level 인증을 뜻하지 않는다.

Source Unlock으로 차량에서 대체한 local checkout은 CI가 빌드·고정한 immutable subject가 아니다. Release metadata가 package 선택과 override 상태를 보고하더라도 그 local source 내용·build output은 signed release, SBOM과 provenance의 운영 보증 범위에서 제외한다. 그런 차량은 개발/진단 상태로 분류하고 공식 release 증거와 섞지 않는다.

차량용 Docker package는 정적 manifest 검사에서 multi-stage final image의 원본 source copy를 차단하고 io.furive.runtime.source-confidential=1 metadata를 요구한다. Release workflow는 각 package image를 실제 build한 뒤 같은 label을 local image에서 읽어 Harbor push 전에 확인한다. 이는 빌드 과정에서 source를 사용하되 최종 runtime image에는 실행 결과만 남기는 경계다. 현재 검사는 image의 모든 파일을 분류하는 deep inventory가 아니며 binary, bytecode와 JavaScript bundle의 역공학 가능성을 없애지 않는다. Source Unlock으로 명시적으로 제공하는 Git checkout은 이 검사 대상이 아니다.

자동으로 바꾸는 곳

대상	CI/CD 동작
Harbor	base image/package image push, release alias 생성
deploy repo	release 파일 export 후 branch push
deploy repo tag	새 revision tag 생성. 이미 존재하는 revision tag가 불일치하면 중단
source repo transition receipt	서명 완료 뒤 read-only validation job과 checkout 없는 최소권한 write job을 거쳐 one-time transition completion tag만 생성

source repo의 기능 코드는 CI/CD가 수정하지 않는다. `validate_transition_receipt` 는 source/deploy를 읽고 bounded receipt만 만들며 `contents: write` 가 없다. `record_transition` 만 source write token을 받고 checkout:repository module:deploy credential 없이 고정 completion tag 하나만 create-only 기록한다.

필요한 권한/입력

항목	쓰는 곳
<code>SOURCE_REPO_SSH_KEY</code>	고정 source tag:top-level private submodule-LFS hydrate step 전용; 이후 삭제
<code>DEPLOY_REPO_READ_KEY</code>	비신뢰 build job과 read-only <code>validate_transition_receipt</code> job의 deploy repo fetch; 두 job이 읽을 수 있는 repository-scoped Secret이며 <code>record_transition</code> 에는 전달하지 않음
<code>DEPLOY_REPO_SSH_KEY</code>	보호된 signing job의 exact lease/atomic push
고정 deploy remote	workflow에서 검토된 <code>git@github.com:Furive-ADS/furive-os-deploy.git</code> ; Variable/Secret override 금지
<code>HARBOR_USERNAME</code> , <code>HARBOR_PASSWORD</code>	Harbor login/push
<code>HARBOR_REGISTRY</code> , <code>HARBOR_PROJECT</code>	image push 대상
<code>FURIVE_OS_TARGET_ARCH</code>	기본 build architecture
<code>FURIVE_RELEASE_SIGNING_PRIVATE_KEY</code> , <code>FURIVE_RELEASE_SIGNING_CERTIFICATE</code>	production-signing job의 임시 온라인 서명자
<code>FURIVE_RELEASE_ROOT_PUBLIC_KEY</code> , <code>FURIVE_RELEASE_REVOCATION_KRL_B64</code>	signing job과 read-only <code>validate_transition_receipt</code> 의 root 승인·폐기 검증; 동일 bytes를 두 job이 읽도록 repository-scoped Secret으로 두고 같은 이름의 Environment shadow 금지. Source-write <code>record_transition</code> 에는 전달하지 않음

온라인 하위 서명키의 현재 운영 결정

게시자 서명의 첫 단계에서는 offline root가 승인하고 회전할 수 있는 온라인 subordinate release 서명키만 GitHub의 protected Environment Secret에 임시 보관한다. Workflow의 격리 서명 job과 차량의 pinned-root 검증 경로는 구현됐지만, 실제 offline root·온라인 인증서·운영키, production-signing Environment 보호 설정과 fleet trust-root 배포는 아직 구성 완료로 확인되지 않았다. 따라서 보안 보증 기준의 SEC-04는 계속 부분 충족이다.

항목	임시 계약
Environment	production-signing
Secret	FURIVE_RELEASE_SIGNING_PRIVATE_KEY
허용 키	폐기·회전할 수 있고 release 서명에만 쓰는 온라인 하위키
잔여 위험	HIGH — export 가능한 CI private key를 탈취하면 안전 삭제 증적 또는 KRL 폐기 전까지 정상 게시자로 위장 가능
금지	GitHub Actions vars, repository/organization-wide secret, offline root 개인키, base64를 암호화로 취급하는 것
검토 기한	2026-11-08 또는 첫 외부 production normal OTA/ISO readiness review 중 먼저 오는 시점
단일 정책 소유자	release identity/OTA 검증 계약. CI는 검증된 대상의 서명 동작만 수행

Self-hosted release job은 source, submodule, package build.sh와 container build를 실행한다. Source/submodule key는 private hydrate step, deploy read-only key는 remote fetch step에서만 사용하고 삭제해 이후 package build에 전달하지 않는다. Source hydrate step은 fixed PATH, 빈 shell/loader env, system/global Git config 차단, core.hooksPath=/dev/null, 빈 credential helper, local SSH/extraheader 제거와 clean submodule repository를 강제한다. 다만 같은 runner의 앞선 code가 background process나 오염 상태를 남길 수 있고 이후 Harbor push credential·Docker socket을 사용하므로, credential을 실행 package code와 완전히 격리한 ephemeral runner 증적은 아직 없어 HIGH 잔여 위험이다.

검증된 commit과 publication plan은 bounded Git bundle로 넘긴다. Secret이 없는 GitHub-hosted validate_publication job이 exact source policy를 먼저 검사하고, 별도 sign_and_publish job만 production-signing Environment와 push key를 받는다. Protected job에는 source checkout·repository Python/module-dependency install이 없고 inline stdlib가 identity, ref lease, manifest, SBOM와 provenance의 bounded 핵심을 다시 검사한다. Source transition receipt는 서명 뒤 contents: read 전용 validation과 checkout 없는 contents: write 전용 recording job으로 나눈다. 회귀 계약은 protected job 전체에 checkout, GITHUB_WORKSPACE, scripts/, sys.path, core import가 없고, 개인키가 build/receipt job에 없으며 exact ref lease로 원자 게시되는지 확인한다. 완전한 generator/validator 동등성은 별도 pinned validator 또는 KMS API와 mutation corpus로 달아야 한다.

과거 v2 서명 계약 이전에 설치된 unsigned main artifact에서 처음 한 차량 test로 넘어가는 경우만 기존 generation receipt 안의 내부 legacy_baseline_attestation을 사용한다. Generation receipt가 아직 하나도 없고 등록 차량 preview가 정확히 한 개일 때, production-signing required reviewer가 승인한 protected job이 main HEAD와 기존 canonical tag, manifest SHA, source commit/tag object, package·external·image lock SHA와 immutable image ID, OTA generation을 다시 확인한다. 확인한 digest는 preview manifest와 normal principal로 서명하는 generation tag message 양쪽에 결박한다. 이후 preview와 normal은 이 exact signed generation receipt만 재사용하며 unsigned 예외를 다시 열지 않는다. 이 절차는 기존 설치판의 identity를 1회 서명 증명하는 bootstrap attestation일 뿐 build 성공, 기능 동작 또는 실차 QA 증거가 아니다. Source와 deploy는 서로 다른 repository라 두 ref를 하나의 Git transaction으로 잠글 수 없으며, protected signer는 source tag를 게시 직전 다시 확인하고 deploy refs만 atomic lease한다. 따라서 source tag 보호 규칙과 workflow run의 수동 승인 기록도 release 증적에 반드시 포함한다.

`validate_transition_receipt`는 `production-signing` Environment와 source write token 없이 exact source SHA를 credential 비지속 방식으로 checkout하고, read-only deploy key와 root/KRL로 원격 signed receipt: branch head:canonical ancestry를 재검증한다. Root/KRL을 Environment에만 두면 이 job은 빈 값으로 실패한다. 실제 GitHub secret scope와 signing/validation job의 root/KRL fingerprint 일치하는 local checkout에서 확인할 수 없으므로 최초 release 전에 운영 증거로 남긴다.

별도 `record_transition`은 checkout, repository module, deploy key, root/KRL, signing private key, deploy write key와 protected Environment가 없다. 앞 job의 크기 제한 receipt를 Python stdlib로 다시 검사하고 exact repository/run/source SHA에 결박된 `legacy-transition-v2-published` tag만 GitHub API로 create-only 기록한다. 이 구조에서는 reviewed source/deploy read credential과 source `contents: write` token이 같은 job에 존재하지 않는다. Workflow/ruleset 변경 승인과 job output provenance는 별도 운영 증거로 확인한다.

첫 사용 전에 다음 조건을 모두 충족해야 한다.

1. build/검증 job과 `sign_and_publish` job의 분리를 유지한다. 후자는 검증된 deploy commit/tree와 release identity만 입력받고 source/submodule checkout, package code, dependency install 또는 build를 실행하지 않는다.
2. 서명 job은 `environment: production-signing`과 별도 ephemeral signing runner를 사용한다. Environment에는 required reviewer, self-review 금지, canonical release tag 제한과 최소 권한을 적용한다. 같은 self-hosted build runner 재사용은 격리로 인정하지 않는다.
3. 차량은 후보가 제공한 키가 아니라 trusted bootstrap이 host에 고정한 root/allowed signer로 서명을 staging 전에 fail-closed 검증한다. key ID, 용도, release 범위, artifact 보존 horizon과 폐기 상태도 서명 계약에 포함하고 activation 직전에 다시 검증한다. OpenSSH가 현재 시각으로 immutable tag를 검증하므로 artifact 인증서는 `-5m:forever` archival validity만 허용하며, protected signer가 유한 만료 인증서를 거부한다.
4. 서명은 모든 image digest와 metadata hash가 확정된 뒤, atomic publish 전에 수행하고 같은 job에서 즉시 검증한다. 개인키는 step-local mode 0600 임시 파일 외에 log, cache, artifact와 `$GITHUB_ENV`에 기록하지 않는다. `set -x`를 사용하지 않고 job 종료 후 runner 폐기 증거를 남긴다.
5. Environment 보호 설정, 승인자, workflow run ID, source/deploy commit, tag, artifact digest, signer key ID:public fingerprint와 서명 검증 결과를 release 증거로 보관한다. secret 값과 개인키는 어떤 증거에도 포함하지 않는다.

키 노출이나 비정상 서명이 의심되면 발행을 중단하고 online key를 폐기한 뒤 영향 release를 식별한다. 검증되지 않은 새 키로 자동 fallback하지 않으며 last-known-good 또는 trusted bootstrap 복구만 허용한다. offline root 개인키는 이 임시 경로를 포함해 어떠한 CI에도 넣지 않는다.

이 임시 키의 잔여 위험은 **HIGH**다. 종료 조건은 차량이 offline root 공개키를 고정하고 root가 승인한 자주 교체되는 online private key의 범위:archival certificate-폐기를 검증하며, 온라인 키를 KMS/HSM 안에서 non-exportable로 생성·사용하고 CI가 정확한 repository/workflow/ref/Environment에 결박된 GitHub OIDC 단기 identity로 서명 API만 호출하도록 전환하는 것이다. Offline root의 hardware-backed 보관·이중 통제와 build/sign 격리, 키 회전·폐기 복구 훈련까지 확인한 뒤 GitHub secret과 runner 잔존물을 삭제해야 export 가능한 온라인 키 부채가 닫힌다.

공통 release channel과 architecture, `normal_platform_targets` 승인 세대는 `config/deploy-targets.yaml`에 둔다. 차량별 profile/channel/release 할당은 `config/fleet-vehicles.yaml`, 고객/package 선택은 `config/capability-profiles.yaml`에서 관리한다. release artifact는 모든 profile에 필요한 package manifest/image lock을 포함하지만 차량 ID는 포함하지 않는다. 기본 release tag는 `main`의 `vYYYY.MM.DD.N` 하나다. main 병합 전 단일 차량 검증만 등록된 `preview/<vehicle_id>`와 `vYYYY.MM.DD.N-preview-<vehicle_id>`를 허용하며, CI는 다른 임의 suffix/branch를 무시한다.

`normal`은 플랫폼별 branch나 별도 artifact를 만들지 않는다. CI는 승인 세대의 모든 플랫폼을 `ota-platform-target-g<세대>-<release>-<플랫폼>-approved` 서명 tag로 한 atomic push에 포함한다. `release` 값은 해당 세대를 처음 게시한 공용판에 영구 결박되고, 같은 세대로 뒤의 공용판을 발행해도 움직이지 않는다. 한 플랫폼을 승격할 때는 세대를 정확히 1 올리고, 시험을 통과한 플랫폼은 `release`, 나머지는 마지막 exact release를 적으며 모든 capability profile을 다시 명시한다. 누락, 같은 세대의 상충 tag, 세대 replay, 서명:canonical release commit:capability 불일치는 발행과 차량 OTA 양쪽에서 fail closed한다. 최초 전환 공용판은 구형 updater가 전역 최신판으로 한 번 받을 수 있으므로 지원 플랫폼 전체에서 안전한 버전이어야 한다. 이 전환 뒤에는 각 플랫폼이 독립 승인 속도를 갖는다.

branch 확장 기준

현재 허용 범위는 공통 `main` 과 차량 목록에 등록된 임시 `preview/<vehicle_id>` 뿐이다. 임의의 고객사 branch나 차량별 영구 branch는 source/deploy 정합성 검사에서 거부한다.

차이는 다음 순서로 표현한다.

1. 설치 버전만 다르면 차량의 `release_pin` 을 바꾼다.
2. 기능이나 추가 package만 다르면 `capability`와 `package` 선택을 바꾼다.
3. 합치기 전 시험만 필요하면 `preview/<vehicle_id>` 를 만들고 시험 종료 후 삭제한다.
4. 위 세 방법으로 표현할 수 없고 독립 release 일정이 장기간 필요할 때만 고객사/제품 release channel을 추가한다.

새 장기 channel을 추가하려면 차량 연결, 허용 branch, 배포 대상, 담당자, 공통 수정·보안 패치 반영 방식, OTA upgrade/rollback 검증을 함께 정의해야 한다. 차량 ID 하나를 영구 branch 이름으로 쓰거나 임시 `preview` 를 승격하는 방식은 허용하지 않는다. 이 기준은 branch 수와 CI fan-out이 차량 수에 비례해 늘어나는 것을 막기 위한 운영 규칙이다.

`publish.py normal` 은 구형 차량용 전환판의 발행 상태를 자동 확인한다. 발행 전에는 설정된 과거 경로를 공용판과 한 번에 발행하고, CI가 모든 전환 artifact의 OTA 검증과 deploy repo 게시를 끝낸 뒤에만 내부 완료 tag를 생성한다. 실패한 run은 tag를 만들지 않으므로 다음 `normal` 발행이 자동 재시도한다. 완료 후에는 공용 `main` 하나만 빌드한다. 이 내부 tag는 release 버전이 아니며 차량 운영자가 선택하거나 이동하지 않는다.

새 호환판·설정 변환 계약을 기존 차량에 다시 전달해야 할 때는 이전 발행 receipt를 재사용하지 않는다. 현재 v2 전환은 별도 receipt를 사용하므로 첫 `furive release normal ... --run` 계획에 과거 여섯 경로가 다시 포함된다. 이 일회성 범위는 release 내부 계획과 CI가 소유하며 일반 `furive release` 요약에는 별도 사용자 개념으로 표시하지 않는다. 사람이 개입해야 하는 차단 상태만 오류로 보고한다. Receipt는 모든 역사 OTA 검증과 atomic deploy 게시 뒤 CI만 만든다. Receipt가 생겨도

`legacy_transition.enabled` 가 true이면 다음 `normal` 발행은 차단된다. 지원 대상 차량의 epoch2 수렴 또는 bootstrap 복구가 fleet inventory로 확인된 리뷰 변경에서만 이 값을 false로 바꾼다.

고객사·차량별 버전 상태 계약

모든 차량은 기본적으로 공통 `main` channel을 따라간다. 실제 SW 버전은 CI 성공 후 발행된 canonical immutable tag와 manifest/digest lock이다.

필드	기준
<code>customer</code> , <code>vehicle_id</code> , <code>capability_profile</code>	설치 시 생성된 차량 active config
<code>release_pin</code>	차량 active config의 선택적 목표 tag. 비어 있으면 <code>normal</code> 은 플랫폼 승인판, <code>test</code> 는 해당 차량 최신판
<code>channel</code> , <code>release_profile</code>	기본 <code>main</code> ; 임시 검증 때만 등록된 <code>preview/<vehicle_id></code>
<code>installed_release</code>	현재 checkout의 deploy release tag/manifest
<code>desired_release</code>	<code>release_pin</code> 이 있으면 고정 tag, 없으면 <code>normal</code> 은 설치 플랫폼의 최고 서명 승인 세대가 가리키는 release
<code>pending_release</code>	staging까지 준비된 다음 release
<code>rollback_release</code>	rollback pin 또는 last-known-good 이전 release
<code>state</code> , <code>drift_reasons</code>	current/update_available/pending/drift와 branch/profile/tag 불일치 원인

master와 agent는 이 값을 OTA status의 `vehicle_release` 객체로 내보낸다. agent heartbeat는 이를 master dashboard 상태로 전달하며, `scripts/ota-status.py --json` 도 동일 계약을 출력한다. 여러 고객사 네트워크를 한 화면에 모으는 중앙 수집기는 별도 운영 서비스의 책임이지만, 수집할 레코드의 의미와 필드는 이 계약으로 고정한다.

deploy 버전 검색 규칙

OTA updater는 원격 `furive-os-deploy` tag 가운데 현재 channel과 suffix가 일치하고 해당 원격 branch에서 도달 가능한 tag만 버전 목록에 넣는다. `normal` 은 설치 플랫폼의 최고 signed generation target이 가리키는 suffix 없는 canonical tag를 선택한다. `preview/<vehicle_id>` 는 정확히 같은 `-preview-<vehicle_id>` tag, 과거 전환 `deploy/<family>/<target>` 은 해당 target suffix tag만 허용한다.

허용 tag는 CalVer의 연·월·일·revision을 숫자로 비교한다. `release_pin` 이 없으면 `normal` 은 플랫폼 승인 tag의 release, `test` 는 해당 차량의 가장 큰 tag가 `desired_release` 가 된다. `release_pin` 이 있으면 형식, 원격 존재 여부, branch 연결성을 모두 검증하고 불일치 시 fallback하지 않는다. CI가 OTA와 artifact 검증을 끝내지 못한 release와 승인 세대는 deploy repo에 원자 게시되지 않으므로 차량 목표에도 노출되지 않는다.

일반 자동 업데이트는 `target > current` 일 때만 허용한다. updater는 HEAD에 tag가 여러 개 있어도 현재 channel과 맞는 tag 전체를 수집한 뒤 가장 높은 tag를 현재판으로 선택한다. 첫 원격 fetch 전에는 오래된 remote-tracking ref를 최신판으로 노출하지 않으며, `target <= current` 인 staging은 상태 응답에서 숨기고 update cycle·수동 적용·부팅 전환 시점에 폐기한다. 명시적 `release_pin` 또는 last-known-good rollback만 이 자동 하향 차단 예외다.

tag는 버전 검색 키이고 실제 내용 식별은 manifest와 image digest lock이 담당한다. 차량은 선택된 tag의 `release/manifest.json` , package lock, digest lock 무결성이 맞지 않으면 정상 release로 취급하지 않는다.

고정 GitHub Ed25519 host key와 signed tag는 server/publisher identity를 검증하지만 최신 release가 숨겨지지 않았다는 freshness 증명은 아니다. 현재 fresh bootstrap과 원격 목록에는 root-owned monotonic minimum 또는 외부 transparency ledger가 없어 오래된 정상 signed view의 freeze를 탐지하지 못한다. Fleet minimum을 운영 gate로 두고 차량의 monotonic floor·독립 ledger를 추가하기 전에는 rollback resistance가 완성됐다고 주장하지 않는다.

자동으로 하지 않는 것

[!WARNING] 별도 검증 필요

CI/CD는 deploy 가능한 release를 만드는 자동화다. 실제 차량/장치 동작까지 보장하지 않는다.

자동으로 안 하는 것	별도 확인 이유
실제 <code>install.sh</code> 전체 설치	host 변경과 package 선택이 환경마다 다름
장시간 <code>run.sh master/agent</code> 실행	runner가 차량 runtime 환경이 아님
RC exact artifact/image semantic smoke	현재는 artifact/digest 정합성과 합성 lifecycle만 검사함. 실제 service 입력·출력 probe가 필요
보호된 <code>normal</code> → <code>test/preview</code> → <code>normal</code> OTA/rollback	등록 시험 차량, 실제 package health와 release별 SHA/digest 수집이 필요

자동으로 안 하는 것	별도 확인 이유
Autoware revision 적용·원복 실증	플랫폼별 workspace root/하위 SHA, build/launch/ROS probe와 rollback 상태 수집이 필요
nightly/bench HIL	CAN·센서·GPU/display·NAS 장치와 장시간 장애 주입 환경이 필요
CAN/VCAN replay 비교	CAN interface, legacy log, UI 시나리오 필요
mgmt UI 버튼/화면 검증	Electron/브라우저/ROS2/CAN 연동 확인 필요
MGMT Node 22 packaged Electron canary	trusted/release CI는 clean <code>npm ci</code> , Node unit test, TypeScript와 Vite build, offline-startup shell을 실행한다. Electron 41.10.4/electron-builder 26.15.3 packaged artifact의 실제 기동은 별도 확인 필요
MGMT telemetry C++/Docker canary	receive-only fail-closed code와 JavaScript 계약은 검증했지만 ROS C++ compile-container 실행은 별도 확인 필요
센서 연결 검증	LiDAR/Camera/CAN 장치 필요
NAS upload/data logging 실동작	NAS credential/network/용량 조건 필요
host 영향 사전 점검	host 설정 변경은 문서/리뷰/벤치 확인 필요
image 내부 deep inventory·CVE 판정	component-level SBOM만 생성함. Harbor scanner의 digest별 결과와 release gate 연결 필요
완전 재현 가능한 base build	ROS base index는 digest snapshot이고 runtime Python은 Ubuntu package지만 apt repository snapshot·package version/hash와 모든 builder tool이 고정된 것은 아님
Source Unlock local build 보증	차량에서 바뀐 source와 build output은 CI가 만든 signed release subject가 아님
보안 기준 전체 검증	source/deploy 정적 계약, 서명 생성·검증 코드와 secret scan은 자동화했지만 실차 network·service·credential, trust-root/Environment 배포와 조직 증적은 별도 확인

위 항목은 [SW 운영/품질 기준 검증](#)에 따라 별도 증적을 남긴다.

운영자가 보면 되는 것

상황	보면 되는 것
release가 안 만들어짐	HEAD에 <code>v*</code> tag가 있는지, tag suffix와 deploy branch가 맞는지
submodule checkout 실패	submodule commit이 해당 submodule remote에 push되어 있는지
package 검증 실패	<code>package.yaml</code> 의 <code>name/type/nodes/run.script/build.script</code> , 중복 normalized package name, <code>.gitmodules</code> 등록 여부
image push 실패	Harbor credential/project 권한

상황	보면 되는 것
deploy 검증 실패	<code>release/manifest.json</code> , <code>package-lock.yaml</code> , image manifest, secret scan 결과
OTA 검증 실패	이전 release tag, target branch, target manifest

[!NOTE] 운영 원칙

deploy repo는 CI/CD가 만드는 산출물이다. 기능 수정은 source repo에서 하고, deploy repo에서는 직접 코드를 수정하지 않는다.

17. 보안·개인정보 기준

이 장의 목적 · 보안 검토자 · 고객 배포 담당자
코드가 강제하는 것과 운영 승인이 소유하는 것을
구분합니다.

완료 판단
이번 변경에 해당하는 보안·개인정보 항목만 실제 증거로
확인합니다.

이 문서는 접근 정보·권한·데이터·차량 제어·원격 업데이트·배포 서명을 바꾸거나 고객용 `normal` 을 준비할 때만 읽는 참고 자료입니다. 일반 기능 개발의 시작 문서가 아닙니다.

세 가지 공개 개념

개념	기본 동작
접근 제어	장비와 배포 버전을 확인하고 접근 정보는 필요한 프로그램에만 전달
데이터 사용	로컬 녹화·외부 전송·자동 삭제는 직접 커기 전까지 꺼지고 HMI는 기본적으로 같은 PC에서만 연결
안전한 업데이트	검증된 차량 버전만 준비하고 실패하면 이전 정상 버전으로 복구

인증서 형식, 내부 상태, 서명 파일과 호환 경로는 구현 세부입니다. 사용자 설정으로 다시 만들지 않습니다.

단계별 최소 기준

단계	필요한 것
일상 개발	실제 접근 정보·개인정보를 쓰지 않기, 변경 범위 테스트, 저장소 비밀값 검사
한 차량 <code>test</code>	바꾼 경계의 실제 동작과 복구, 실제 데이터를 쓴다면 담당자 승인
고객용 <code>normal</code>	<code>furive check</code> , 운영 접근 키 검토, 해당 데이터 흐름의 고객 배포 승인

ISO, ISMS, 자동차 규정이나 고객별 계약 증적은 적용 시장·법인 역할·계약이 정해진 뒤 선택합니다. 미래 인증 전체를 일상 개발의 완료 조건으로 사용하지 않습니다.

현재 강제하는 것과 강제하지 않는 것

구분	현재 기준
코드가 강제	main↔edge 장비 확인과 암호화 연결, 접근 정보 파일 보호, 녹화·외부 전송의 직접 활성화, 업데이트 내용·대상·복구 확인, Git 비밀값 검사

구분	현재 기준
운영자가 해야 함	실제 접근 키 발급·전달·교체, 고객 데이터 처리 승인, 접근 검토, 사고·백업·삭제 기록
아직 제공 안 함	법적 동의 자동 판정, 조직 인증, 원본 rosbag·영상의 저장 암호화, 모든 외부 프로그램·사용권 목록, 패키지별 완전 격리

기술적으로 기능을 켜 사실은 정보주체의 동의나 다른 법적 근거를 증명하지 않습니다. 코드 테스트 통과도 실제 운영 key와 GitHub 보호 설정이 배포됐다는 증거가 아닙니다.

추가 검토를 여는 변경

다음 중 하나를 바꿀 때만 이 문서의 관련 항목을 검토합니다.

- 접근 정보, 신뢰할 인증 기관, 원격 접근 또는 패키지 권한
- 녹화, 저장, 외부 전송, 지도·API·NAS 연계 또는 삭제
- CAN 송신과 다른 차량 제어
- 원격 업데이트, 배포 버전 서명, 설치·복구 또는 설정 변환
- 실제 차량에서 쓰는 외부 프로그램, Docker 이미지 또는 출처 기록

해당하지 않으면 **N/A** 한 줄이면 충분하며 새 설정, 상주 프로그램, 동의 처리 프로그램이나 상태 파일을 만들지 않습니다.

normal 발행 확인 항목

기존 문서와 검증 기록의 식별자는 다음 표 하나에서 유지합니다. 모든 변경에 열 항목을 전부 요구하지 않고 바뀐 경계와 배포 단계만 확인합니다.

ID	확인
SEC-01	credential을 source·log에 넣지 않고 필요한 구성요소에만 전달
SEC-02	로컬 녹화·외부 API·NAS 자동 전송·전송 후 삭제는 명시적 활성화 전까지 동작하지 않음
SEC-03	package와 host service가 필요한 권한만 사용
SEC-04	OTA 대상·내용·시작 확인·이전 정상 버전 복구가 일치
SEC-05	변경된 production dependency의 중요 취약점과 예외 확인
SEC-06	취약점을 비공개로 신고하고 대응할 경로 유지
SEC-07	main과 edge가 역할·차량에 맞는 서로 다른 장비 신원을 사용
SEC-08	실제 데이터의 목적·처리근거·수신자·보유·삭제를 고객 배포 전에 승인
SEC-09	release component와 고정 image의 생성·출처 증적을 번조 없이 결박
SEC-10	고객용 release 변경이 review와 제한된 CI 권한을 거침

SEC-08 은 조직의 고객 배포 승인자가 소유합니다. package나 Dashboard가 법적 승인 상태를 별도 config로 복제하지 않습니다.

개인정보와 차량 데이터

위치, 영상, 차량 ID, 주행 기록과 log가 사람·계정·고객과 연결될 수 있으면 고객 시험 전에 다음 다섯 가지를 한 장의 배포 기록에 남깁니다.

1. 목적과 필요한 데이터
2. 수신자와 처리 국가
3. 보유기간과 삭제 방법
4. 고지·동의 또는 다른 처리근거
5. 승인자와 변경일

현재 주요 경계는 다음과 같습니다.

기능	안전한 기본값	고객 배포에서 확인할 것
Data Logging-NAS	로컬 녹화, NAS 사용·자동 upload·upload 후 삭제 꺼짐. 전송에는 완전한 대상·credential·HTTPS·인증서 검증 필요	topic·영상 범위, local/NAS 보유·삭제, storage·backup 보호
Autonomy Studio	안전한 endpoint와 credential이 모두 없으면 위치 수집·전송·재전송 queue를 시작하지 않음	계정·위치·경로·event 수신자, 국가, 보유·철회
Dashboard 지도	build token이 없으면 외부 지도·검색 요청 없는 offline 화면	token이 있는 배포의 좌표·검색어 수신자와 고지
HMI 연동	관리 실행은 같은 PC의 고정 연결기만 사용하고, HMI 설정 변경은 외부 API·자동 upload를 켜지 않음	같은 PC의 다른 process를 아직 인증하지 않는 한계와 고객 HMI binary의 source 일치·외부 통신을 확인

사용자가 자신의 PC에 Data Logging을 설치하고 자신의 NAS만 대상으로 설정한 기본 경로에서는 Furive가 녹화 원본을 수신하거나 원격으로 읽지 않습니다. 이 경우 PC·NAS 계정, 공유 폴더, backup, 보유기간과 삭제의 운영 책임은 사용자 또는 고객 운영자에게 있습니다. Furive 원격지원이 원본에 접근하거나 Furive 운영 서버·제3자에게 전송하는 배포로 바뀌면 SEC-08 검토를 다시 열고 실제 접근권한·계약·운영 형태에 맞춰 개인정보 처리 역할을 판단합니다.

원본 dataset의 암호화, 법적 보존 중지와 안전한 파기 증적은 각 데이터 package와 고객 운영 환경이 소유합니다. 저장소의 기본값만으로 완료됐다고 주장하지 않습니다.

현재 알려진 우선 한계

- 실제 offline release root, 보호된 signing 환경과 fleet key 교체 증적은 저장소만으로 확인할 수 없습니다.
- main/edge와 여러 native package가 공유 runtime 계정과 Docker 권한을 사용하므로 package별 완전 격리가 아닙니다.
- 원본 rosbag·영상·센서 dataset은 application-level 저장 암호화를 기본 강제하지 않습니다.
- integration-gateway 의 loopback 연결은 차량 밖 접근을 막지만 같은 PC의 process identity를 인증하지 않습니다. HMI/DLS legacy adapter도 이 경계를 사용하며, 고객용 HMI binary가 검토한 source와 일치하는지는 release 과정에서 별도로 확인해야 합니다.
- component/image identity 증적은 image 내부 모든 OS·language dependency와 license 판정을 대신하지 않습니다.
- 실제 repository ruleset, reviewer, secret scanning과 운영 접근 기록은 별도 조직 증적입니다.

세부 공격 경로와 당시 조치는 [2026-08-10 적대적 검토](#), release workflow와 서명 운영은 [CI/CD 기준](#)이 소유합니다. 이 문서에 날짜별 조사 기록을 복사하지 않습니다.

단일 소유자

경계	소유자
원격 장비 신원·메시지 검증	<code>core/control_security.py</code>
package credential·권한 전달	<code>core/package_manager.py</code>
같은 PC의 package 요청·응답·이벤트 전달	<code>integration-gateway</code>
데이터 수집·보유·삭제	해당 데이터 package
OTA 정책·목표·준비·적용·복구	기술 구조의 하나의 OTA 흐름
고객 데이터의 법적·계약 승인	고객 배포 승인 기록

Dashboard는 상태와 입력을 전달할 뿐 위 정책의 별도 소유자가 아닙니다.

검증과 신고

공통 코드 계약은 다음 한 명령으로 확인합니다.

● ● ● TERMINAL

furive check

실제 차량에서는 바뀐 경계만 별도로 확인합니다. 설치·systemd·Docker·장치·network를 실제로 실행하지 않았다면 검증 기록에 그 사실을 남깁니다.

취약점, credential 노출이나 위치·영상 유출 가능성은 공개 issue에 재현 정보를 올리지 않고 [보안 정책](#)에 따라 비공개로 신고합니다.

TOPIC INDEX

용어로 찾아보기

페이지 번호는 이 PDF의 편집 페이지이며, 링크를 누르면 해당 계약의 시작 장으로 이동합니다.

Autoware 모델	87	Autoware revision	40	bundle	47
Dashboard	55	edge	16	health	77
host 영향	52	Integration Gateway	55	main	16
normal	47	OTA 복구	90	package.yaml	77
release	47	Source Unlock	77	test	47
검증 단계	44	로그	23	보안·개인정보	122
설치	22	플랫폼	40	CI/CD	98